Claude Code Tutorial

Complete Guide: 21 Articles

From first install to production AI systems: models, CLAUDE.md, hooks, skills, subagents, MCP, plugins, cost optimisation, and agent frameworks.

Source: https://blog.devops-monk.com/tutorials/claude-code/ (Devops Monk)

Compiled into a single document. Images and interactive elements from the web pages are not included; code blocks, tables, and lists are preserved.

Contents

Claude Models in 2026: Opus, Sonnet, and Haiku Compared **11**

The Current Model Lineup . . . 11
What Each Model Is For . . . 11
Choosing a Model: Decision Framework . . . 13
The Advisor Tool Pattern . . . 13
Prompt Caching: Making Any Model Cheaper . . . 14
Batch API: 50% Off for Non-Urgent Work . . . 14
Deprecation Timeline: What to Migrate Away From . . . 15
Extended and Adaptive Thinking . . . 15
Quick Reference . . . 16

Mastering Claude Code CLI: The Complete Guide for DevOps Engineers **17**

What Is Claude Code? . . . 17
Installation . . . 18
Three Ways to Talk to Claude Code . . . 18
Claude Code Modes . . . 19
CLAUDE.md — Your Project's Permanent Memory . . . 20
Saving Tokens — How to Keep Costs Low . . . 22
MCP — Connecting Claude to Everything . . . 23
Hooks — Automating the Boring Parts . . . 24
Permissions — Fewer Prompts, More Flow . . . 27
Essential Slash Commands . . . 29
The Desktop App . . . 30
IDE Integration . . . 31
Sessions, Context & Checkpoints . . . 31
Claude Code + GitHub . . . 34
DevOps-Specific Workflows . . . 37
Voice Dictation . . . 38
Output Styles . . . 39
Working Directories . . . 40
Status Line . . . 40
Tips Most Engineers Miss . . . 41
Putting It All Together: A Recommended Setup . . . 42

Writing a CLAUDE.md That Actually Works **44**

The Core Mental Model . . . 44
The Instruction Budget . . . 45
File Locations . . . 45
Structure That Works . . . 46
Output Styles: Teaching Claude How to Talk to You . . . 47
Before vs After: Real Examples . . . 48
Testing Your Rules . . . 50
Personal Global CLAUDE.md . . . 51
Imports: Keep CLAUDE.md Lean . . . 51
What to Include vs What to Skip . . . 52
CLAUDE.md vs Skills vs Hooks vs Subagents . . . 53
Path-Scoped Rules for Large Projects . . . 54
The Memory System . . . 55
Session Patterns That Compound Over Time . . . 58
Common Mistakes . . . 58
Quick Start . . . 59
A Note on Auto Memory . . . 60

Claude Code Hooks, Commands, Skills, and Subagents: The Complete Guide 62

Hooks: Deterministic Side Effects . . . 62
Custom Slash Commands: Reusable Prompts . . . 69
Skills: Context-Aware Automatic Instructions . . . 71
Subagents: Parallel Context Windows . . . 75
Piping: Claude in Your Shell Workflows . . . 80
When to Use Which: Decision Framework . . . 81
Getting Started Checklist . . . 82

Claude Code Skills: Building Specialized AI Workflows 84

What Is a Skill? . . . 84
Skill Structure . . . 84
Invoking Skills . . . 85
Real Skill Examples for Each Audience . . . 85
Steps . . . 86
Sharing Skills Across Projects . . . 90
Common Mistakes . . . 91

Claude Code Sub-Agents: Specialist Assistants and Delegation 92

Sub-Agents vs Skills vs Hooks . . . 92

How Sub-Agents Work . . . 92
Creating Sub-Agents . . . 93
Real Sub-Agent Examples . . . 94
Report format . . . 95
Code Review — [COMMIT_HASH] . . . 95
Analysis . . . 96
Report format . . . 96
Test Report — [DATE] . . . 96
Security checks . . . 97
Compliance checks . . . 98
Operational health . . . 98
Report format . . . 98
Infrastructure Audit Report — [DATE] . . . 98
Sub-Agent Memory . . . 99
Worktree Isolation . . . 99
Parallel Execution Patterns . . . 100
When to Use Sub-Agents . . . 100
Tips for Effective Sub-Agents . . . 100
Common Mistakes . . . 101
Sub-Agents in Teams . . . 101
Monitoring Agent Progress . . . 102

Claude Code Session Management: Parallel Work and Persistent Context 103

What Is a Session? . . . 103
Session Lifecycle . . . 103
Directory-Scoped Sessions . . . 104
Git Worktrees: Parallel Sessions . . . 104
Session Resumption and Forking . . . 105
Session Naming . . . 106
Five Developer Habits That Scale . . . 106
Parallel Workflows . . . 108
Session State and Permissions . . . 109
Session Storage and Cleanup . . . 110
Tips Most Engineers Miss . . . 110
Session Patterns for Teams . . . 111
Common Mistakes . . . 111
Advanced: Custom Session Configuration . . . 112

Claude Code Checkpoints: Your AI Safety Net for Code Changes **113**

What Are Checkpoints? . . . 113

How Checkpoints Work . . . 114

When Checkpoints Are Essential . . . 114

How to Rewind . . . 115

Critical Limitation: What Checkpoints DON'T Track . . . 116

Checkpoints vs Git: Complementary, Not Competitive . . . 116

Best Practices . . . 116

Common Mistakes . . . 117

Next Steps . . . 117

MCP Servers Worth Installing: For Developers, Testers, and DevOps Teams **118**

How MCP Fits Into Your Workflow . . . 118

For Developers . . . 118

For Testers . . . 121

For DevOps and Platform Engineers . . . 123

Role-Based Starter Stacks . . . 126

Servers to Skip (and Why) . . . 127

Project-Level vs Global Configuration . . . 127

Multi-MCP Workflow Scenarios . . . 128

The Evaluation Framework . . . 128

MCP in Claude Code: Connect to External Tools, Databases & APIs **130**

The Problem Without MCP . . . 130

What Is MCP? . . . 130

Three Perspectives on MCP . . . 131

MCP in Action: Three Real Examples . . . 131

Common MCP Scenarios & Outcomes . . . 133

MCP Tool Search & Context Window Efficiency . . . 134

Setting Up MCP for Your Team . . . 134

Real-World Workflow Examples . . . 135

Building Custom MCP Servers . . . 135

Comparison: MCP vs Traditional Integrations . . . 136

Security & Scoping . . . 136

Next Steps . . . 136

Resources . . . 136

Claude Code Plugins: The Complete Guide to Building and Sharing Extensions **138**

Plugins vs Standalone Configuration . . . 138
Plugin Structure . . . 138
The Plugin Manifest . . . 139
Building a Real Plugin: DevOps Helper . . . 140
3. Identify target environment . . . 141
4. Deploy . . . 141
5. Verify . . . 141
6. Report . . . 142
Incident Summary — [DATE] . . . 142
Testing Your Plugin . . . 145
Installing Plugins . . . 146
Converting Existing Config to a Plugin . . . 146
Publishing to the Anthropic Marketplace . . . 147
When to Build a Plugin . . . 147
Plugin Ideas to Build Next . . . 148
GitHub Actions Bot: Automated @claude Integration . . . 148

The Claude Code /loop Command: In-Session Automation Explained 156

What /loop Does . . . 156
When It Is Actually Useful . . . 157
/loop vs Cron: Choosing the Right Tool . . . 157
The /schedule Skill: Persistent Loops Beyond the Session . . . 158
Combining /loop with Hooks . . . 158
Practical Usage Tips . . . 159
What /loop Is Not . . . 159

Stop Burning Tokens: A Practical Guide to Claude Code Cost Optimization 160

Why Tokens Are Worth Caring About . . . 160
Habit 1: Trim Your CLAUDE.md . . . 160
Habit 2: Ask for Specific Lines, Not Whole Files . . . 161
Habit 3: Route Tasks to the Right Model . . . 162
Habit 4: Front-Load Your Prompts . . . 162
Habit 5: Scope Your Subagent Briefs . . . 163
Habit 6: Disable Unused MCP Servers . . . 163
Bonus: The Advisor Tool Pattern . . . 164
Tracking Your Usage . . . 164
Quick Reference . . . 164

Claude Prompt Caching: Cut Your API Costs by 90% 166

How It Works . . . 166
Basic Implementation . . . 167
Cache TTL: Ephemeral vs. Persistent . . . 168
Multi-Turn Conversations with Caching . . . 168
Automatic Caching . . . 169
Checking Cache Performance . . . 170
Why Your Cache Might Not Be Hitting . . . 170
DevOps Use Cases . . . 171
Combining Caching with the Batch API . . . 173
What Can and Cannot Be Cached . . . 174
Summary: When to Use Each Approach . . . 174

Voice Dictation in Claude Code: Speak Instead of Type 175

The /voice Command . . . 175
How Voice Dictation Works Behind the Scenes . . . 175
When Voice Dictation Shines . . . 176
Real-World Example . . . 176
Requirements . . . 176
Best Practices . . . 177
Cost Implications . . . 177
Troubleshooting . . . 177
Comparison: Voice vs Typed Prompts . . . 178
Workflow Tip: Hybrid Approach . . . 178
Future-Looking: Voice Features . . . 178
Next Steps . . . 179

Claude Extended and Adaptive Thinking: Making Claude Reason Before It Answers 180

Extended Thinking vs. Adaptive Thinking . . . 180
How Thinking Works Under the Hood . . . 181
Extended Thinking — Python . . . 181
Adaptive Thinking — Python . . . 183
Adaptive Thinking — TypeScript . . . 184
Multi-Turn Conversations with Thinking . . . 184
When to Enable Thinking . . . 185
Interleaved Thinking in Agentic Workflows . . . 186
Migrating from Opus 4.6 to Opus 4.7 . . . 187

Cost Estimation . . . 187
Quick Reference . . . 187

Claude Computer Use for DevOps: When to Use It and When to Use an API 189

How Computer Use Actually Works . . . 189
The Two-Layer Tool Model . . . 189
Practical DevOps Use Cases . . . 190
The Dispatch Pattern . . . 190
When Not to Use Computer Use . . . 191
Getting Started . . . 191
Claude Design: A Complementary Visual Tool . . . 191
Summary . . . 192

Claude Code as a Security Scanner: Beyond Pattern Matching 193

What Traditional Scanners Cannot Find . . . 193
How to Run a Security Audit . . . 194
The Audit Workflow . . . 194
What Claude Catches vs What It Misses . . . 195
Practical Tips . . . 195
Integration with CI . . . 196
Claude Code Security: The Managed Product . . . 196

Claude Managed Agents: Deploy AI Agents Without Managing Infrastructure 198

Managed Agents vs. the Agent SDK: What to Choose . . . 198
How It Works . . . 199
Creating Your First Agent Session . . . 199
Streaming Progress with Server-Sent Events . . . 200
Reconnecting to a Running Session . . . 201
Memory Across Sessions . . . 201
Scheduling Recurring Agent Runs . . . 202
DevOps Use Cases . . . 203
Pricing . . . 206
New in May 2026: Dreaming, Outcomes, and Multiagent Orchestration . . . 206
Limits and Constraints (Beta) . . . 208
The ant CLI . . . 209

Building a Personal AI Assistant with Claude Agent SDK and Bun 210

The Architecture . . . 210

Why Bun 210
Setting Up 211
The Agent Loop 211
Agent SDK V2: Simpler Multi-Turn Conversations 212
The Prompt Template 212
Memory: Avoiding Repetition 212
Delivery via Telegram 213
Scheduling with Cron 213
The Full Entry Point 213
Extending the Pattern 214
When to Use Managed Agents Instead 214

You Don't Need a Framework to Build an AI Assistant 216

The Architecture 216
The SOUL.md File 216
The Memory System 217
Prompt Templates 217
The Shell Script 218
Scheduling 219
Why This Beats a Framework 220
Extending It 220

Part 1: Foundations

Claude Models in 2026: Opus, Sonnet, and Haiku Compared

Published 2026-05-03 | Source: https://blog.devops-monk.com/2026/05/claude-models-guide-2026/

A practical guide to every current Claude model — specs, pricing, benchmarks, and a decision framework for choosing the right model for every task.

Picking the wrong Claude model is expensive. Opus on every task costs 5x more than Sonnet for comparable results on most work. Haiku on a complex reasoning task produces worse output than just asking Sonnet. And if you are still using models from early 2025, some of them are deprecated — or will be soon.

This guide covers every current Claude model, what each is good at, how much they cost, and a concrete decision framework for choosing the right one.

The Current Model Lineup

As of May 2026, Anthropic has three active model families: Opus (flagship), Sonnet (balanced), and Haiku (fastest). Each family has reached version 4.5 or higher.

Model	API ID	Context	Max Output	Speed	Input	Output
Claude Opus 4.7	`claude-opus-4-7`	1M tokens	128k tokens	Moderate	$5/MTok	$25/MTok
Claude Sonnet 4.6	`claude-sonnet-4-6`	1M tokens	64k tokens	Fast	$3/MTok	$15/MTok
Claude Haiku 4.5	`claude-haiku-4-5-20251001`	200k tokens	64k tokens	Very fast	$1/MTok	$5/MTok

MTok = per million tokens.

What Each Model Is For

Claude Opus 4.7 — Flagship Reasoning

Opus 4.7 is Anthropic's most capable model. It uses **adaptive thinking** — the model decides when a question requires deep reasoning and applies it automatically. You cannot manually control the thinking budget on Opus 4.7; it handles that decision internally.

Compared to its predecessor Opus 4.6, Anthropic describes Opus 4.7 as a "step-change improvement" specifically in agentic coding — multi-step tasks where the model has to plan, execute, verify, and loop.

Best for:

- Complex multi-file refactors where understanding the whole system matters
- Architecture decisions involving trade-offs and long-term consequences
- Agentic pipelines where the model runs autonomously over many steps

- Security audits requiring reasoning about business logic, not just pattern matching
- Any task where you have tried Sonnet and found the output quality lacking

Not worth it for:

- Standard implementation tasks (writing a function, adding a test, fixing a syntax error)
- Summarisation or reformatting work
- Simple lookups or one-question queries

Tokenizer Note: The Opus 4.7 tokenizer produces up to 35% more tokens from the same input text compared to Opus 4.6 (multiplier ranges 1.0x–1.35x depending on content type). Cost estimates based on Opus 4.6 usage will be understated when migrating — re-measure token counts on your specific inputs before committing to a budget.

Benchmarks:

- SWE-bench Verified: 80.8% (industry-leading code editing)
- OSWorld computer use: ~72.7%
- Knowledge cutoff: January 2026

Claude Sonnet 4.6 — The Everyday Default

Sonnet 4.6 is what most engineers should reach for by default. It supports both extended thinking and adaptive thinking, has a 1M token context window (the same as Opus), and costs 40% less. In developer surveys, 70% of engineers prefer Sonnet 4.6 over its predecessor Sonnet 4.5, and 59% prefer it over Opus 4.5 — meaning it punches well above what the price difference suggests.

The 1M token context window became generally available in March 2026, meaning you can feed an entire large codebase, a day’s worth of logs, or hundreds of documents into a single prompt without any special flags.

Best for:

- Daily coding work: implementing features, fixing bugs, writing tests
- Code review on pull requests
- DevOps automation: reviewing Terraform, generating Kubernetes manifests, debugging CI failures
- Long-context tasks: analysing large log files, reading entire codebases
- Any task where you want a capable model at a reasonable price

Benchmarks:

- SWE-bench Verified: 79.6% (only 1.2 percentage points behind Opus at one-third the cost)
- OSWorld computer use: 72.5%
- Knowledge cutoff: August 2025

Claude Haiku 4.5 — Speed and Scale

Haiku 4.5 runs at approximately 97 tokens per second — 83% faster than Sonnet 4.6 and 116% faster than Opus 4.6. At $1/$5 per million tokens, it is also five times cheaper than Sonnet and twenty-five times cheaper than Opus.

Despite being the “budget” option, Haiku 4.5 scores 73.3% on SWE-bench Verified. That is competitive with models that cost far more from other providers.

Best for:

- High-volume, low-latency tasks: processing thousands of files, batch classification, bulk code formatting
- Simple lookups: “what does this function return?”, “rename this variable”
- First-pass triage before escalating to a more capable model
- Cost-sensitive production applications where response speed matters
- The “executor” role in an Advisor Tool setup (more on that below)

Limitations:

- 200k token context only (vs 1M for Opus and Sonnet)
- Knowledge cutoff: February 2025 — over a year out of date
- Will miss nuance in complex reasoning tasks

Choosing a Model: Decision Framework

```
flowchart TD
    Q1{Is the task complex reasoning?\nArchitecture, multi-step agentic, security audit} -->|Yes|
Opus[Use Opus 4.7]
    Q1 -->|No| Q2{Is it standard dev work?\nImplementation, debugging, code review, DevOps}
    Q2 -->|Yes| Sonnet[Use Sonnet 4.6\nDefault choice]
    Q2 -->|No| Q3{Simple, fast, or high-volume?\nFormatting, lookups, summaries, bulk processing}
    Q3 -->|Yes| Haiku[Use Haiku 4.5]
    Q3 -->|No| Sonnet
```

The practical default: **start with Sonnet 4.6**. Upgrade to Opus only when you find Sonnet’s output quality is not good enough for the specific task. Downgrade to Haiku for tasks where speed or cost matters more than depth.

The Advisor Tool Pattern

One advanced pattern that dramatically reduces cost for long agent runs: pair a fast executor model with a high-intelligence advisor model. The advisor provides strategic guidance mid-run; the executor does the actual work.

```
import anthropic

client = anthropic.Anthropic()

response = client.messages.create(
    model="claude-haiku-4-5-20251001",   # fast executor
    max_tokens=4096,
    extra_headers={"anthropic-beta": "advisor-tool-2026-03-01"},
    tools=[
        {
            "type": "advisor",
            "advisor_model": "claude-opus-4-7",  # intelligent advisor
        }
    ],
    messages=[
        {
            "role": "user",
            "content": "Refactor our entire test suite to use the new async patterns"
        }
    ]
)
```

The Haiku executor handles the repetitive file editing and test running. When it encounters a decision point — which pattern to use, how to handle an edge case — it calls the Opus advisor. You get near-Opus quality at a fraction of the cost for long pipelines.

Prompt Caching: Making Any Model Cheaper

All three models support prompt caching, which stores a prefix of your prompt server-side and charges a fraction of the normal rate on repeated requests. A cached read costs 90% less than a fresh input.

```
import anthropic

client = anthropic.Anthropic()

response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    system=[
        {
            "type": "text",
            "text": open("large-codebase-context.txt").read(),
            "cache_control": {"type": "ephemeral"}  # cache this prefix
        }
    ],
    messages=[{"role": "user", "content": "Why is the auth service slow?"}]
)
```

The cache has a 5-minute TTL by default (1.25x write cost, 0.1x read cost). A 1-hour cache is available at 2x write cost, 0.1x read cost. For a codebase context you reuse many times in a session, this pays for itself quickly.

As of February 2026, **automatic caching** is available — add one `cache_control` field and Anthropic's infrastructure automatically advances the cache breakpoint as your context grows. No manual management required.

Batch API: 50% Off for Non-Urgent Work

When you do not need an immediate response, the Batch API cuts costs in half across all models. Jobs complete within 24 hours and can return up to 300,000 output tokens per request.

```
import anthropic

client = anthropic.Anthropic()

batch = client.messages.batches.create(
    requests=[
        {
            "custom_id": f"review-{i}",
            "params": {
                "model": "claude-sonnet-4-6",
                "max_tokens": 1024,
                "messages": [{"role": "user", "content": f"Review PR #{i}"}]
            }
        }
        for i in range(100)
    ]
)
print(f"Batch created: {batch.id}")
```

Combined prompt caching and batch processing can reduce costs by up to 95% versus standard per-request pricing.

Deprecation Timeline: What to Migrate Away From

If you are running any of these model IDs in production, migrate them now:

Model	Status	Deadline
`claude-sonnet-4`	Deprecated	Retires **June 15, 2026**
`claude-opus-4`	Deprecated	Retires **June 15, 2026**
`claude-opus-4-1`	Active but expensive	$15/$75 MTok — replace with Opus 4.7 at $5/$25
`claude-sonnet-4-5`	Active, 200k context only	Migrate to 4.6 for 1M context
`claude-3-7-sonnet`	Retired Feb 19, 2026	Already retired
`claude-haiku-3`	Retired Apr 20, 2026	Already retired
`claude-opus-3`	Retired Jan 5, 2026	Already retired

The migration from any retired `claude-opus-4` or `claude-sonnet-4` to the current generation models is straightforward — just change the model ID. No API changes required.

Extended and Adaptive Thinking

All three current models support some form of reasoning:

- Haiku 4.5

  : Extended thinking (manual budget)

- Sonnet 4.6

  : Extended thinking (manual budget) + adaptive thinking

- Opus 4.7

: Adaptive thinking only (

breaking change

— does not support manual

type: "enabled"

)

If you are migrating code from Opus 4.6 to Opus 4.7 and using extended thinking, you must switch from:

```
# Opus 4.6 — manual extended thinking
thinking={"type": "enabled", "budget_tokens": 10000}
```

to:

```
# Opus 4.7 — adaptive thinking
thinking={"type": "adaptive"},
effort="high"  # low, medium, high (default), max
```

Adaptive thinking means the model decides when to reason deeply. The `effort` parameter controls how selective it is — `max` applies reasoning to almost everything, `low` applies it sparingly.

Quick Reference

Use case	Recommended model	Why
Complex architecture review	Opus 4.7	Reasoning quality
Daily coding and debugging	Sonnet 4.6	Best balance
Large log file analysis	Sonnet 4.6	1M context
Long agentic pipelines	Opus 4.7 or Advisor pattern	Reliability
Bulk file processing	Haiku 4.5	Speed and cost
Security audit	Opus 4.7	Reasoning depth
Simple reformatting	Haiku 4.5	Overkill otherwise
Production app (latency-sensitive)	Haiku 4.5	~97 tokens/sec
Non-urgent batch jobs	Any + Batch API	50% discount
Cost-sensitive workloads with large context	Haiku 4.5	Lowest per-token cost

The model family is evolving fast. Anthropic releases new versions every few months, and each generation brings meaningful capability improvements at similar or lower prices than the generation before. The practical implication: when a new Sonnet or Haiku version ships, it is worth re-evaluating whether you still need Opus for tasks that previously required it.

Article 2 of 21

Mastering Claude Code CLI: The Complete Guide for DevOps Engineers

Published 2026-04-26 | Source: https://blog.devops-monk.com/2026/04/mastering-claude-code-cli/

A practical deep-dive into Claude Code CLI — covering MCP servers, saving tokens, CLAUDE.md files, hooks, permissions, and how to get the most out of it for real DevOps projects.

If you have been using Claude in a browser tab to help with code, you are leaving most of its capability on the table. Claude Code CLI brings the full power of Claude directly into your terminal — it reads your actual codebase, runs real commands, edits files, commits code, and integrates with every tool in your DevOps stack. This guide covers everything from installation to advanced patterns that most engineers never discover.

What Is Claude Code?

Claude Code is an **agentic coding assistant** that lives in your terminal, IDE, or browser. Unlike a chat interface where you paste code snippets, Claude Code:

- Reads your entire codebase
  — it understands the real structure of your project
- Runs actual commands
  —
  git
  ,
  docker
  ,
  kubectl
  ,
  terraform
  ,
  npm
  , whatever you need
- Edits files directly
  — makes changes across multiple files in a single operation
- Integrates with your IDE
  — VS Code and JetBrains show diffs in their native viewers

- Connects to external tools

  — databases, GitHub, AWS, Slack, and more via MCP

The key difference from a coding chatbot: Claude Code is not a passive assistant waiting for your next message. It is an active participant that takes actions, checks results, and iterates. AI can write code — but Claude Code can also follow your rules, protect your codebase, automate your workflows, and work as part of a team.

For DevOps engineers specifically, this means you can say “review our Terraform configs for security issues, fix what you find, and open a PR” — and it will actually do it.

Installation

macOS / Linux:

```
curl -fsSL https://claude.ai/install.sh | bash
```

Homebrew:

```
brew install --cask claude-code
```

Windows (PowerShell):

```
irm https://claude.ai/install.ps1 | iex
```

After installing, run `claude` in any project directory to start a session. Run `claude --help` to see all options.

Quick start:

```
cd my-project
claude                          # interactive session
claude "Fix the login bug"      # start with a specific task
claude -p "explain this project structure"   # one-off query and exit
claude --model opus             # start with a specific model
claude -c                       # continue your last session
```

Three Ways to Talk to Claude Code

Claude Code has three distinct types of input — knowing which to use saves time:

Type	When	Examples
CLI Commands	Before starting a session (in your terminal)	`claude`, `claude -p "query"`, `claude --model opus`
Slash Commands	During a session (inside the REPL)	`/clear`, `/compact`, `/model`, `/memory`
Special Notations	Shortcuts while typing a prompt	`@file.ts`, `!git status`, `# note`

Special Notations at a Glance

Notation	What it does
`@src/main.py`	Reference a specific file — Claude reads it immediately

Notation	What it does
`!git status`	Run a shell command directly without Claude's conversational processing
`# remember this`	Add a quick note to Claude's memory for this session

The `!` prefix (Bash Mode) is particularly useful: prefix any input with `!` to run a shell command straight to your terminal, bypassing the LLM entirely. No tokens consumed. Tab autocomplete works for commands and paths:

```
> !pwd
> !npm run test
> !ls -la
```

Claude Code Modes

Think of modes like gears in a car — you shift them based on the road. Each mode gives Claude a different level of freedom to act on your code. Switch between them anytime with **Shift+Tab**:

Default Mode → Auto-Accept Mode → Plan Mode → (back to Default)

Mode	What Claude can do	Best for
Default	Asks permission before every change	Careful, step-by-step work; production code; new to Claude Code
Auto-Accept	Edits files without asking	Fast, repetitive tasks; cleanup; following a well-defined plan
Plan	Read-only — cannot change anything	Research, planning, exploring before committing

The current mode is always visible at the bottom of the terminal.

Default Mode — The Safe Driver

This is the standard mode you start with. Claude asks for your confirmation before editing any file or running any command. Think of it as a co-pilot who always checks with you before taking the wheel:

```
Claude proposes a change → You review it → You approve or reject → Claude executes
```

Use Default Mode when working on production code, when you are new to Claude Code, or when dealing with sensitive files and configurations.

Auto-Accept Mode — The Fast Lane

Claude automatically applies file edits without waiting for your approval. The terminal shows "auto-accept edit on" when active.

Safety tip: Always commit your work before entering Auto-Accept Mode. If anything goes wrong, you can easily revert with `git checkout`.

Use it for: repetitive refactoring across multiple files, code cleanup tasks, following a plan you have already reviewed.

Plan Mode — The Architect

Claude enters a read-only state — it can look at your code but cannot change anything. Think of it as "Architect Mode": observe, analyse, plan, but never execute.

What Claude CAN do in Plan Mode:

- Read files and analyse code
- Search through your codebase
- Understand project structure and dependencies
- Build implementation strategies
- Ask you clarifying questions

What Claude CANNOT do in Plan Mode:

- Edit or create files
- Run Bash commands
- Install packages
- Make any modifications whatsoever

The recommended workflow:

1. Enter Plan Mode (Shift+Tab twice) — give your instruction:

   "I want to add OAuth2 authentication. Create a detailed plan."
2. Review and refine — ask follow-up questions until the plan looks right
3. Switch to Default or Auto-Accept — press Shift+Tab, say "Now implement this plan."

Why this matters: **Planning consumes fewer tokens than execution.** Do your thinking in Plan Mode, then execute efficiently.

CLAUDE.md — Your Project's Permanent Memory

Every time you start a Claude Code session, it reads `CLAUDE.md` files automatically. Think of it as a briefing document — you write it once and never have to re-explain your project setup.

Where to put them

File	Scope	Shared via Git?
`~/.claude/CLAUDE.md`	All your projects	No — personal preferences
`./CLAUDE.md`	This project	Yes — team-wide conventions
`./.claude/CLAUDE.md`	This project (alt)	Yes
`./CLAUDE.local.md`	This project	No — gitignored overrides

What to put in a project CLAUDE.md

The golden rule: **include what Claude cannot infer from reading the code**. Commands, gotchas, team conventions, and architecture decisions that are not obvious from the files.

```
# My Infrastructure Platform

## Build and Test
- Build: `npm run build`
- Tests: `npm test` — run single tests with `npm test -- --grep "auth"`
- Lint: `npm run lint`
- Type check: `npm run typecheck`

## Architecture
- API handlers in `src/api/handlers/`
- Terraform configs in `infra/`
- Each service has its own `Dockerfile.prod`

## Code Style
- ES modules only (import/export) — no CommonJS
- 2-space indentation
- camelCase for variables, PascalCase for types

## Git Workflow
- Feature branches: `feature/description`
- Always create a PR — never push directly to main

## Common Commands
- Deploy staging: `npm run deploy:staging`
- Deploy prod: `npm run deploy:prod`
- View production logs: `npm run logs:prod`
- Run migrations: `npm run db:migrate`

## Secrets
- Never hardcode credentials — all secrets in AWS Secrets Manager
- Use `.env.example` for documentation, never `.env` itself

## Known Gotchas
- The auth service requires `NODE_ENV=production` to initialise properly
- Database migrations run automatically on deploy — no manual step needed
- Some integration tests require a local Redis instance on port 6379
```

Path-scoped rules

For large projects, create `.claude/rules/` with topic-specific files that only load when Claude is working in the relevant area:

```
.claude/
├── CLAUDE.md          ← always loaded
└── rules/
    ├── api.md          ← loads when editing src/api/**
    ├── security.md     ← loads when editing any auth-related file
    └── infra.md        ← loads when editing infra/**
```

Example `.claude/rules/security.md`:

```
---
paths:
  - "src/api/**/*.ts"
  - "infra/**/*.tf"
---

# Security Rules

- Always validate and sanitise user input at API boundaries
- Use parameterised queries — never string interpolation in SQL
- Never log secrets, tokens, or PII — scrub before logging
- Return generic error messages externally — no stack traces
- Require authentication on all non-public endpoints
```

Keep CLAUDE.md under 200 lines. Longer files reduce adherence — Claude loses track of rules buried at line 350.

Saving Tokens — How to Keep Costs Low

Context is the most important resource to manage. Every message you send includes your entire conversation history, all the files Claude has read, and every command output. It adds up fast.

Check your usage

```
/cost      # estimated session cost
/usage     # detailed token breakdown
/context   # see what is consuming context space
```

Clear between tasks

The single most effective thing you can do:

```
/clear
```

When you finish a task and start something unrelated, clear the context. Stale conversation history about your auth bug wastes tokens on every message when you switch to asking about your CI pipeline.

Compact your context

```
/compact focus on infrastructure changes
```

`/compact` summarises the conversation history but lets you tell it what to preserve. Use it when a session gets long but you are not ready to clear completely.

Add a compact instruction to your CLAUDE.md:

```
## Compact instructions
When compacting, always preserve:
- Code changes made so far
- Test results and error messages
- Architecture decisions reached
```

Choose the right model for the task

```
/model claude-sonnet-4-6    # default — best balance of cost and capability
/model claude-opus-4-7      # for complex architectural decisions
/model claude-haiku-4-5     # for simple repetitive tasks
```

Sonnet handles 95% of DevOps tasks well. Save Opus 4.7 for genuinely complex problems — architecture decisions, security audits, and long agentic pipelines where reasoning depth matters.

Control extended thinking

Extended thinking is powerful but expensive. Turn it down for simple tasks:

```
/effort low     # minimal thinking — fast and cheap
/effort medium  # moderate reasoning
/effort high    # default — thinks when it helps
/effort max     # maximum thinking — best answer on hard problems
```

Use subagents for investigation

When you need Claude to explore a large codebase to answer a question, spawn a subagent instead of doing it in your main session:

```
Use a subagent to investigate why our authentication service is slow
```

Subagents run in their own context and report back a summary — keeping your main conversation clean.

MCP — Connecting Claude to Everything

MCP (Model Context Protocol) is what lets Claude Code talk to external systems — databases, GitHub, AWS, Slack, your custom APIs. Once configured, Claude can query them directly without you having to copy-paste data.

Adding MCP servers

```
# Add a GitHub MCP server
claude mcp add --transport http github https://api.github.com/mcp/ \
  --header "Authorization: Bearer YOUR_GITHUB_PAT"

# Add a local Postgres server
claude mcp add --transport stdio postgres \
  "mcp-postgres --connection-string postgresql://user:pass@localhost/db"
```

Or configure directly in `.claude/settings.json`:

```
{
  "mcp": {
    "servers": {
      "github": {
        "transport": "http",
        "url": "https://api.github.com/mcp/",
        "headers": {
          "Authorization": "Bearer YOUR_GITHUB_PAT"
        }
      },
      "postgres": {
        "transport": "stdio",
        "command": "mcp-postgres",
        "args": ["--connection-string", "postgresql://user:pass@localhost/db"]
      }
    }
  }
}
```

Useful MCP servers for DevOps

Server	What it enables
GitHub	Search PRs, read comments, create issues, review code diffs

Server	What it enables
Postgres / MySQL	Query databases directly, analyse schemas, debug data issues
AWS	List resources, check CloudWatch logs, manage S3
Web Fetch	Pull in documentation, error pages, API references
Filesystem	Read files outside your project root
Slack	Post notifications, read channel history
Memory	Persist information across sessions
Docker	Inspect containers, images, network config

Check what is loaded

```
/mcp    # list configured servers and their context cost
```

Disable servers you are not using — each one adds to your context overhead.

How MCP Manages Your Context Window

A common concern: "If I connect 10 MCP servers with 90 tools, won't that blow up my context?" The answer is no — MCP uses progressive loading.

At session start Claude fetches only the **tool names** from all connected servers, not the full schemas. Think chapter titles, not chapters. The complete JSON schema for a tool only loads when Claude actually needs it to handle your prompt.

Example: You have 10 MCP servers, 90 tools total.

- Your prompt says: "Check if the database connection is working"
- Claude scans the lightweight name list, identifies the Postgres tool
- Loads only that tool's full schema
- All other 89 tools stay as one-line name references

Result: **~95% less context consumption** compared to loading all schemas upfront. You can connect a large number of MCP servers without meaningfully affecting your available context.

Parallel MCP reconnection (May 2026)

When Claude Code re-initialises MCP servers — during subagent startup or SDK reconfiguration — servers now connect in parallel rather than serially. If you have six MCP servers configured, startup time drops from the sum of six serial connections to roughly the longest single connection. For setups with many MCP servers, this meaningfully reduces session startup latency.

Hooks — Automating the Boring Parts

Hooks are shell commands that run automatically at specific points in Claude's lifecycle. Unlike CLAUDE.md instructions (which are advisory), hooks are deterministic — they always run.

Hook events

Event	When it fires
`SessionStart`	At the beginning of every session
`UserPromptSubmit`	When you submit a prompt, before Claude begins processing — good for input sanitisation
`PreToolUse`	Before Claude runs any tool (can block the tool from running)
`PermissionRequest`	When Claude displays a permission request dialog
`PostToolUse`	After a tool succeeds — includes `duration_ms` (tool execution time)
`PostToolUseFailure`	After a tool fails — also includes `duration_ms`
`Notification`	When Claude Code sends a notification to the user
`SubagentStart`	When a subagent is created
`SubagentStop`	When a subagent completes its task
`Stop`	When Claude finishes generating its response
`TeammateIdle`	When an agent team member is about to become idle
`TaskCompleted`	When a task is marked completed
`ConfigChange`	When a config file is modified during the session
`WorktreeCreate`	When a Git worktree is created via `--worktree` or isolation settings
`WorktreeRemove`	When a worktree is removed (session ends or subagent finishes)
`PreCompact`	Before Claude performs context compaction
`SessionEnd`	When the session closes

Execution behaviour: All hooks matching the same event run in **parallel**, not sequentially. If multiple hooks contain the identical command, Claude Code automatically deduplicates — the command runs only once.

Creating hooks interactively: Type `/hooks` inside any session to open an interactive menu. You can add, configure, and delete hooks without editing JSON manually.

Choosing a Hook Language

Hooks run synchronously — Claude waits for them to finish before continuing. For high-frequency events like `PreToolUse` and `PostToolUse`, startup time accumulates fast:

Language	Startup time	Best for
Bash	~10–20ms	Simple tasks — string checks, file guards, quick env loads
Node.js	~50–100ms	High-frequency hooks (PreToolUse, PostToolUse) — worth the tradeoff for logic-heavy hooks
Python	~200–400ms	Less frequent hooks (SessionStart, SessionEnd) — or during debugging/exploration

A hook that takes 300ms on every `PostToolUse` call adds up to minutes of dead time in a long session. Use Bash or Node for anything that runs often; save Python for hooks that fire once.

Configuration

Hooks go in `.claude/settings.json`:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit",
        "hooks": [
          {
            "type": "command",
            "command": "~/.claude/hooks/format-on-save.sh"
          }
        ]
      }
    ]
  }
}
```

Practical hook examples

Auto-format files after Claude edits them:

```
#!/bin/bash
# ~/.claude/hooks/format-on-save.sh
input=$(cat)
files=$(echo "$input" | jq -r '.tool_output.files_edited[]' 2>/dev/null)

for file in $files; do
  case "$file" in
    *.ts|*.js) npx prettier --write "$file" 2>/dev/null ;;
    *.py)      black "$file" 2>/dev/null ;;
    *.go)      gofmt -w "$file" 2>/dev/null ;;
    *.tf)      terraform fmt "$file" 2>/dev/null ;;
  esac
done

exit 0
```

Block destructive commands:

```
#!/bin/bash
# ~/.claude/hooks/block-destructive.sh
input=$(cat)
cmd=$(echo "$input" | jq -r '.tool_input.command')

BLOCKED="rm -rf|kubectl delete|terraform destroy|DROP TABLE|truncate"

if echo "$cmd" | grep -qE "$BLOCKED"; then
  jq -n '{
    hookSpecificOutput: {
      hookEventName: "PreToolUse",
      permissionDecision: "deny",
      permissionDecisionReason: "Destructive command blocked by hook. Review and run manually."
    }
  }'
else
  exit 0
fi
```

Load environment variables at session start:

```
{
  "env": {
    "NODE_ENV": "development",
    "DEBUG": "app:*",
    "POSTGRES_HOST": "localhost",
    "AWS_REGION": "us-east-1"
  }
}
```

Or use a SessionStart hook to load from a `.envrc`:

```
#!/bin/bash
# ~/.claude/hooks/load-env.sh
[ -f .envrc ] && source .envrc
exit 0
```

Permissions — Fewer Prompts, More Flow

By default, Claude asks for permission before running commands or editing files. Here is how to reduce that friction without giving up safety.

The Three Permission Guards

Every permission in Claude Code falls into one of three categories — Allow, Ask, or Deny:

- Allow

  — Claude can do this without prompting you

- Ask

  — Claude pauses and asks for confirmation before proceeding

- Deny

  — Claude cannot do this, period

These map directly to the `allow` and `deny` arrays in settings, with the default being Ask for anything not listed.

Permission modes

Press `Shift+Tab` to cycle through Default → Auto-Accept → Plan modes. You can also set a mode at launch without changing persistent settings:

```
claude --permission-mode acceptEdits    # fast lane for this session only
claude --permission-mode bypassPermissions  # skip all prompts (containers/VMs only)
```

All five modes:

Mode	Behaviour	When to use
`default`	Asks permission on first use of each tool	Daily work, careful changes
`acceptEdits`	Auto-approves all file edits for the session	Repetitive refactoring, cleanup
`plan`	Can read/analyse but cannot modify anything	Research, planning phase

Mode	Behaviour	When to use
`dontAsk`	Denies everything unless explicitly pre-approved	Locked-down environments
`bypassPermissions`	Skips all prompts	Containers/VMs only — dangerous on local

Or set a persistent default in `.claude/settings.json`:

```
{
  "permissions": {
    "defaultMode": "acceptEdits"
  }
}
```

Settings scopes hierarchy

Settings cascade through five levels — more specific always wins:

Scope	File	Who controls it
Managed (highest)	`managed-settings.json` in Claude Code install	System admin — cannot be overridden by anyone
CLI Args	Session flags only	You, per session
Local	`.claude/settings.local.json`	You, per project (gitignored)
Project	`.claude/settings.json`	Team, committed to git
User (lowest)	`~/.claude/settings.json`	You, global defaults

If a permission is allowed in your user settings but blocked in the project settings, the project rule wins — it is more specific.

Use the `/status` **command** to see exactly which layer is controlling any setting and debug unexpected behaviour.

Company-wide announcements

Enterprise teams can broadcast rules to every Claude session by adding `companyAnnouncements` to the managed settings:

```
{
  "companyAnnouncements": [
    "Never commit secrets, tokens, or credentials to the repository.",
    "All production changes require a PR — never push directly to main.",
    "Write meaningful commit messages — future teammates will thank you."
  ]
}
```

These strings are shown to Claude at session start, regardless of what project CLAUDE.md says.

Allowlist common operations

Stop being asked about things you always approve:

```
{
  "permissions": {
    "allow": [
      "Bash(npm run *)",
      "Bash(git add *)",
      "Bash(git commit *)",
      "Bash(git status)",
      "Bash(git diff *)",
      "Bash(docker build *)",
      "Bash(docker ps)",
      "Bash(kubectl get *)",
      "Bash(terraform plan *)",
      "Bash(terraform init)",
      "Bash(terraform fmt *)",
      "Edit",
      "WebFetch(domain:github.com)",
      "WebFetch(domain:docs.aws.amazon.com)"
    ],
    "deny": [
      "Bash(terraform apply)",
      "Bash(terraform destroy *)",
      "Bash(kubectl delete *)",
      "Bash(rm -rf *)",
      "Read(.env)"
    ]
  }
}
```

Separate team and personal settings

Put shared permissions in `.claude/settings.json` (committed to git) and personal overrides in `.claude/settings.local.json` (gitignored):

```
// .claude/settings.local.json (your personal overrides)
{
  "permissions": {
    "allow": [
      "Bash(brew *)",
      "Bash(nvim *)"
    ]
  }
}
```

Essential Slash Commands

Session & context:

Command	What it does
`/clear`	Reset conversation history — do this between tasks
`/compact [focus]`	Summarise history, optionally preserving specific areas
`/cost`	Show estimated session cost
`/usage`	Detailed token usage breakdown
`/context`	Show what is taking up context space
`/rename [name]`	Name the current session for easy resumption
`/resume [name]`	Resume a named session
`/rewind`	Open checkpoint menu (double-tap Esc also works)

Command	What it does
`/btw [question]`	Ask a quick side question that never enters history

Configuration & settings:

Command	What it does
`/config`	Open interactive settings UI — one-stop shop for all Claude Code settings
`/status`	Show all active settings and which layer (managed/project/user) controls each
`/permissions`	Interactive UI for managing allow/deny/ask rules in real time, no restart needed
`/output-style`	Switch response style: Default / Explanatory / Learning
`/voice`	Push-to-talk dictation — hold Space, speak, release

Models & tools:

Command	What it does
`/model [name]`	Switch Claude model mid-session
`/effort [low|high|xhigh]`	Control extended thinking
`/mcp`	Manage and check MCP servers
`/add-dir [path]`	Add an additional directory to Claude's working scope

Project & memory:

Command	What it does
`/init`	Generate a starter CLAUDE.md for the current project
`/memory`	View and edit CLAUDE.md and auto memory
`/hooks`	View configured hooks

Code quality:

Command	What it does
`/review`	Run a code review subagent
`/ultrareview`	Multi-agent cloud review — finds and verifies real bugs before you merge (costs $5–$20 per run)
`/statusline`	Generate a status bar script and configure it automatically
`/doctor`	Diagnose installation issues

The Desktop App

Alongside the CLI, Claude Code has a redesigned desktop application (Mac and Windows) that adds capabilities the terminal interface cannot match:

- Session sidebar

  — all your open and recent sessions visible at a glance; switch between them without losing context

- Drag-and-drop workspace

— drag files and images directly into the conversation

- Integrated terminal and file editor

  — edit files and run commands without switching windows

- Faster diffs

  — file change proposals render immediately with improved performance

- Expanded previews

  — see rendered HTML, SVG, and markdown output inline

- Parallel task panel

  — run multiple independent tasks simultaneously and monitor each

The desktop app runs the same underlying model and tools as the CLI. It is particularly useful for visual tasks (computer use, screenshot analysis, design review) and for engineers who prefer not working in a terminal.

Download it from claude.ai/download. The CLI and desktop app share session history and CLAUDE.md files.

IDE Integration

VS Code

Install the **Claude Code** extension from the marketplace, or:

```
code --install-extension anthropic.claude-code
```

Useful keyboard shortcuts:

Action	Shortcut
Focus Claude input	`Cmd+Esc`
Insert file reference	`Option+K`
New conversation	`Cmd+N`
Open in new tab	`Cmd+Shift+Esc`

When Claude proposes file changes, VS Code shows them in its native diff viewer — you can approve, reject, or edit each change individually before accepting.

JetBrains (IntelliJ, PyCharm, GoLand, WebStorm)

Install **Claude Code** from Settings → Plugins → Marketplace.

Connect your terminal session to the IDE:

```
claude
/ide
```

This links the terminal session to your open IDE so diffs appear there instead of in the terminal.

Sessions, Context & Checkpoints

The Context Window

Every Claude Code session has a finite context window — the “invisible fuel tank.” Here is how the 1 million token window is distributed:

Component	Approximate size
System prompt (Claude Code instructions)	~100K tokens
Conversation history	~400K tokens
Tool call results	~50K tokens
Skills (loaded on demand)	~50K tokens
Available to work	**~400K tokens**

When context fills up, Claude first removes older tool outputs, then summarises parts of the conversation. Use `/context` to see exactly what is consuming space.

```
/cost      # estimated session cost
/context   # see what is consuming context space
/compact focus on infrastructure changes   # summarise history, preserve what matters
/clear     # reset entirely — do this between unrelated tasks
```

Advanced pattern for very long tasks: have Claude write its progress into a `.md` file, run `/clear` to start completely fresh, then tell Claude to read that file. You get preserved knowledge with a clean context.

Sessions Are Directory-Scoped, Not Branch-Scoped

Each Claude Code session is tied to your current **directory**, not to a git branch. When you switch branches, the same session continues — Claude reads files from the newly checked-out branch, but the conversation history stays unchanged.

This means Claude still remembers everything you discussed, even though the underlying files are now different. If you need **separate Claude sessions per branch**, use git worktrees — they create separate folders, and each folder gets its own independent Claude session.

Resuming and Forking Sessions

Resume your last session or a named one:

```
claude -c                           # resume most recent session
claude --resume auth-implementation   # resume by name
```

When you resume, you get the full conversation history back. Session-scoped permissions reset — you will need to re-approve them, like access badges expiring when you reopen the building.

Fork a session to try a different approach without losing the original:

```
claude --continue --fork-session
```

This copies the entire conversation history into a new session. The original stays untouched. It is like photocopying your notebook and trying different approaches on the copy.

	Resume	Fork
Session ID	Same	New
Behaviour	Continues from where you left off	Copies history, branches off

	Resume	Fork
Original session	Gets new messages added	Stays unchanged
Use when	Continuing the same work	Experimenting with a different approach

Smart Session Habits

These five habits separate engineers who get consistent results from those who fight Claude:

1. Clear Between Tasks Run `/clear` after finishing one task before starting the next. Bug-fixing conversations pollute feature work — every leftover token is wasted context and a potential source of confusion.

2. Compact at Natural Breakpoints Don't wait for autocompact to kick in at 80–85% context usage. Run `/compact` when *you* decide — after finishing a subtask, before starting a new phase. Add a focus instruction so Claude knows what to preserve:

```
/compact Keep only the test results and the current implementation plan
```

3. Name Your Sessions Type `/rename payment-gateway-fix` as soon as you start meaningful work. Future you will thank present you when staring at a list of 50 unnamed sessions and trying to find the right one to resume.

4. Monitor Your Context Run `/context` periodically — think of it as checking your fuel gauge. If you are above 60% and still have significant work ahead, compact or clear. The status line can show context percentage at all times so you never have to ask.

5. The Document & Clear Pattern For very large tasks that would fill the entire context window: ask Claude to write its plan and progress into a `.md` file, then run `/clear` and start a fresh session telling Claude to read that file and continue. You get a clean context window with fully preserved knowledge.

Forking Sessions with Worktrees

For parallel work on multiple branches simultaneously, use the `--worktree` flag (or its shorthand `-w`):

```
claude --worktree feature-auth     # creates worktree + branch, opens isolated session
claude --worktree bugfix-api       # second independent session
```

If you omit the name, Claude generates a random one. What happens under the hood:

- A new folder is created at

  <repo>/.claude/worktrees/<name>

- A new branch is created automatically:

  worktree-<name>

- Claude opens inside that isolated workspace with its own session

Each worktree has its own files and its own branch, but shares the same Git history and remote. Parallel edits never conflict.

Creating a worktree mid-session — you don't need to restart. Just say:

- “Start a worktree”
- “Work in a worktree”

Claude creates one automatically and moves your work into it.

Checkpoints: Ctrl+Z on Steroids

Every prompt you submit triggers a silent snapshot of your code state before Claude touches anything. These checkpoints are automatic — you do not press a save button. They **persist across sessions** (you can close the terminal and still rewind later) and expire after **30 days**.

Open the checkpoint menu by double-tapping **Esc** or running `/rewind`. You see a list of checkpoints — pick one and choose how to restore:

Option	What it does
Restore Code Only	Reverts file changes, keeps the conversation — good when code went wrong but context is still useful
Restore Conversation Only	Rewinds the chat, keeps the code — good when you want to re-prompt differently without changing files
Restore Both	Full reset — code and conversation go back together
Summarize from here	Condenses messages from that checkpoint forward without restoring anything — surgical alternative to `/compact`

What checkpoints do NOT track:

- Files modified by bash commands (

  rm

  ,

  mv

  , shell scripts) — only Claude’s direct Edit/Write tool calls are captured
- Manual changes you made in your own editor outside Claude
- After 30 days — use Git for permanent history

Best practice: checkpoints cover the session; Git covers everything else. Before any large task, commit first:

```
git checkout -b feature/refactor-auth
git add . && git commit -m "before AI refactor"
```

If everything breaks, `git reset --hard` brings you back instantly.

Claude Code + GitHub

Claude Code integrates with GitHub at three levels — pick the right one for your workflow:

Level	What it means	Best for
Terminal	Run Claude locally with `gh` CLI and GitHub MCP	Pair programming, daily dev work

Level	What it means	Best for
GitHub Actions (@claude bot)	Tag `@claude` on any issue or PR comment; bot runs Claude automatically	Code review, bug fixes, async team workflows
Headless / Scripting	`claude -p "..."` flag in CI/CD pipelines, cron jobs, scripts	Automated code review, nightly audits

Natural Language Git Commands

The fastest way to do git work is to just describe it:

```
# Instead of memorising git commands:
"commit and push with a descriptive message"
"create a feature branch called feature/auth-refresh and switch to it"
"push this branch and create a PR with a proper description"
"look at issue #42 and fix it"
"stage only the test files and commit them separately"
```

Claude translates these into actual `git` and `gh` commands, runs them, and reports back. It reads your diff, writes a meaningful commit message, and handles the mechanics.

How Claude Creates a PR (Behind the Scenes)

When you ask Claude to open a PR, here is the 4-step sequence it runs automatically:

1 Gather context

— runs

git status

,

git diff

,

git log

, and

git diff main..HEAD

to understand every change

2 Prepare the branch

— checks that you are on a feature branch (creates one if needed) and pushes to remote

3 Analyse changes

— reads the diff, identifies the purpose of each change, groups related changes

4 Create the PR

— runs

gh pr create

with a generated title, structured description (summary + details + test plan), and appropriate labels

The result is a PR that reads like a human wrote it — because it is based on actual understanding of the change, not just the commit message.

commit-commands Plugin

The `commit-commands` plugin adds three high-signal commands:

Command	What it does
`/commit`	Stages changed files and writes a smart commit message based on the diff
`/commit-push-pr`	Commits, pushes the branch, and opens a PR — all in one step
`/clean_gone`	Removes local branches that have been deleted on the remote (post-merge cleanup)

Install the plugin once and use these commands across every project.

Automated PR Creation

After making changes, instead of switching to GitHub:

```
claude "create a PR for these changes with a detailed description"
```

Claude runs the full 4-step sequence above and opens the PR with description, test plan, and reviewers if you have them configured.

Pipe GitHub Data Directly

```
# Review a PR diff
gh pr diff 42 | claude -p "review this PR for bugs, security issues, and best practices"

# Summarise recent issues
gh issue list --limit 20 | claude -p "what are the most common pain points here?"

# Check failed CI runs
gh run list --status failure --limit 5 | claude -p "what patterns do you see in these failures?"
```

Setting Up the @claude Bot

The fastest path is a single slash command:

```
/install-github-app
```

What it does step by step:

1. Installs the Claude GitHub App from

   github.com/apps/claude

   to your repo
2. Creates a PR containing

   .github/workflows/claude.yml
3. Before merging that PR, add

   ANTHROPIC_API_KEY

   as a GitHub Secret:

Settings → Secrets and variables → Actions

4 Merge the PR — setup complete

Test it immediately: go to any PR or issue and type `@claude review this PR` or `@claude fix this bug`. The bot responds with comments, code changes, or a new PR — and shows real-time progress via visual checkboxes that update as it works.

The @claude Bot on GitHub Actions

Tag `@claude` in any issue comment or PR review, and the bot responds:

```
@claude please review this PR for security issues
@claude this test is failing — can you fix it and push the fix?
@claude look at issue #234 and implement the described feature
```

The bot runs Claude in a GitHub Actions environment, reads the full issue/PR context, and either comments with an analysis or opens a commit/PR with a fix. Useful for async team workflows where you want AI involvement without blocking your own terminal session.

CI/CD Integration (Headless Mode)

```
name: Claude Code Review
on: [pull_request]

jobs:
  review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: AI Code Review
        run: |
          claude -p "Review the changes in this PR for bugs, security issues, and best practices. Be
specific and include file:line references." \
            --output-format json > review.json
        env:
          ANTHROPIC_API_KEY: ${{ secrets.ANTHROPIC_API_KEY }}
```

The `-p` flag runs Claude non-interactively — perfect for CI. Combine with `--output-format json` for structured output you can parse downstream.

DevOps-Specific Workflows

Terraform

```
# Review configs for issues
claude "review infra/main.tf for security issues and best practices"

# Plan changes safely
claude "run terraform plan and explain what would change"

# Generate module documentation
claude "generate README documentation for our terraform modules"
```

Restrict permissions so Claude can plan but never apply:

```
{
  "permissions": {
    "allow": ["Bash(terraform plan *)", "Bash(terraform init)", "Bash(terraform fmt *)"],
    "deny": ["Bash(terraform apply)", "Bash(terraform destroy *)"]
  }
}
```

Kubernetes

```
# Review manifests
claude "review all manifests in k8s/ for security and best practices"

# Generate a Helm chart
claude "create a Helm chart for our Node.js API service"

# Debug a failing deployment
cat deployment-error.log | claude -p "what is causing this failure and how do I fix it?"
```

CI/CD with GitHub Actions

```
name: Claude Code Review
on: [pull_request]

jobs:
  review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: AI Code Review
        run: |
          claude -p "Review the changes in this PR for bugs, security issues, and best practices. Be
specific." \
            --output-format json > review.json
        env:
          ANTHROPIC_API_KEY: ${{ secrets.ANTHROPIC_API_KEY }}
```

Pipe data directly

```
# Debug log output
cat app.log | claude -p "what errors are here and what is causing them?"

# Review recent git history
git log -20 --oneline | claude -p "summarise what changed recently"

# Analyse docker logs
docker logs myapp --tail 100 | claude -p "what is wrong with this service?"
```

Voice Dictation

Claude Code has a built-in voice interface — no third-party app required.

How to use:

1 Press

/voice

to start a voice session

2 Hold

Space

to record, release to submit

3 Claude transcribes and responds in text

Important details:

- Cloud-powered (not on-device) — audio is sent to a transcription service
- Free

  — voice transcription does not consume your API tokens or add to session cost
- Requires a

  Claude.ai login

  (not just an API key)
- Not available

  with raw API keys, AWS Bedrock, or Google Vertex — only with the Claude.ai-connected CLI

Voice is particularly useful when your hands are busy (reviewing logs on one screen, talking through an approach) or when you think faster by speaking than typing.

Output Styles

Use `/output-style` to change how Claude structures its responses for the rest of the session:

Style	What it does	Best for
Default	Concise, direct answers — just the result	Production work, experienced users
Explanatory	Adds "Insight" blocks after each section explaining *why* a choice was made	Code review, understanding unfamiliar patterns
Learning	Leaves `TODO(human):` markers where you should fill in the next step yourself	Actively learning, practising, onboarding

Example prompt:

```
/output-style learning
```

This is a session-level setting — it resets when you start a new session. For permanent preference, use `defaultOutputStyle` in your user settings.

When to switch styles:

- Pair programming with a junior: Explanatory so they see the reasoning
- Debugging a new codebase: Explanatory to understand what Claude discovers
- Practising a new language: Learning so you write the next line yourself
- Getting things done fast: Default (or omit the command entirely)

Working Directories

By default, Claude works within the directory you launched it from. For monorepos, microservices, or any project where relevant files live outside that root, you need to add directories explicitly.

Three ways to add directories:

```
# 1. At launch (CLI flag)
claude --add-dir ../shared-lib ../api-service

# 2. Inside a session (slash command — no restart needed)
/add-dir ../frontend ../design-system

# 3. Permanently in settings (persists across sessions)
```

```
// .claude/settings.json
{
  "additionalDirectories": [
    "../shared-lib",
    "../api-service",
    "../docs"
  ]
}
```

Common use cases:

Scenario	What to add
Monorepo with shared utilities	`../packages/utils`, `../packages/types`
Microservices (touching multiple)	`../auth-service`, `../payment-service`
Docs outside the repo	`../company-docs`, `~/notes/project`
Frontend + backend in separate repos	`../frontend` from the backend directory

Adding a directory lets Claude read and edit files there — same as if those files were in your working directory. Permissions apply equally.

Status Line

The status line is a real-time display in your shell prompt showing Claude's current state. It is powered by a JSON pipeline: Claude emits a JSON blob after each interaction, a script reads it from stdin and formats it, and the output appears in your prompt.

JSON schema Claude emits:

```
{
  "model": "claude-opus-4",
  "contextPercentage": 23,
  "estimatedCost": 0.042,
  "gitDirectory": "/Users/you/project",
  "currentBranch": "feature/auth"
}
```

Example bash script (`~/.claude-statusline.sh`):

```
#!/bin/bash
input=$(cat)
branch=$(echo "$input" | jq -r '.currentBranch // ""')
ctx=$(echo "$input" | jq -r '.contextPercentage // ""')
cost=$(echo "$input" | jq -r '.estimatedCost // ""')

echo "[$branch] ctx:${ctx}% \$${cost}"
```

Set up with one command:

```
/statusline
```

The `/statusline` command auto-generates this script and adds it to your shell config — you do not need to write it manually. After running it, open a new terminal and the status line appears automatically.

Useful for catching context creep before it becomes a problem: when `contextPercentage` climbs past 70%, it is time to `/compact` or `/clear`.

Tips Most Engineers Miss

1. Ask side questions without polluting context:

```
/btw what Node version does this project need?
```

The answer appears in an overlay and is never saved to history.

2. Pipe directly from the shell:

```
cat errors.log | claude -p "what is causing this?"
```

3. Use `!` for quick shell commands inside a session:

```
> ! git log --oneline -5
> ! docker ps
```

4. Name your sessions:

```
claude -n "oauth-implementation"
# Later
claude --resume oauth-implementation
```

5. Use worktrees for parallel work:

```
# Two independent sessions, two branches
claude -w feature-auth     # terminal 1
claude -w bugfix-api       # terminal 2
```

Built-in git worktree support is now available in the CLI (previously desktop-only). Each `claude -w <branch>` session gets an isolated checkout — parallel edits never conflict. Add `isolation: worktree` to a subagent's frontmatter to give each spawned subagent its own worktree too.

6. Set a cost limit:

```
claude -p "audit entire infrastructure" --max-budget-usd 2.00
```

7. Use `--add-dir` in monorepos:

```
claude --add-dir ../shared-lib ../api-service -p "refactor the shared auth code"
```

8. Double-tap Escape to rewind without typing `/rewind`.

9. Generate your CLAUDE.md automatically:

```
/init
```

Claude reads your project and generates a starter CLAUDE.md. Edit it from there.

10. Check what the auto-memory system saved:

```
/memory
```

Claude saves learnings across sessions automatically. This shows what it remembers about you and your project.

Putting It All Together: A Recommended Setup

Here is a practical starting point for a DevOps project:

`.claude/settings.json` (commit to git):

```
{
  "permissions": {
    "defaultMode": "acceptEdits",
    "allow": [
      "Bash(npm run *)",
      "Bash(git add *)",
      "Bash(git commit *)",
      "Bash(git status)",
      "Bash(git diff *)",
      "Bash(git log *)",
      "Bash(terraform plan *)",
      "Bash(terraform init)",
      "Bash(terraform fmt *)",
      "Bash(docker build *)",
      "Bash(docker ps)",
      "Bash(kubectl get *)"
    ],
    "deny": [
      "Bash(terraform destroy *)",
      "Bash(terraform apply)",
      "Bash(kubectl delete *)",
      "Bash(rm -rf *)",
      "Read(.env)"
    ]
  },
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit",
        "hooks": [{ "type": "command", "command": ".claude/hooks/format.sh" }]
      }
    ]
  }
}
```

`CLAUDE.md` (commit to git):

```
# Project Name

## Build and Test
- Build: `npm run build`
- Test: `npm test`
- Lint: `npm run lint`

## Deployment
- Staging: `npm run deploy:staging`
- Production requires manual approval — create a PR

## Architecture
- API: `src/api/` — Express + TypeScript
- Infra: `infra/` — Terraform on AWS
- K8s manifests: `k8s/`

## Rules
- No secrets in code — use AWS Secrets Manager
- All infra changes via Terraform — no manual console changes
- Every feature needs a PR and passing tests
```

Run `/init` to let Claude generate a starter version based on your actual project, then edit it.

Claude Code is at its best when it has context about your project, constraints on what it can touch, and clear instructions about your conventions. Invest 30 minutes in your CLAUDE.md and settings once, and every session after that starts with a Claude that already understands your project.

The full documentation is at docs.anthropic.com/claude-code.

Part 2: Configuring Claude's Behaviour

Writing a CLAUDE.md That Actually Works

Published 2026-04-26 | Source: https://blog.devops-monk.com/2026/04/writing-claude-md-that-works/

CLAUDE.md is behavioral programming, not documentation. Learn output styles, before/after examples, rule testing, personal global config, and when to use hooks, skills, and subagents instead.

Every `CLAUDE.md` file gets loaded into context on every session. Most teams treat it like documentation — a place to describe the project, list the tech stack, explain what the tests do. That is the wrong mental model and it is why most CLAUDE.md files are both too long and too ineffective.

`CLAUDE.md` is **behavioral programming**. Its job is to change how Claude makes decisions, not to describe facts that Claude can read from the codebase itself. Claude is stateless — it has nothing about your project at the start of each session. CLAUDE.md is the primary onboarding mechanism that enters every conversation.

The common failure mode: your CLAUDE.md grows into a 2,000-line monster. Claude reads all of it on every prompt. Slow, expensive, and half of it is not even relevant. The solution is to break that monolith into three lean, purpose-built systems:

- Modular Rules

  — small, scoped rule files that load only when relevant

- Auto Memory

  — a notebook Claude writes and reads automatically so you never have to repeat yourself

- Skills

  — deep expertise Claude loads on demand, not on every prompt

This post covers CLAUDE.md itself. The rule for each section: if it does not change how Claude makes a decision, cut it.

The Core Mental Model

```
flowchart LR
    subgraph Wrong[Wrong approach - Documentation]
        D1[What React is]
        D2[Tech stack list]
        D3[What the tests do]
        D4[Architecture overview]
    end
    subgraph Right[Right approach - Behavioral programming]
        R1[Commands Claude cannot guess]
        R2[Conventions that differ from defaults]
        R3[Decision rules for ambiguous situations]
        R4[Known gotchas and failure modes]
    end
    Wrong -->|Wastes tokens\nClaude ignores it| X[Poor results]
    Right -->|Shapes decisions\nHighly actionable| Y[Consistent results]
```

Claude already knows what React is. It can read your `package.json` to learn your stack. It understands standard testing patterns. Writing those things in `CLAUDE.md` wastes context without influencing behaviour.

What Claude *cannot* know without being told: the custom deploy command your team built, the specific reason you chose a non-standard folder structure, the production database host name format, the fact that your auth service requires a specific environment variable to initialise.

A good CLAUDE.md breaks down into three layers:

1 What

— project-specific commands, structure, and environment quirks

2 Why

— the reasoning behind non-obvious architectural decisions

3 How

— the specific ways Claude should operate within your project

---

The Instruction Budget

Before writing a single line, understand the constraint you are working within.

Claude Code’s built-in system prompt already consumes roughly 50 instructions before you write a word. Research shows frontier LLMs can follow approximately 150–200 total instructions with reasonable consistency before degradation sets in. That leaves you **100–150 slots** for your project-specific rules.

A bloated CLAUDE.md does not just waste space — it competes with your actual rules. When the file is too long, Claude begins filtering what it applies. Rules near the bottom are the first casualties. If Claude keeps doing something you have a rule against, the file is probably too long and the rule is getting lost.

The practical target: **under 200 lines**. Some experienced teams run under 60. Anthropic’s own internal teams keep their CLAUDE.md around 100 lines.

File Locations

CLAUDE.md files can live in several places and Claude loads them based on what you are editing:

Location	Scope	Notes
`~/.claude/CLAUDE.md`	All sessions, all projects	Personal preferences and global rules
`./CLAUDE.md`	Project-wide	Commit to git — shared with the whole team
`./CLAUDE.local.md`	Project-wide, personal	Add to `.gitignore` — not shared
`./subdir/CLAUDE.md`	Loads when editing files in that directory	Great for monorepos

More specific files override more general ones. Use `CLAUDE.local.md` for anything personal: your preferred verbosity level, tools you like, local path overrides. Your team does not need to see those.

Parent directory files also load automatically. In a monorepo, both `root/CLAUDE.md` and `root/services/api/CLAUDE.md` are pulled in when you edit a file inside `root/services/api/`.

Structure That Works

Here is a template that covers what matters without bloating the file:

```
# Project: Platform API

## Commands
Commands Claude cannot guess from reading the code:
- Build: `npm run build`
- Tests: `npm test` — run a single test with `npm test -- --grep "auth"`
- Staging deploy: `./scripts/deploy.sh staging`
- Database migrations: `npm run db:migrate`
- Seed development data: `npm run db:seed`

## Directory Structure
src/
  api/         # REST endpoints — one file per resource
  lib/         # Pure utility functions, no side effects
  db/queries/  # Raw SQL only, no ORM
  auth/        # Auth service — requires NODE_ENV=production

## Conventions (where we differ from defaults)
- 4-space indentation, not 2
- camelCase for functions and variables, PascalCase for classes and types
- ES modules only — never use CommonJS require()
- All async functions must have explicit error handling — no unhandled rejections

## Architecture Decisions
- Raw SQL only in db/queries/ — we evaluated Prisma and rejected it for performance reasons
- Tests colocate with source: `auth.ts` → `auth.test.ts` in the same folder

## Model Routing
- Haiku: renaming, formatting, simple lookups, one-liners
- Sonnet: standard implementation, debugging, code review, tests
- Opus: architecture decisions, complex refactors, security analysis

## Known Gotchas
- Auth service requires NODE_ENV=production to initialise — set it in tests
- Database connections pool at 10 — do not exceed in tests or you get timeouts
- The payment processor sandbox rejects amounts over $10,000 — use $99.99 in tests
- Migrations do not auto-run in tests — call `db.migrate()` manually in test setup

## Git
- Feature branches: `feature/description`
- All changes via PR — never push to main directly
- PR titles must include the ticket number: `PLAT-123: description`

## Compact Instructions
When compacting, always preserve: the full list of modified files, test commands, and any open decisions.
```

The `## Compact Instructions` section is worth calling out: it tells Claude what to preserve when the context window auto-compacts during long sessions, so critical state survives summarisation.

Output Styles: Teaching Claude How to Talk to You

One of the most underused capabilities in CLAUDE.md is controlling how Claude communicates — not just what it does, but how it responds. This is called output style configuration, and getting it right eliminates a surprising amount of back-and-forth.

Add an `## Output Style` section to your CLAUDE.md:

```
## Output Style

- Be terse and direct — no filler phrases, no "Great question!", no "Certainly!"
- When making code changes, show the diff only — do not re-print the whole file
- When explaining decisions, lead with the decision, then the reason
- For error messages, always include: what failed, why, and the exact command to fix it
- Do not add a trailing summary of what you just did — I can read the change
- When you are uncertain, say so explicitly — do not guess and present it as fact
- Ask one clarifying question at a time, not a list of five
```

Or the opposite — if you prefer more explanatory responses:

```
## Output Style

- Explain your reasoning before making changes, especially for architectural decisions
- After completing a task, summarise what changed and why
- When I ask about an error, include context about what might have caused it
- Use code comments to explain non-obvious logic you add
```

The key insight: Claude defaults to a balanced verbosity that works for most people but is perfect for nobody. An explicit output style instruction takes about 5 lines and eliminates the most common frustrations — over-explanation when you just want the fix, under-explanation when you need to understand the decision.

Verbosity per task type

You can specify different verbosity for different contexts:

```
## Output Style

- Code changes: minimal output — diff only, no explanation unless I ask
- Architecture discussions: detailed — include trade-offs and alternatives
- Error diagnosis: step by step — show your reasoning so I can verify it
- Refactoring: list the files changed and why, then show diffs
- Tests: show what scenarios are covered and what is explicitly not covered
```

Controlling response format

```
## Output Style

- Use markdown headers for any response over 3 paragraphs
- Use code blocks for all commands — even one-liners
- For tables, use markdown tables not plain text
- Mermaid diagrams for architecture, never ASCII art
- Numbered lists for sequential steps, bullet lists for options
```

Before vs After: Real Examples

Abstract rules are easy to forget. Here are concrete before/after pairs showing what the rules look like in practice.

Bad: Too much narrative

```
## About the Project

This is a Node.js REST API built with Express that handles user authentication
and manages subscription billing. The project uses TypeScript for type safety
and was started in 2022. We use PostgreSQL as our primary database because
of its reliability and the team's familiarity with it. Tests are written
with Jest and we follow the AAA pattern (Arrange, Act, Assert).
```

Why it fails: Claude already knows what Node.js, Express, TypeScript, PostgreSQL, and Jest are. Nothing here changes a decision Claude would otherwise make incorrectly.

Good: Commands and gotchas only

```
## Commands
- Build: `npm run build`
- Test: `npm test` — single test: `npm test -- --testNamePattern "auth"`
- Migrate: `npm run db:migrate` — does NOT auto-run in tests, call manually
- Lint: `npm run lint:fix` (not `lint` — that one only reports, does not fix)

## Gotchas
- Auth service requires `NODE_ENV=production` to initialise — even in test files
- DB pool max is 10 — never exceed in concurrent tests or you get ECONNREFUSED
- Payment sandbox rejects amounts over $10,000 — use 9999 in tests
```

Why it works: Every line changes what Claude would do without it. The `npm run lint` vs `npm run lint:fix` distinction alone prevents broken CI runs.

Bad: Self-evident convention

```
## Code Quality

- Write clean, readable code
- Use meaningful variable names
- Handle errors properly
- Add comments where necessary
- Follow SOLID principles
```

Why it fails: These apply to every codebase and Claude already does them. Zero decision impact.

Good: Deviations from defaults

```
## Conventions (where we differ from defaults)

- 4-space indentation — our .editorconfig enforces this
- Named exports only — no default exports anywhere in the codebase
- Error handling: use Result<T, E> from `neverthrow` — no try/catch in business logic
- All database queries in `src/db/queries/` only — never inline SQL in service files
- camelCase for files: `userService.ts` not `user-service.ts`
```

Why it works: Every rule here deviates from what Claude would naturally do. Without these, Claude would use default exports, try/catch, and kebab-case file names.

Bad: History instead of instruction

```
## Architecture

We evaluated several ORMs including Prisma, TypeORM, and Sequelize before
settling on raw SQL. The main reasons were performance, the team's SQL expertise,
and avoiding the abstraction leakiness we experienced with ORMs in previous projects.
```

Why it fails: Informative, but gives Claude no decision guidance.

Good: Decision + rule

```
## Architecture Decisions

- Raw SQL only in `src/db/queries/` — no ORM, no query builder
  - DO NOT suggest Prisma or TypeORM — evaluated and rejected
  - New queries follow the pattern in `src/db/queries/users.ts`
- REST only — no GraphQL. Reason: team expertise, not a tech constraint
```

Why it works: The explicit “DO NOT suggest” is the key addition. Without it, Claude will recommend ORMs when discussing database work because that is usually the right call.

Testing Your Rules

Adding a rule to CLAUDE.md is not the same as the rule being followed. Test each rule you add:

The verification method

1. Write the rule
2. Start a fresh session (

   /clear

   or close and reopen)
3. Give Claude a task where the rule applies
4. Check whether Claude’s output reflects the rule
5. If it does not: the rule is either buried too deep, worded too weakly, or contradicted elsewhere

Common failure patterns

Rule is buried: Move the rule earlier in the file. Rules in the first 100 lines are followed more consistently than rules at line 180.

Rule is too weak: “Prefer parameterised queries” is soft — Claude may override it when it decides the alternative is acceptable. “NEVER use string interpolation in SQL — always use parameterised queries” is hard.

Rule is contradicted: Search the file for any instruction that could be interpreted as an exception to your rule. Pick one direction and remove the other.

Rule is already followed by default: Claude will follow standard best practices without being told. Test whether the rule changes anything. If Claude behaves the same way without the rule, delete the rule.

The self-correction instruction

Add this line to your CLAUDE.md and it will catch stale rules automatically:

```
## Meta

When you encounter a CLAUDE.md rule that seems incorrect or outdated based on
what you observe in the codebase, say so explicitly and suggest a correction.
```

This turns Claude into a co-maintainer of the file — it flags contradictions rather than silently ignoring them.

Personal Global CLAUDE.md

The `~/.claude/CLAUDE.md` file loads in every project and every session. Most developers leave it empty. That is a missed opportunity.

Your global CLAUDE.md is the right place for:

- Personal preferences

  that apply everywhere — verbosity, formatting, response style
- Model routing

  defaults you always want
- Tools and workflow

  preferences that are about you, not any specific project
- Things you are tired of repeating

  at the start of every session

Example `~/.claude/CLAUDE.md`:

```
# Personal Preferences (Abhay)

## Output Style
- Be direct and terse — no filler, no trailing summaries
- When making code changes, show what changed and why in one line — not a paragraph
- If you are uncertain about something, say "I'm not sure — here is my best guess"
- Never use emoji in code comments or documentation

## Model Routing
- Default to Sonnet for everything
- Switch to Opus automatically for: architecture decisions, security analysis,
  anything involving production data
- Use Haiku for: renaming, reformatting, simple one-liner lookups

## Work Style
- Before making large changes, outline the plan and ask me to confirm
- When I say "fix this", fix the specific thing — do not refactor surrounding code
- Run tests before reporting a task as complete
- If tests fail, investigate and fix before telling me the task is done

## Always
- Use British English spelling (colour, behaviour, favour)
- Prefer `const` over `let` unless mutation is required
- Never suggest adding a dependency for something achievable in 10 lines of standard library
```

Keep it under 60 lines. Anything project-specific goes in the project's CLAUDE.md. Global rules should apply to everything you do, everywhere.

Imports: Keep CLAUDE.md Lean

CLAUDE.md supports `@path/to/file` imports. Use them to reference existing documents instead of duplicating content:

```
# Project: Platform API

See @README.md for project overview and @package.json for available npm commands.

## Additional Conventions
- Git workflow: @docs/git-instructions.md
- Security rules: @docs/security-checklist.md
- Personal overrides: @~/.claude/my-project-instructions.md

## Known Gotchas
- Auth service requires NODE_ENV=production to initialise — set it in tests
- Migrations do not auto-run in tests — call `db.migrate()` manually in test setup
```

One important caveat: `@` imports embed the entire file at load time. Importing a 500-line document burns your instruction budget before your actual rules load. For large documents, use descriptive text links instead of `@` imports:

```
# Project: Platform API

## Additional Context
For migration procedures, see `docs/migrations.md`.
For API design conventions, see `docs/api-design.md`.
```

This way Claude fetches the file only when relevant to the current task, rather than loading it into every session.

What to Include vs What to Skip

```
flowchart TD
    Q[New piece of information] --> A{Can Claude learn this\nby reading the codebase?}
    A -->|Yes - it is in the code| Skip[Skip it\nWaste of context]
    A -->|No - it requires\nexternal knowledge| B{Does it affect\nhow Claude makes decisions?}
    B -->|No - it is just a fact| Skip
    B -->|Yes - it changes behaviour| Include[Include it\nHigh value]
```

Apply the **removal test** before every line: *"Would removing this cause Claude to make mistakes?"* If the answer is no, cut it. If Claude already does something correctly without the instruction, it is wasting a slot.

Include	Exclude
Bash commands Claude can't guess	Anything Claude can figure out by reading code
Code style rules that differ from defaults	Standard language conventions Claude already knows
Testing instructions and preferred test runners	Detailed API documentation (link to docs instead)
Repository etiquette (branch naming, PR conventions)	Information that changes frequently
Architectural decisions specific to your project	Long explanations or tutorials
Developer environment quirks (required env vars)	File-by-file descriptions of the codebase
Common gotchas or non-obvious behaviors	Self-evident practices like "write clean code"
Model routing guidelines	Technology definitions and descriptions

Writing craft tip: every "never" should be paired with an alternative direction. "Never use string interpolation in SQL queries" is weak on its own. "Never use string interpolation in SQL queries — use parameterised queries: `db.query(sql, [params])`" gives Claude somewhere to

go. Prohibitions without alternatives leave Claude stuck.

To improve adherence to critical rules, mark them explicitly: `IMPORTANT: Never commit secrets to git` or `YOU MUST validate user input at the API boundary`. Use this sparingly — emphasis scales poorly. If every rule is marked important, the emphasis becomes invisible and stops working.

CLAUDE.md vs Skills vs Hooks vs Subagents

These four tools are often confused. They serve different purposes:

Tool	Purpose	When to use
`CLAUDE.md`	Persistent context, always loaded	Conventions and rules that apply to every session
Skills (`.claude/skills/`)	Domain knowledge, loaded on demand	Workflows and deep context for specific tasks
Hooks (`.claude/settings.json`)	Deterministic enforcement	Actions that must happen every time with no exceptions
Subagents (`.claude/agents/`)	Isolated specialist context	Tasks that read many files or need specialised focus

CLAUDE.md instructions are **advisory** — Claude reads them and applies judgement. Hooks are **deterministic** — they run regardless of what Claude decides. If you want a linter to run after every file edit, a hook guarantees it. A CLAUDE.md instruction asking Claude to run the linter is a suggestion.

Skills

Use **Skills** for task-specific knowledge that would bloat CLAUDE.md if it were always loaded: a deep guide to your API design conventions, a workflow for fixing GitHub issues, a checklist for database migrations. Skills load when relevant and keep your base context lean.

Create a skill by adding a directory with a `SKILL.md` to `.claude/skills/`:

```
---
name: fix-github-issue
description: Fix a GitHub issue end-to-end with tests and PR
disable-model-invocation: true
---

Analyze and fix the GitHub issue: $ARGUMENTS.

1. Use `gh issue view` to get the issue details
2. Understand the problem described in the issue
3. Search the codebase for relevant files
4. Implement the necessary changes to fix the issue
5. Write and run tests to verify the fix
6. Ensure code passes linting and type checking
7. Create a descriptive commit message
8. Push and create a PR
```

Run `/fix-github-issue 1234` to invoke it. Use `disable-model-invocation: true` for workflows with side effects you want to trigger explicitly.

Subagents

Subagents run in their own isolated context with their own set of allowed tools. They are useful for tasks that read many files without cluttering your main conversation. Create one in

`.claude/agents/`:

```
---
name: security-reviewer
description: Reviews code changes for security vulnerabilities
tools: Read, Grep, Glob, Bash
model: claude-opus-4-7
---

You are a senior security engineer. Review code for:
- Injection vulnerabilities (SQL, XSS, command injection)
- Authentication and authorization flaws
- Secrets or credentials in code
- Insecure data handling

Provide specific file:line references and suggested fixes.
```

Invoke directly: *"Use the security-reviewer subagent to review these changes."* Because subagents run in separate context windows, they are also useful for quality-focused workflows — a fresh context improves code review since the reviewer is not biased toward code it just wrote.

Path-Scoped Rules for Large Projects

The Priority Saturation Problem

A CLAUDE.md that grows to 400+ lines with all topics mixed together stops working. When React rules, database rules, API rules, and infrastructure rules all load every session, Claude treats every instruction as equally important — which is the same as treating nothing as important. The rules compete with each other and with the conversation context.

Before: monolithic CLAUDE.md (400 lines, always consumed)

```
.claude/
└── CLAUDE.md    ← React rules, DB rules, API rules, infra rules, test rules... all 400 lines
```

Every session loads 400 lines of context — even when you are only editing a Terraform file and the React and test rules are irrelevant.

After: modular rules (only ~100 lines active at any time)

```
.claude/
├── CLAUDE.md           ← 50 lines: universal rules only
└── rules/
    ├── api.md          ← 50 lines: loads when editing src/api/**
    ├── frontend.md     ← 50 lines: loads when editing src/frontend/**
    ├── infra.md        ← 50 lines: loads when editing infra/**
    └── tests.md        ← 50 lines: loads when editing **/*.test.*
```

Total words on disk: 250 lines. Active at any time: ~100 lines (CLAUDE.md + one scoped file). Signal density goes up, cost goes down.

Always-Loaded vs Path-Scoped Rules

Always-loaded — no frontmatter, loads every session:

```
<!-- .claude/rules/git-workflow.md — no frontmatter = always loaded -->

# Git Workflow
- Never commit to main directly
- Branch naming: feat/, fix/, chore/ prefixes
```

Path-scoped — frontmatter with `paths:` key, loads only when Claude works on matching files:

```
---
paths:
  - "src/api/**/*.ts"
  - "src/api/**/*.js"
---

# API Rules
- All endpoints must validate the Authorization header before any business logic
- Return 422 (not 400) for validation errors — include field-level error details
- Never return stack traces in error responses
- Log request IDs for all errors using logger.error({ requestId, error })
```

The `paths:` key uses glob patterns. When Claude reads or edits a file matching any pattern, the rule file loads automatically. When it is not working on matching files, the rule is invisible and consumes no context.

The decision table:

Put it in CLAUDE.md	Put it in a rules/ file
Universal: applies to every task	Domain-specific: only relevant to one area
Short: under 50 lines	Detailed: can be longer because it loads conditionally
Foundational: other rules depend on it	Self-contained: works without other context
Security / legal: must never be missed	Performance / style: helpful but not critical

Personal rules across all projects — place rules in `~/.claude/rules/` (user-level) to apply them everywhere, not just in one project. These load alongside CLAUDE.md in every session regardless of which project you open. Good for personal style preferences you want universally: preferred comment style, your go-to test patterns, tools you always use.

For monorepos where you want to prevent other teams' rules from loading, use `claudeMdExcludes` in `.claude/settings.local.json` (outside version control):

```
{
  "claudeMdExcludes": [
    "**/other-team/.claude/rules/**"
  ]
}
```

The Memory System

Claude Code has three distinct layers of memory. Understanding which is which prevents confusion when Claude seems to "forget" something or "remember" something you did not tell it.

The Three Layers

Layer	What it is	When it loads	You control it?
CLAUDE.md	Your rules — commit to git	Every session, always	Yes — edit the file

Layer	What it is	When it loads	You control it?
Auto Memory (MEMORY.md)	Claude's observations about you and your project — written by Claude	First 200 lines at startup; rest on-demand per topic	Yes — but Claude writes it
Session Memory	The conversation history from your last session	Only on `claude -c` (resume)	No — auto-managed

Think of CLAUDE.md as the textbook you hand Claude before it starts. Auto Memory is the notebook it keeps for itself. Session Memory is what it remembers from your last conversation.

Layer 1: CLAUDE.md — your instructions, committed to git, shared with your team, loaded every session.

Layer 2: Auto Memory — Claude writes what it observes ("user prefers pytest over unittest", "this project uses custom error types from `lib/errors.ts`") into `~/.claude/projects/<project>/memory/`. The `MEMORY.md` index file is loaded in full at startup. Individual topic files (e.g., `user_prefs.md`, `feedback.md`) are loaded on demand as they become relevant. The first 200 lines of `MEMORY.md` are always in context — Claude uses the index to decide which topic files to pull in.

Layer 3: Session Memory — when you run `claude -c` to resume, the conversation history from your previous session is restored. Permissions and session-scoped settings reset (you re-approve them), but Claude's understanding of what you were working on comes back.

Auto Memory Structure

```
~/.claude/projects/my-project/memory/
├── MEMORY.md          ← index file — first 200 lines loaded at startup
├── user_prefs.md      ← your personal preferences
├── feedback.md        ← corrections and lessons learned
├── project_ctx.md     ← project-specific context
└── decisions.md       ← architectural decisions reached
```

Sample `feedback.md`:

```
---
name: Feedback - Database queries
type: feedback
---

Always use parameterised queries in this project, never string interpolation.

**Why:** A previous incident where a dynamic filter was built with string concatenation
caused a data leak in the staging environment.

**How to apply:** Any time writing a database query, use `db.query(sql, [params])` not
`db.query(sql + userInput)`.
```

The "Why" and "How to apply" sections matter — they help Claude judge edge cases rather than blindly applying a rule.

Triggering Memory Explicitly

Auto Memory doesn't only rely on Claude's own judgment — you can instruct it directly in plain English:

- "remember that we use pnpm, not npm"

- “save to memory that the API tests require a local Redis instance”
- “note that the staging environment uses port 3001”

Claude writes the fact into its memory files immediately. This is the fastest way to seed memory for something you know Claude will need repeatedly.

Watching Memory in Action

When Claude interacts with its memory files, you see status messages in the Claude Code interface:

- “writing memory”

  — Claude is saving something it learned
- “recalled memory”

  — Claude is reading from its saved notes

These are the visual cues that Auto Memory is working. If you never see them, either auto memory is disabled or the session hasn’t produced anything worth persisting yet.

Checking and Editing Memory

```
/memory
```

The `/memory` command opens an interactive selector that shows:

- All CLAUDE.md files currently loaded in your session (project, user, rules, etc.)
- The Auto Memory toggle — an ON/OFF switch
- A link to open the auto memory folder so you can browse what Claude has saved

From here you can toggle auto memory on/off, open any memory file in your editor, and browse all saved notes. Edit or delete stale entries directly in the files — same as editing CLAUDE.md. If Claude has learned something wrong, correct it and tell Claude you did so.

One important note: memory is scoped to a directory, not a git branch. All branches and worktrees of the same repo share one memory directory. This is usually what you want — your preferences apply regardless of branch. If you need branch-specific memory (rare), use CLAUDE.md instead, since that is committed to git and can differ per branch.

Configuring Auto Memory

```
// ~/.claude/settings.json (user-level)
{
  "autoMemoryEnabled": false          // disable auto memory entirely
}
```

Or via environment variable (useful in CI/CD):

```
CLAUDE_CODE_DISABLE_AUTO_MEMORY=1 claude -p "..."
```

For a custom memory directory (e.g., keeping all Claude memory in a shared Dropbox folder):

```
{
  "autoMemoryDirectory": "/Users/you/Dropbox/claude-memory/my-project"
}
```

Session Patterns That Compound Over Time

The Session Retrospective

At the end of each meaningful session, ask Claude to summarise what it learned:

```
What did you learn this session that should be persisted?
- General concepts → CLAUDE.md
- Architectural choices → decision records in docs/
- Technical skills → skill files in .claude/skills/
- Personal preferences → memory files
```

Documentation that persists between sessions means Claude avoids repeating the same mistakes. Over weeks, this compounds into a genuinely sharp project assistant.

Never Patch Bugs Yourself Mid-Session

When Claude misses a bug you spot, resist the urge to fix it directly. Instead, have Claude investigate and document the cause. Since documentation persists across sessions, this builds institutional knowledge the agent can draw on later. Quick fixes that bypass investigation lose the lesson.

The Clear/Rewind Pattern

Rather than correcting Claude mid-conversation, use `/rewind` or `/clear` to return to the last stable state when something goes wrong. Flawed responses stay in context and pollute subsequent attempts. Two failed corrections in a row is a signal to clear context and re-prompt with what you learned from those failures.

Parallel Sessions: Writer / Reviewer

Run multiple Claude sessions in parallel for quality-critical work:

Session A (Writer)	Session B (Reviewer)
`Implement a rate limiter for the API endpoints`	
	`Review @src/middleware/rateLimiter.ts — look for edge cases, race conditions, and consistency with existing middleware patterns`
`Here is the review feedback: [output]. Address these issues.`	

A fresh context improves review quality because the reviewer is not biased toward code it just wrote.

Common Mistakes

1. Writing for humans, not for Claude

Your team lead reads `CLAUDE.md` and adds context like “We switched from Mongoose to Prisma in 2023 because of type safety issues.” Interesting history, but it gives Claude no actionable

instruction. Cut it.

2. Contradicting yourself

“Always write tests before code” and “tests are optional for hotfixes” in the same file creates inconsistent behaviour. Claude will apply one or the other unpredictably. Pick one and remove the other.

3. Exceeding the instruction budget

With ~50 instructions already in the system prompt and a cap of ~200 before degradation, you have roughly 150 slots. A 300-line CLAUDE.md likely burns most of them. Some rules at the bottom will be routinely ignored. Better to have 80 lines that are always followed than 300 lines that are sometimes followed.

4. Stale rules

`CLAUDE.md` needs maintenance. Review it every two weeks and remove anything that is no longer true. Stale rules reduce trust in the rules that remain. Treat it like code: review it when Claude makes an unexpected decision, prune it regularly, and verify that changes actually shift Claude’s behaviour. Add this standing instruction to CLAUDE.md itself: `"When you encounter a bad assumption, suggest a correction to this file."`

5. Accumulation without pruning

The most common long-term failure mode: rules pile up, nobody prunes, Claude filters out half of them. A rule that Claude already follows correctly without being told is wasting a slot. Every few weeks, audit the file and delete anything that passes the removal test.

6. Using CLAUDE.md for enforcement

If a rule must happen every time with zero exceptions — running the linter, blocking writes to the migrations folder, posting a Slack notification — use a hook. CLAUDE.md instructions are advisory. Hooks are guarantees. Claude Code lets you ask it to write hooks for you: *“Write a hook that runs eslint after every file edit.”*

7. Large `@` imports burning instruction budget

Importing a large file with `@` embeds its entire content at session start. A 500-line architecture doc imported in CLAUDE.md costs you context on every single session, even when you are not working on that area. Use descriptive text references instead; Claude fetches the file only when needed.

8. Adding rules before Claude makes mistakes

Add rules to CLAUDE.md only in response to actual mistakes, not hypothetical ones. Speculative rules add noise and rarely trigger. Wait until Claude does something wrong, then codify the correction.

Quick Start

Step 1: Generate a starter file

```
claude
/init
```

Claude reads your project and generates a starter CLAUDE.md. Do not accept it as-is — this is a first draft, not a finished file.

Step 2: Prune the output

Read every line of the generated file. For each line, ask the removal test question: *"Would removing this cause Claude to make a mistake?"* Delete every line where the answer is no. You will typically cut 40–60% of the generated content.

Step 3: Add what the generator missed

The generator can read your code but cannot know:

- Why you made unusual architectural choices
- What commands you actually run (vs what the README says)
- The production gotchas your team has discovered the hard way
- Your personal output style preferences

Add those now. They are the highest-value lines in the file.

Step 4: Add an Output Style section

Even 5 lines here saves hours of back-and-forth over a month:

```
## Output Style
- Be direct — no filler phrases, no trailing summaries of what you just did
- Show diffs, not full file reprints
- If uncertain, say so — do not guess and present it as fact
```

Step 5: Test the file

Close the session, reopen it, and give Claude a task where your rules apply. If the output does not reflect your rules, the rule is either too weak, too buried, or contradicted elsewhere.

The ongoing loop:

1. Claude makes an unexpected decision → add a rule
2. Rule already being followed correctly → delete the rule (wasting a slot)
3. Domain knowledge growing the file → move it to a Skill
4. Rule must always happen → convert it to a Hook
5. Review every two weeks — prune anything stale

Every mistake becomes a rule. Every rule that is no longer needed gets cut. The longer a team works this way, the sharper the agent gets in that specific codebase.

A Note on Auto Memory

Claude Code maintains a persistent memory system at `~/.claude/projects/<project>/memory/`. Across sessions, Claude reads these files to recall facts about your project and your preferences — without you repeating them.

This is complementary to, not a replacement for, CLAUDE.md:

	CLAUDE.md	Auto Memory
Purpose	Behaviour rules and conventions	Facts and preferences learned over time
Who writes it	You, deliberately	Claude, from sessions
Scope	Shared with the team (if committed)	Personal (in your home directory)
Token cost	Every session	Every session

The key distinction: CLAUDE.md sets the rules for how Claude should work in your project. Auto memory records what Claude has learned about you and your project — specific facts, corrections you have made, preferences you have stated. Both load every session, so keeping both lean matters.

Article 4 of 21

Claude Code Hooks, Commands, Skills, and Subagents: The Complete Guide

Published 2026-04-26 | Source:
https://blog.devops-monk.com/2026/04/claude-code-hooks-subagents-piping/

How to create and use all four Claude Code automation layers — hooks, custom slash commands, skills, and subagents — with real examples and a decision framework for when to reach for each.

Most teams use Claude Code reactively — they type a prompt, Claude responds, they type another. That is fine, but it leaves significant value on the table. Claude Code has four automation layers that let you turn it from a reactive assistant into an active workflow participant.

Consider the core problem each layer solves: you tell Claude never to touch a file, it touches it anyway. You ask it to skip secrets, it commits an API key. CLAUDE.md instructions are hopes — they are advisory, and Claude applies judgement. **Hooks turn hopes into guarantees.** And when you need parallel deep expertise instead of a single opinion, subagents let you spin up a team.

Layer	What it does	When to reach for it
Hooks	Shell or HTTP calls that fire on lifecycle events	“This must happen every time, without exception”
Custom Commands	Reusable slash commands for repeatable prompts	“I type the same prompt repeatedly”
Skills	Context-aware instructions Claude loads automatically	“Claude should always do X when working on Y”
Subagents	Separate Claude instances for isolated, parallel work	“This task is noisy and the main session only needs a summary”

This post covers how to create each one and when to use them.

Hooks: Deterministic Side Effects

A hook is a shell command or HTTP call that fires automatically when Claude Code reaches a specific lifecycle point. Unlike CLAUDE.md instructions (which are advisory), hooks are **deterministic** — they always run, regardless of what Claude decides to do.

```
flowchart TD
    SS[SessionStart] --> Work[Developer works]
    Work --> PTU[PreToolUse\nruns before each tool]
    PTU -->|exit 2 = block| Block[Action blocked]
    PTU --> Tool[Tool executes\nBash, Edit, Write, etc.]
    Tool --> PTUS[PostToolUse\nruns after tool succeeds]
    Tool --> PTUF[PostToolUseFailure\nruns after tool fails]
    PTUS --> Work
    PTUF --> Work
    Work --> SE[SessionEnd]
```

Hook lifecycle events

All hooks for the same event run **in parallel**. If multiple hook entries contain the same command, Claude Code automatically deduplicates — it runs only once.

Event	When it fires	Common use
`SessionStart`	Once at startup	Load context, warm caches, load `.envrc`
`UserPromptSubmit`	When you submit a prompt, before Claude processes it	Input sanitisation, logging
`PreToolUse`	Before every tool call (can block the tool)	Block dangerous commands, dry-run checks
`PermissionRequest`	When Claude needs your approval	Custom approval workflows
`PostToolUse`	After every successful tool call	Run tests, format, log
`PostToolUseFailure`	After every failed tool call	Alert on errors, cleanup
`Notification`	When Claude Code sends a notification	Relay to Slack, mobile push
`SubagentStart`	When a subagent is created	Log subagent activity
`SubagentStop`	When a subagent completes	Aggregate results
`Stop`	When Claude finishes generating its response	Notify user, trigger follow-up
`TeammateIdle`	When an agent team member becomes idle	Reassign work
`TaskCompleted`	When a task is marked completed	Trigger downstream steps
`ConfigChange`	When a config file is modified during the session	Reload settings
`WorktreeCreate`	When a Git worktree is created	Initialise worktree-specific config
`WorktreeRemove`	When a worktree is removed	Cleanup
`PreCompact`	Before Claude performs context compaction	Save current state
`SessionEnd`	Once on exit	Summarise session, commit artifacts

Creating hooks interactively: type `/hooks` in any session to add, edit, or delete hooks through a menu — no JSON editing required.

Hook types

There are four handler types:

```
{ "type": "command",  "command": ".claude/hooks/script.sh" }
{ "type": "http",     "url": "https://internal.api/hook", "method": "POST" }
{ "type": "prompt",   "prompt": "Summarise what just changed" }
{ "type": "agent",    "agent": "code-reviewer" }
```

Command hooks are the most common — a shell script that receives JSON on stdin and returns JSON on stdout.

HTTP hooks call a URL instead of a local script — useful for sending events to an audit log, a Slack webhook, or an internal policy service.

Prompt hooks feed the event data back to Claude as a follow-up prompt — rarely needed but useful for logging natural-language summaries.

Agent hooks invoke a named subagent (see Subagents below) in response to a lifecycle event.

How to create hooks

Hooks live in `.claude/settings.json` (project-wide) or `~/.claude/settings.json` (your personal config):

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit",
        "hooks": [
          {
            "type": "command",
            "command": ".claude/hooks/on-edit.sh"
          }
        ]
      }
    ],
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": ".claude/hooks/pre-bash.sh"
          }
        ]
      }
    ]
  }
}
```

The `matcher` field filters by tool name: `"Edit"`, `"Bash"`, `"Write"`, `"*"` (all tools). Each hook script receives a JSON payload on stdin:

```
{
  "tool_name": "Edit",
  "tool_input": { "path": "src/api.ts", "old_string": "...", "new_string": "..." },
  "cwd": "/Users/dev/my-project",
  "session_id": "abc123",
  "duration_ms": 120
}
```

Exit codes matter in PreToolUse:

- Exit

0

— allow the action

- Exit

  2

  — block the action (Claude sees the reason you print to stdout)
- Any other non-zero — hook error, action proceeds anyway

When to use hooks

Hooks are for **enforcement and side effects that must happen without exception**:

- Auto-running tests after every edit
- Blocking destructive commands before they execute
- Enforcing formatting standards before code is saved
- Logging every tool call to an audit trail
- Sending notifications when long tasks finish

If you find yourself writing “always run X after Y” in your CLAUDE.md, it probably belongs in a hook instead — hooks guarantee execution, CLAUDE.md only requests it.

Example: Scan for secrets before any file is written

This is the most important hook to add first. It runs before Claude writes any file and blocks the action if secrets or oversized files are detected — before they can be committed:

```
#!/bin/bash
# .claude/hooks/pre-write-scan.sh
input=$(cat)
file=$(echo "$input" | jq -r '.tool_input.path // ""')
content=$(echo "$input" | jq -r '.tool_input.content // ""')

# Block if secrets detected in content
SECRET_PATTERNS='(api[_-]?key|secret|password|token|credential)\s*[=:]\s*["\x27][A-Za-z0-9+/=]{8,}'

if echo "$content" | grep -qiE "$SECRET_PATTERNS"; then
  jq -n '{
    hookSpecificOutput: {
      hookEventName: "PreToolUse",
      permissionDecision: "deny",
      permissionDecisionReason: "Secret pattern detected in file content. Remove credentials before
writing."
    }
  }'
  exit 0
fi

# Block writes to files above 1MB (likely a mistake)
if [ -n "$content" ] && [ "$(echo "$content" | wc -c)" -gt 1048576 ]; then
  jq -n '{
    hookSpecificOutput: {
      hookEventName: "PreToolUse",
      permissionDecision: "deny",
      permissionDecisionReason: "File content exceeds 1MB. Review before writing."
    }
  }'
  exit 0
fi

exit 0
```

Wire it up for all Write and Edit operations:

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Write",
        "hooks": [{ "type": "command", "command": ".claude/hooks/pre-write-scan.sh" }]
      },
      {
        "matcher": "Edit",
        "hooks": [{ "type": "command", "command": ".claude/hooks/pre-write-scan.sh" }]
      }
    ]
  }
}
```

Example: Shield critical files from all edits

A shell script that physically prevents Claude from touching specific files — regardless of what it was asked to do:

```
#!/bin/bash
# .claude/hooks/shield-files.sh
input=$(cat)
tool=$(echo "$input" | jq -r '.tool_name // ""')
file=$(echo "$input" | jq -r '.tool_input.path // ""')

# Files Claude must never touch
SHIELDED_FILES=(
  ".env"
  ".env.production"
  "config/secrets.yml"
  "infra/terraform.tfvars"
  "k8s/secrets/"
)

for shield in "${SHIELDED_FILES[@]}"; do
  if [[ "$file" == *"$shield"* ]]; then
    jq -n --arg f "$file" '{
      hookSpecificOutput: {
        hookEventName: "PreToolUse",
        permissionDecision: "deny",
        permissionDecisionReason: ("Shielded file: " + $f + " is protected from AI edits. Edit
manually.")
      }
    }'
    exit 0
  fi
done

exit 0
```

Example: Auto-run tests after every edit

Every time Claude edits a file, tests run automatically. Claude sees the results immediately and can fix breaking changes without prompting.

```
#!/bin/bash
# .claude/hooks/on-edit.sh
input=$(cat)
CWD=$(echo "$input" | jq -r '.cwd // "."')

cd "$CWD"

if [ -f "package.json" ]; then
  npm test --silent 2>&1 | tail -20
elif [ -f "go.mod" ]; then
  go test ./... 2>&1 | tail -20
elif [ -f "requirements.txt" ]; then
  python -m pytest --tb=short -q 2>&1 | tail -20
fi

exit 0
```

Example: Block destructive Bash commands

```
#!/bin/bash
# .claude/hooks/pre-bash.sh
input=$(cat)
cmd=$(echo "$input" | jq -r '.tool_input.command // ""')

DANGEROUS="rm -rf|kubectl delete|terraform destroy|DROP TABLE"

if echo "$cmd" | grep -qiE "$DANGEROUS"; then
  jq -n --arg cmd "$cmd" '{
    hookSpecificOutput: {
      hookEventName: "PreToolUse",
      permissionDecision: "deny",
      permissionDecisionReason: "Blocked: destructive command. Run manually after review."
    }
  }'
  exit 0
fi

exit 0
```

Example: Auto-format on save

```
#!/bin/bash
# .claude/hooks/format.sh
input=$(cat)
file=$(echo "$input" | jq -r '.tool_input.path // ""')

[ -z "$file" ] && exit 0

case "$file" in
  *.ts|*.tsx|*.js|*.jsx) npx prettier --write "$file" 2>/dev/null ;;
  *.py) black "$file" 2>/dev/null ;;
  *.go) gofmt -w "$file" 2>/dev/null ;;
  *.tf) terraform fmt "$file" 2>/dev/null ;;
esac

exit 0
```

Example: Sanitise tool output before Claude reads it

`PostToolUse` hooks can replace a tool's output using `hookSpecificOutput.updatedToolOutput` before Claude processes it. Use this to strip secrets from command output:

```
#!/bin/bash
# .claude/hooks/sanitise-output.sh
input=$(cat)
tool=$(echo "$input" | jq -r '.tool_name // ""')

if [ "$tool" = "Bash" ]; then
  raw_output=$(echo "$input" | jq -r '.tool_result // ""')
  cleaned=$(echo "$raw_output" | sed 's/[A-Za-z0-9_-]\{20,\}/[REDACTED]/g')

  jq -n --arg out "$cleaned" '{
    hookSpecificOutput: {
      updatedToolOutput: $out
    }
  }'
  exit 0
fi

exit 0
```

Example: Desktop notifications for long-running tools

Hooks can emit desktop notifications using `terminalSequence` — no controlling terminal required:

```
#!/bin/bash
# .claude/hooks/notify.sh
input=$(cat)
tool=$(echo "$input" | jq -r '.tool_name // ""')
duration=$(echo "$input" | jq -r '.duration_ms // 0')

if [ "$duration" -gt 10000 ]; then
  jq -n --arg tool "$tool" --arg ms "$duration" '{
    hookSpecificOutput: {
      terminalSequence: "",
      desktopNotification: {
        title: "Claude Code",
        body: ($tool + " completed in " + $ms + "ms")
      }
    }
  }'
fi

exit 0
```

Example: HTTP hook to audit log

Send every tool call to an internal audit service:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "*",
        "hooks": [
          {
            "type": "http",
            "url": "https://audit.internal/claude-tool-log",
            "method": "POST",
            "headers": {
              "Authorization": "Bearer ${AUDIT_TOKEN}"
            }
          }
        ]
      }
    ]
  }
}
```

Custom Slash Commands: Reusable Prompts

> ***Note:*** *Custom commands (`.claude/commands/`) are the legacy approach and are now superseded by Skills. Commands still work for backward compatibility, but for new work, use Skills instead — they support auto-discovery, supporting files (`scripts/`, `references/`, `assets/`), and more frontmatter options. Migrate a command to a skill when it becomes complex enough to warrant supporting files or automatic invocation.*

Custom slash commands let you save prompt templates as files and invoke them with `/command-name [arguments]`. They are the simplest form of automation in Claude Code — no code, just a markdown file.

How to create custom commands

Create a markdown file in `.claude/commands/` (project) or `~/.claude/commands/` (personal):

```
your-project/
└── .claude/
    └── commands/
        ├── security-audit.md
        ├── write-tests.md
        └── pr-summary.md
```

The file content becomes the prompt. Use `$ARGUMENTS` to pass arguments from the command line:

```
<!-- .claude/commands/security-audit.md -->
Review $ARGUMENTS for security vulnerabilities.

Trace data flows end-to-end across related files — not just the file provided.

Check for:
- Injection risks: SQL, command injection, path traversal
- Authentication and authorisation gaps
- Sensitive data exposure in logs or responses
- Business logic flaws specific to this application
- Insecure Direct Object References

Rate each finding: critical / high / medium / low
Include: file path, line number, description, and why it is exploitable.
Do not make any code changes — report only.
```

Then run it:

```
/project:security-audit src/api/payments/
```

`$ARGUMENTS` is replaced by everything you type after the command name.

More command examples

Write tests for a component:

```
<!-- .claude/commands/write-tests.md -->
Write comprehensive tests for $ARGUMENTS.

Cover:
- Happy path with realistic inputs
- Edge cases: empty input, boundary values, nulls
- Error cases: invalid input, missing dependencies

Use the same test framework as the existing tests in this project.
Do not modify the source file.
```

Generate a PR description:

```
<!-- .claude/commands/pr-summary.md -->
Generate a pull request description for the current branch.

Run: git diff main...HEAD --stat
Then: git log main...HEAD --oneline

Write:
1. A one-line title (imperative, under 70 chars)
2. A 3-bullet summary of what changed and why
3. A testing checklist

Format as GitHub Markdown.
```

Explain a complex function:

```
<!-- .claude/commands/explain.md -->
Explain $ARGUMENTS in plain English.

Assume the reader is a senior engineer unfamiliar with this specific codebase.
Cover: what it does, inputs, outputs, side effects, and any non-obvious behaviour.
Keep it under 200 words.
```

When to use custom commands

Commands are for **repeatable prompt workflows you'd otherwise type from scratch each time**:

- Security audit templates you run before every release
- Test generation prompts tailored to your framework
- PR description formats specific to your team's style
- Onboarding prompts that explain specific parts of a codebase
- Incident review templates

The rule of thumb: if you've typed the same opening paragraph in a chat prompt more than twice, it belongs in a command file.

Commands vs CLAUDE.md instructions: Commands are explicit — you invoke them deliberately. CLAUDE.md instructions are ambient — they apply to every session. Use commands for tasks you run occasionally; use CLAUDE.md for behaviour you always want.

Skills: Context-Aware Automatic Instructions

Skills are the evolution of custom commands. A skill is a markdown file that Claude loads automatically when it is relevant to the current task — you do not need to invoke it manually. Claude reads skill descriptions at startup and pulls the full content when it determines the skill applies.

Skills are based on the **Agent Skills open standard** — designed to work across different AI tools, not just Claude Code. Claude Code adds extra features on top: better invocation control, sub-agent support, and dynamic context injection. A skill you write today is compatible with any tool that implements the same standard.

Skills vs custom commands

	Custom Commands	Skills
How invoked	You type `/command-name` explicitly	Claude invokes automatically, or you type `/skill-name`
When to use	Occasional, deliberate workflows	Recurring patterns Claude should handle without prompting
Frontmatter	None	Supported: model, tools, invocation, behaviour flags
Location	`.claude/commands/*.md`	`.claude/skills/<name>/SKILL.md`
Status	Legacy (still works)	Recommended

Progressive Disclosure: How Skills Stay Cheap

Skills use a “phone app” model for context efficiency. At session start, Claude loads only the **name and description** (~100 tokens) for every skill — like seeing app icons on your home screen. The full instructions only load when a skill is actually invoked.

This means you can have 20 skills defined without paying the context cost of 20 full instruction sets. A skill for database migration sits idle (just its 100-token description) until you actually ask Claude to write a migration, at which point its full body loads.

This is why the `description` frontmatter field is so important — it is the always-visible trigger. Write it precisely: “Invoked when the user asks to write, add, or generate tests.” A vague description means Claude either invokes the skill when it shouldn’t, or misses opportunities to invoke it when it should.

How to create a skill

Skills live in a named subdirectory under `.claude/skills/`, and can include supporting files alongside `SKILL.md`:

```
your-project/
└── .claude/
    └── skills/
        ├── test-writer/
        │   ├── SKILL.md            ← required: instructions
        │   ├── scripts/            ← optional: bash/python scripts Claude can run
        │   │   └── run-tests.sh
        │   ├── references/         ← optional: docs Claude can read
        │   │   └── vitest-api.md
        │   └── assets/             ← optional: templates, configs, fixtures
        │       └── test-template.ts
        ├── security-reviewer/
        │   └── SKILL.md
        └── migration-generator/
            ├── SKILL.md
            └── assets/
                └── migration-template.sql
```

The `scripts/`, `references/`, and `assets/` subdirectories mirror how a human expert thinks: they bring the tool (script), the manual (reference), and the starting material (asset) to the job. Claude can reference these files in skill instructions:

```
<!-- SKILL.md body -->
1. Run `scripts/run-tests.sh` to execute the test suite
2. Consult `references/vitest-api.md` for API details on complex matchers
3. Use `assets/test-template.ts` as the starting point for new test files
```

Each `SKILL.md` has a frontmatter block followed by the instruction body:

```
---
name: test-writer
description: Writes tests for TypeScript components using Vitest. Invoked when the user asks to write,
add, or generate tests.
user-invocable: true
allowed-tools:
  - Read
  - Edit
  - Bash
---

When writing tests for this project:

1. Use Vitest as the test framework (`import { describe, it, expect } from 'vitest'`)
2. Co-locate test files next to source: `component.ts` → `component.test.ts`
3. Cover: happy path, edge cases (null, empty, boundary), error conditions
4. Mock external dependencies using `vi.mock()` — never make real network calls in tests
5. Run `bun test` to verify tests pass before finishing

For React components, use `@testing-library/react` for rendering. For API handlers, test the handler
function directly without starting a server.
```

Skill frontmatter options

```
---
name: security-reviewer           # slug used for /skill-name invocation
description: >                    # shown at startup (~100 tokens) — be specific
  Reviews code for security vulnerabilities. Auto-invokes when
  the user asks to audit, review security, or check for vulnerabilities.
user-invocable: true              # can be called with /security-reviewer
disable-model-invocation: false   # true = skip LLM, run tools directly (for side-effecting skills)
allowed-tools:                    # restrict which tools this skill can use
  - Read
  - Bash
  - Grep
model: claude-opus-4-7            # override the model for this skill
agent: true                       # run as a subagent (own context window)
---
```

The `description` field is the most important — Claude reads all skill descriptions at session start and uses them to decide which skills to invoke automatically. Write the description as a clear trigger statement: “Invoked when the user asks to…”.

Skill invocation modes

Automatic invocation: Claude reads the description and decides the skill is relevant. If you ask “write tests for this component”, Claude will auto-invoke the `test-writer` skill without you typing anything.

Manual invocation: Type `/skill-name` to invoke it directly:

```
/security-reviewer src/api/checkout.ts
```

Disabled auto-invocation: Set `user-invocable: true` and omit the trigger language from the description if you only want manual invocation.

More skill examples

Migration generator for a specific ORM:

```
---
name: migration-generator
description: Generates database migrations using Drizzle ORM. Auto-invokes when the user asks to add a
column, create a table, or write a migration.
user-invocable: true
allowed-tools:
  - Read
  - Write
  - Bash
---

When generating a migration for this project:

1. Read `src/db/schema.ts` first to understand the current schema structure
2. Create migration files in `src/db/migrations/` using Drizzle's format
3. Name files: `NNNN_description.ts` where NNNN is the next sequential number
4. Always include both `up()` and `down()` functions
5. For large tables (check row count with `SELECT COUNT(*) FROM table`), add a comment noting the table
size and whether the migration will lock

Run `bun db:migrate --dry-run` to validate before finishing.
```

Kubernetes deployment helper:

```
---
name: k8s-deploy
description: Helps with Kubernetes deployments and kubectl operations. Auto-invokes when the user
mentions kubectl, deployments, pods, or Kubernetes.
user-invocable: true
allowed-tools:
  - Bash
  - Read
---

For Kubernetes operations in this project:

- Always use the `staging` namespace for tests: `kubectl -n staging`
- Production namespace is `prod` — require explicit confirmation before any prod changes
- Deployment manifests live in `k8s/` — read them before suggesting changes
- After deploying, verify with `kubectl rollout status deployment/<name>`
- If a rollout fails, run `kubectl rollout undo deployment/<name>` and report what happened

Never delete pods directly. Use `kubectl rollout restart` for restarts.
```

Code style enforcer:

```
---
name: style-guide
description: Enforces this project's code style rules. Auto-invokes when writing or editing TypeScript
files.
disable-model-invocation: false
allowed-tools:
  - Read
  - Edit
---

This project's TypeScript conventions:

- Use named exports only — no default exports
- Type everything explicitly — no implicit `any`
- Error handling: use `Result<T, E>` from `neverthrow` — no try/catch in business logic
- Async: always `async/await` — no raw Promise chains
- File names: `kebab-case.ts`
- Test files: co-located `component.test.ts`

When editing existing files, fix violations you encounter but do not refactor unrelated code.
```

When to use skills

Skills are for **recurring behaviours Claude should apply automatically without you saying so every time**:

- Project-specific conventions (your ORM, your test framework, your naming patterns)
- Workflows that follow the same steps every time (generate migration, verify, run dry-run)
- Tool restrictions for certain task types (security reviewer should only Read, not Edit)
- Specialised expertise that applies to a domain of tasks in this codebase

The decision rule: If you find yourself typing the same 3-10 line explanation at the start of a task ("when writing tests, use Vitest, co-locate files, and mock external deps"), that belongs in a skill. Claude will load it automatically next time.

Subagents: Parallel Context Windows

A subagent is a separate Claude instance with its own context window. It runs in parallel with your main session, investigates independently, and reports back a summary. This keeps noisy exploration out of your main conversation context.

Every subagent starts with a blank slate. It receives only its own system prompt (from the agent markdown file) and basic environment info like the working directory. It does **not** inherit the main session's conversation history, CLAUDE.md, or tool permissions. This is by design — the blank slate is what keeps subagents focused and prevents them from carrying the baggage of a long main conversation.

```
sequenceDiagram
    participant Dev as Developer
    participant Main as Main Session
    participant SA as Subagent

    Dev->>Main: Implement the new rate limiter
    Main->>SA: Spawn: read src/middleware/ and summarise the existing chain
    SA->>SA: Reads files, explores codebase independently
    SA-->>Main: Summary: 3 middleware layers, auth runs before rate limit
    Main->>Main: Uses summary to implement correctly
    Main-->>Dev: Rate limiter implemented, sits after auth middleware
```

How to create a subagent

The interactive path — open the agents menu inside Claude Code:

```
/agents
```

Select "Create New Agent", choose project-level or user-level, then select "Generate with Claude (recommended)". Provide a brief description of what the agent should do. Choose the tools it needs, model, and other config. Claude generates the file automatically.

The manual path — define subagents as markdown files in `.claude/agents/` (project-level) or `~/.claude/agents/` (user-level):

```
your-project/
└── .claude/
    └── agents/
        ├── security-reviewer.md
        ├── codebase-explorer.md
        └── doc-writer.md
```

Each agent file uses frontmatter for configuration and a body that becomes the system prompt:

```
---
name: security-reviewer
description: Reviews code for security vulnerabilities. Use when auditing auth, payment, or API code.
model: claude-opus-4-7
tools:
  - Read
  - Bash
  - Grep
  - Glob
isolation: worktree
---

You are a security-focused code reviewer. Your job is to read code and report vulnerabilities.

Rules:
- Read files thoroughly before drawing conclusions
- Trace data flows end-to-end across imports
- Report findings with: severity (critical/high/medium/low), file path, line number, description, and why
 it is exploitable in this specific codebase
- Do NOT make any code changes — report only
- If you find something critical, say so clearly at the top of your response
```

Subagent frontmatter options

```
---
name: codebase-explorer          # slug used to invoke this agent
description: >                   # shown to Claude when it decides which agent to spawn
  Fast read-only exploration agent. Use for "find where X is defined"
  or "which files reference Y" tasks.
model: claude-haiku-4-5          # use a cheaper/faster model for simple tasks
tools:                           # restrict available tools
  - Read
  - Bash
  - Grep
  - Glob
isolation: worktree              # give this agent its own git worktree (required if it writes files)
hooks:                           # agent-specific hooks
  PostToolUse:
    - matcher: "*"
      hooks:
        - type: command
          command: ".claude/hooks/agent-log.sh"
---
```

Built-in agent types you can reference without creating a file:

- Explore

  — fast read-only Haiku agent for locating code

- Plan

  — context-gathering agent that returns a structured plan

- general-purpose

— default Claude instance

Subagent Persistent Memory

Subagents can accumulate knowledge across sessions via the `memory` frontmatter field. When enabled, the agent reads and writes to its own memory files — storing patterns it discovers, recurring issues, design decisions, anything worth remembering for next time.

```
---
name: test-analysis-agent
description: Analyses test cases and tracks recurring failures
model: claude-sonnet-4-6
tools:
  - Read
  - Bash
memory: project
---

You are a testing specialist. Before analysing, check your memory
for known flaky tests and recurring failure patterns. After each
session, save new findings to memory so future runs benefit.
```

Memory scope options:

Value	What it means
`user`	Shared across all projects — personal learning
`project`	Scoped to this project only
`local`	Single-machine only, not synced

When memory is enabled, Claude automatically adds Read/Write/Edit to the agent's available tools and provides the first 200 lines of `MEMORY.md` in every session. Guide the agent explicitly: "Check your memory before starting" and "Save important findings to memory when done."

Foreground vs Background Execution

Subagents can run in two modes depending on how much visibility you want:

	Foreground	Background
Main session	Waits until agent finishes	Continues while agent runs
Permissions	Shown to you as they arise	All requested upfront before start
Visibility	Full — you see everything	Minimal — summary at the end
Best for	Tasks needing oversight	Long-running, fire-and-forget tasks

For background execution, Claude asks for all required permissions before the subagent starts. After you approve, it runs automatically without further interruption.

Invoking subagents

From a prompt:

```
Use a subagent to investigate the payment service.
Read only: src/services/payment/, src/models/order.ts
Task: summarise the data flow from order creation to payment confirmation.
Report back a concise summary — do not make any changes.
```

By name (if you have defined a .claude/agents/ file):

```
Spawn the security-reviewer agent on src/api/checkout.ts
```

From Claude Code CLI (for scripting):

```
claude --agent security-reviewer -p "Audit src/api/checkout.ts" --output-format json
```

Worktree isolation for parallel writes

If a subagent needs to edit files, add `isolation: worktree` to prevent conflicts with the main session or other parallel subagents:

```
---
name: refactor-agent
description: Refactors a module independently
model: claude-sonnet-4-6
tools:
  - Read
  - Edit
  - Bash
isolation: worktree
---

You are a refactoring agent. You receive a module path and a refactoring goal.
Apply the refactoring, run the tests, and report what you changed.
```

Each agent with `isolation: worktree` gets its own git branch and working tree. Changes are isolated until you merge them. This is the safe default for any subagent that writes files.

Example: Parallel specialist team for code review

This is the divide-and-conquer pattern that turns Claude Code from a single agent into a team. Instead of one review from one perspective, you spin up three specialists in parallel:

```
Spawn three subagents in parallel to review src/api/payments/:

1. Use the security-reviewer agent — audit for injection vulnerabilities,
   auth gaps, and data exposure. Report findings with severity ratings.

2. Use the performance-reviewer agent — identify N+1 queries, missing indexes,
   synchronous operations that should be async, and inefficient data structures.

3. Use the style-reviewer agent — check naming consistency, error handling patterns,
   test coverage, and adherence to our API conventions.

When all three finish, compile a consolidated report with findings grouped by severity.
```

Define each specialist in `.claude/agents/`:

```
---
name: security-reviewer
description: Security-focused reviewer. Audits auth, injection, data exposure.
model: claude-opus-4-8
tools: [Read, Bash, Grep]
---
You are a security specialist. Review for: injection risks, auth gaps, sensitive data
in logs/responses, IDOR, insecure dependencies. Rate each finding critical/high/medium/low.
Report file:line references. Do NOT make any code changes.
```

```
---
name: performance-reviewer
description: Performance-focused reviewer. Finds N+1s, slow queries, blocking calls.
model: claude-sonnet-4-6
tools: [Read, Bash, Grep]
---
You are a performance specialist. Review for: N+1 queries, missing database indexes,
synchronous I/O that should be async, large in-memory operations, inefficient algorithms.
Estimate impact (latency, memory). Do NOT make any code changes.
```

```
---
name: style-reviewer
description: Code style and convention reviewer. Checks naming, tests, API consistency.
model: claude-haiku-4-5
tools: [Read, Grep]
---
You are a code style specialist. Review for: naming consistency, test coverage gaps,
error handling patterns, API consistency with existing endpoints.
Do NOT make any code changes.
```

You get three deep reviews in the time of one. Each agent has a fresh context — not biased toward code it just wrote — and specialised focus that produces sharper findings than a general review.

When to use subagents

Subagents are for **self-contained tasks where the main session only needs a summary, not the full investigation**:

- Exploring a large codebase to answer a specific question before you start implementing
- Running a security or code review scan while you continue other work
- Fetching and summarising external documentation or API specs
- Running parallel implementations of the same feature (different approaches, compare results)
- Any task that would fill your context window with output you do not need long-term

When not to use subagents: If the task requires tight coordination with your main session (shared state, sequential steps), run it in the main session instead. Subagent overhead is not worth it for small tasks.

Example subagents for common workflows

Dependency auditor:

```
---
name: dep-auditor
description: Audits project dependencies for vulnerabilities and outdated packages.
model: claude-sonnet-4-6
tools:
  - Bash
  - Read
---

Run the following and report findings:
1. `npm audit --json` — parse and summarise CVEs by severity
2. `npm outdated` — list major-version outdated packages
3. Read `package.json` — flag any packages that are known to be deprecated

Return a structured report: critical issues first, then high, then informational.
Do not make any changes.
```

Architecture explorer:

```
---
name: arch-explorer
description: Reads the codebase and produces an architecture summary. Use before starting large features.
model: claude-haiku-4-5
tools:
  - Read
  - Glob
  - Bash
---

You are a read-only codebase explorer. Your goal is to produce a concise architecture summary.

Read:
1. Top-level directory structure
2. Package.json / go.mod / requirements.txt — understand the tech stack
3. Main entry points
4. Key data models

Return: tech stack, main modules, data flow from request to response, anything non-obvious.
Keep the summary under 400 words. Do not make any changes.
```

Piping: Claude in Your Shell Workflows

The `-p` flag makes Claude Code headless — single prompt in, stdout out. This enables Claude to participate in shell pipelines like any other Unix tool.

Basic piping

```
# Analyse log output
docker logs myapp --tail 200 | claude -p "what errors are here and what is causing them?"

# Summarise git history
git log --oneline -20 | claude -p "summarise what changed this week in plain English"

# Review a diff before committing
git diff --staged | claude -p "review this diff for bugs or issues before I commit"
```

Chaining Claude calls

```
# Generate migration, then review it
claude -p "generate a SQL migration to add a soft-delete column to the users table" \
  > migration.sql

claude -p "review this migration for safety — will it lock the table? Is rollback safe?" \
  < migration.sql
```

Integrating with CI

```
# Post a natural-language summary of a deploy to Slack
deploy_output=$(./deploy.sh 2>&1)
summary=$(echo "$deploy_output" | claude -p "summarise this deploy output in 2 sentences for a Slack
message")
curl -X POST "$SLACK_WEBHOOK" -d "{\"text\": \"$summary\"}"
```

Daily standup from git log

```
#!/bin/bash
# scripts/slack-standup.sh
diff=$(git log --since="yesterday" --oneline --all)
update=$(echo "$diff" | claude -p "write a casual 3-bullet Slack standup update from these commits. First
 person, past tense, no jargon.")
echo "$update"
```

When to Use Which: Decision Framework

```
flowchart TD
    Q1{Must this happen\nevery time?} -->|Yes| Hook[Hook]
    Q1 -->|No| Q2{Is it a repeatable\nprompt you type often?}
    Q2 -->|Yes, you invoke it| Cmd[Custom Command]
    Q2 -->|Yes, Claude should auto-detect| Skill[Skill]
    Q2 -->|No| Q3{Is the task noisy,\nlong, or parallel?}
    Q3 -->|Yes| SA[Subagent]
    Q3 -->|No| Main[Main session prompt]
```

In plain terms:

- Hook

  — “Run

  prettier

  on every file Claude edits. No exceptions.”

- Custom command

  — “I write the same security audit prompt every sprint. Save it as

  /security-audit

  .”

- Skill

  — “Every time Claude writes TypeScript in this project, it should follow our specific conventions — but I don’t want to type them every time.”

- Subagent

— “Before I implement this feature, have Claude read the entire legacy module and give me a summary. I don’t need all that detail in my main session.”

- Piping

  — “Pipe

  docker logs

  into Claude to classify errors without opening an interactive session.”

They are not mutually exclusive. The most powerful patterns combine them:

```
flowchart TD
    Edit[Claude edits a file] --> Hook[PostToolUse hook fires]
    Hook --> Tests[Run test suite]
    Tests --> Fail{Tests pass?}
    Fail -->|Yes| Continue[Claude continues working]
    Fail -->|No| SA[Spawn arch-explorer subagent]
    SA --> SA2[Subagent reads test output + edited file]
    SA2 --> Fix[Subagent suggests fix]
    Fix --> Main[Main session applies fix]
    Main --> Edit
```

Getting Started Checklist

Start here — in this order. Each step adds immediate value before you move to the next:

1. Add the secrets scanner hook

   — the most important safety net; blocks accidental credential commits before they happen

2. Add the file shield hook

   — protect

   .env

   and other sensitive files from AI edits entirely

3. Add destructive command block

   — prevents

   rm -rf

   ,

   terraform destroy

   , and

   kubectl delete

   accidents

4. Add auto-format hook

   — low risk, immediate value, no false positives

5. Create one custom command

   — save a prompt you type repeatedly (security audit template, PR description format)

6. Try the test-on-edit hook

   — only in a project with fast tests (<10s total)

7. Create one skill

   — start with your project's test conventions or deployment workflow

8. Try one subagent investigation

   — before implementing a feature, not during; get the summary, then implement

9. Try a parallel specialist review

   — spin up security + performance + style reviewers on your next PR

10. Add `isolation: worktree`

    to any subagent that writes files

You do not need all of these at once. The first three hooks add immediate safety with no downside. Add the rest when you feel the friction each one solves.

Claude Code Skills: Building Specialized AI Workflows

Published 2026-06-01 | Source: https://blog.devops-monk.com/2026/06/claude-code-skills-guide/

Master Claude Code Skills — progressive disclosure, custom task automation, and specialized workflows for developers, testers, and DevOps engineers.

A Skill in Claude Code is a specialized workflow that Claude can invoke automatically or you can trigger manually with a slash command. Unlike CLAUDE.md rules (which advise on every decision) or Hooks (which enforce deterministic actions), Skills are **progressive disclosure** — deep knowledge that loads only when relevant, keeping your base context lean while providing specialized expertise when needed.

This guide covers how Skills work, when to use them, and how to build real Skills that serve developers, testers, and DevOps engineers.

What Is a Skill?

A Skill is a `.md` file that teaches Claude how to do something specific — deeply. It lives in `.claude/skills/` and contains:

- A frontmatter header with metadata (name, description, allowed tools)
- Detailed instructions for a workflow or task type
- Examples and step-by-step guidance

Skills differ from other Claude Code tools:

Tool	Purpose	Loads when
CLAUDE.md	Rules that apply to every decision	Every session, always
Skills	Deep knowledge for specific workflows	You invoke them, or automatically when relevant
Hooks	Deterministic automation (no exceptions)	Specific lifecycle events (PreToolUse, PostToolUse, etc.)
Subagents	Isolated specialist assistants	You delegate a task to them

Skills are the sweet spot: they contain knowledge that would bloat CLAUDE.md if always loaded, but are specialized enough that they shouldn't be in the base system prompt.

Skill Structure

Create a skill in `.claude/skills/my-skill-name/SKILL.md`:

```
.claude/
└── skills/
    └── my-skill-name/
        └── SKILL.md
```

Inside `SKILL.md`, start with frontmatter:

```
---
name: my-skill-name
description: What this skill does in one sentence
user-invocable: true              # can you call /my-skill-name manually?
allowed-tools:                    # restrict what tools Claude can use
  - Read
  - Bash
  - Edit
disable-model-invocation: true    # optional: for workflows with side effects you want to trigger
explicitly
---

# Skill: My Skill Name

[Your detailed instructions here]
```

Frontmatter Fields

Field	Purpose	Example
`name`	Skill identifier (used in `/skill-name` command)	`run-tests`
`description`	One-sentence summary shown in help	`Run tests and report coverage`
`user-invocable`	Can you manually invoke this with `/skill-name`?	`true` or `false`
`allowed-tools`	Restrict which tools Claude can use	`[Bash, Read, Edit]`
`disable-model-invocation`	Require explicit trigger (no auto-invocation)	`true` for side-effect workflows

---

Invoking Skills

Manually — type the slash command:

```
/my-skill-name
/my-skill-name with arguments
```

Automatically — Claude recognizes when the skill is relevant and invokes it without you asking. To enable auto-invocation, include trigger patterns in your skill description:

```
---
name: run-tests
description: >
  Run the test suite and report coverage.
  Auto-invokes when the user asks to "run tests", "test this", or "check coverage".
user-invocable: true
---
```

When Claude sees a prompt like “run the test suite,” it recognizes the skill matches and invokes it.

Real Skill Examples for Each Audience

Skill 1: Test Runner (For Developers)

Developers need to run tests frequently but with nuance — sometimes you want a single test file, sometimes the full suite, sometimes only tests matching a pattern.

```
---
name: run-tests
description: >
  Run the test suite with filtering and coverage reporting.
  Auto-invokes when asked to "run tests", "test this", or "check coverage".
user-invocable: true
allowed-tools:
  - Bash
  - Read
---

# Test Runner Skill

## Workflow

1. Understand what the user wants to test (all tests, single file, pattern match)
2. Run the appropriate test command
3. Parse the output and report results
4. Show coverage if requested

## Commands

```bash
# Run all tests
npm test

# Run single file
npm test -- path/to/test.ts

# Run tests matching a pattern
npm test -- --testNamePattern "auth"

# Run with coverage
npm test -- --coverage
```

Steps

1. Ask for clarification if needed: “Which tests? All, a specific file, or a pattern?”
2. Run the command using the appropriate filter
3. Interpret output:
   - Passing: list count and mark ✓
   - Failing: show assertion errors
   - Coverage: show percentage and uncovered lines
4. Suggest next steps:
   - Tests pass: “All passing. Coverage 87%. Ready to commit.”
   - Tests fail: “3 failed. Here’s what broke. Fix before committing.”
```

````
### Skill 2: Cross-Browser Test Coordinator (For Testers)

Testers need to run tests across multiple browsers and capture screenshots for comparison.

```markdown
---
name: cross-browser-test
description: >
  Run tests across Chrome, Firefox, and Safari using Playwright.
  Auto-invokes when asked to "test across browsers" or "cross-browser test".
user-invocable: true
allowed-tools:
  - Bash
  - Read
  - Edit
---

# Cross-Browser Test Coordinator Skill

Test the same flows against:
- Chrome (desktop)
- Firefox (desktop)
- Safari (desktop)
- Mobile Chrome (simulated iPhone)

## Workflow

1. Read the test file to understand test cases
2. Run across all browser configurations:
```bash
npx playwright test tests/e2e/checkout.spec.ts --project=chromium
npx playwright test tests/e2e/checkout.spec.ts --project=firefox
npx playwright test tests/e2e/checkout.spec.ts --project=webkit
npx playwright test tests/e2e/checkout.spec.ts --project="Mobile Chrome"
````

1. Capture and compare screenshots
2. For visual differences:
   - List which browsers show differences
   - Show screenshot diffs
   - Recommend: update baseline if intended, or investigate if unexpected
3. Report structured results:

```
✓ Chrome — 3 passed
✓ Firefox — 3 passed
⚠ Safari — 2 passed, 1 visual diff
⚠ Mobile Chrome — 2 passed, 1 layout shift

Action items:
- Safari button styling: investigate CSS
- Mobile layout: review responsive breakpoints
```

````
### Skill 3: Infrastructure Audit (For DevOps)

DevOps engineers need to audit infrastructure for security, compliance, and drift.

```markdown
---
name: infra-audit
description: >
  Audit Terraform/Kubernetes for security issues, compliance violations, and drift.
  Auto-invokes when asked to "audit infrastructure" or "security check".
user-invocable: true
allowed-tools:
  - Bash
  - Read
  - WebFetch
---

# Infrastructure Audit Skill

## Audit checklist

### Security
- No hardcoded secrets (passwords, API keys, credentials)
- All network traffic encrypted in transit (TLS/HTTPS)
- IAM policies follow least-privilege (no `*` permissions)
- Database backups are encrypted and off-site
- VPC security groups restrict inbound to necessary ports only

### Compliance
- All resources tagged with owner, cost center, environment
- Logging enabled on all data stores (CloudTrail, VPC Flow Logs, RDS logs)
- Retention policies comply with regulatory requirements (GDPR, SOC2, etc.)
- Encryption keys managed (not inline, use AWS KMS or HashiCorp Vault)

### Operational
- No single point of failure (replicas/multi-AZ for critical services)
- Resource limits and requests configured (prevents node exhaustion)
- Monitoring/alerting configured for critical metrics
- Disaster recovery plan documented and tested

## Workflow

1. Gather infrastructure code:
```bash
find infra -name "*.tf"
find k8s -name "*.yaml"
````

1. Scan Terraform for:
   - Hardcoded values
   - Missing tags
   - Overly broad IAM permissions
   - Encryption settings
2. Scan Kubernetes for:
   - Resource requests/limits
   - Image registries (no public registries for private code)
   - Secrets handling
   - RBAC policies
3. Check for drift:

```
terraform plan -out=plan.json
```

1 Generate report:

```
## Infrastructure Audit Report — [DATE]

**Status**: [✓ Secure | ⚠ Warnings | ✗ Critical]

### Critical Issues
- Issue 1: [Impact] — Fix: [Action]

### Warnings
- Warning 1: [Impact] — Recommendation: [Action]

### Passing checks
- ✓ Secrets in AWS Secrets Manager
- ✓ Multi-AZ RDS
- ✓ Security groups restricted
- ✓ All resources tagged

### Action items
1. [Priority 1 fix]
2. [Priority 2 fix]
```

```
---

## When to Use Skills

**Use a Skill when:**
- The workflow is complex (more than 5 steps)
- You use it repeatedly across projects
- It requires deep knowledge that would bloat CLAUDE.md
- Different team members need different variants

**Don't use a Skill when:**
- It's a simple one-liner that CLAUDE.md can cover
- It's too specialized to one project
- It's a rule that must always apply (use CLAUDE.md instead)
- It's a deterministic action with zero exceptions (use Hooks)

---

## Skills vs Other Tools

| Tool | Best for | Invoked how | Token cost |
|------|----------|------------|-----------|
| **CLAUDE.md** | Conventions and rules | Always active | High (always loaded) |
| **Skills** | Complex multi-step workflows | Manual `/skill-name` or auto | Low (loads on demand) |
| **Hooks** | Deterministic enforcement | Specific events (no choice) | Medium (fires on events) |
| **Subagents** | Isolated specialists | Explicit delegation | Medium (separate context) |

**Decision tree:**
- "I want Claude to always follow this rule" → CLAUDE.md
- "I want this workflow automated but invoked on demand" → Skill
- "I want this to happen every time with no exceptions" → Hook
- "I want a specialist assistant with fresh context" → Subagent

---

## Tips for Effective Skills

**1. Be specific about what the skill does**

 Bad: "Development helper"
✓ Good: "Run unit tests, report failures, and suggest fixes"

**2. Use allowed-tools to restrict access**

```markdown
---
allowed-tools:
  - Read
  - Bash(npm test)
  - Bash(npm run build)
---
```
```

Claude can only run those specific commands.

3. Include examples in the skill

Show expected output format so Claude knows what you want to see.

4. Test the skill first

```
/skill-name test-arg
```

Verify it works as expected before relying on it.

Sharing Skills Across Projects

For skills you use everywhere:

```
mkdir -p ~/.claude/skills/my-skill-name
cp your-skill-content ~/.claude/skills/my-skill-name/SKILL.md
```

Global skills load in every project.

Common Mistakes

1. Skills too long (>500 lines)

Break into smaller skills. A 500-line skill means loading too much context.

2. Auto-invocation too broad

“Auto-invokes when user mentions testing” is too broad. Be specific: “when user asks to ‘run tests’ or ’test this’”.

3. No error handling

Include contingency steps:

- “What if the test file doesn’t exist?”
- “What if the deployment fails?”

4. Mixing concerns

One skill per responsibility. Don’t combine “test, build, lint, deploy” — make four separate skills.

Claude Code Skills bridge general guidance (CLAUDE.md) and deterministic automation (Hooks). Invest in well-written Skills and your daily workflows become significantly faster.

Claude Code Sub-Agents: Specialist Assistants and Delegation

Published 2026-06-05 | Source: https://blog.devops-monk.com/2026/06/claude-code-subagents-delegation/

Master sub-agents in Claude Code — creating specialized assistants, delegation patterns, persistent memory, and parallel task execution for developers, testers, and DevOps.

A sub-agent is a specialized Claude instance that runs in an isolated context window with its own allowed tools, memory, and instructions. While your main Claude session handles the conversation, a sub-agent spins up to handle a specific task — investigation, code review, security audit, testing — in parallel with your work.

Sub-agents excel at tasks that require:

- Reading many files without cluttering main context
- Specialized expertise in one domain
- Fresh perspective (the agent isn't biased by earlier conversation)
- Parallel execution (multiple agents working simultaneously)

Sub-Agents vs Skills vs Hooks

Before diving into sub-agents, understand the distinctions:

Tool	Purpose	Isolation	Context	Best for
Skills	Complex workflows in your session	None — same context	Shared	Sequential tasks you invoke
Sub-Agents	Specialized tasks in separate context	Full isolation	Fresh window	Parallel work, fresh perspective
Hooks	Deterministic side effects	None — inline	Minimal	Automatic enforcement (no exceptions)
CLAUDE.md	Behavioral rules for main session	None — always loaded	Shared	Universal conventions

Sub-agents are for: "I need a specialist to review this code while I continue coding." **Skills are for:** "I need to run a complex workflow in my current session." **Hooks are for:** "I need this to happen automatically every time."

How Sub-Agents Work

When you delegate a task to a sub-agent:

1. **Spin up** — Claude Code creates a new isolated session with:
   - Fresh context window (no conversation history pollution)
   - Specified allowed tools (restricted permissions)

- Custom instructions (the agent.md frontmatter)
- Optional: isolated git worktree (separate branch)

2. **Run** — The agent executes independently:
   - Reads files, runs commands, analyzes code
   - Makes changes within its scope
   - Maintains its own memory (not shared with main session)
   - Reports progress and findings
3. **Return** — Agent reports back:
   - Summary of what was found/done
   - Specific file:line references
   - Recommendations for the main session
   - Optional: opens a PR with proposed changes
4. **Cleanup** — Agent context is freed:
   - Session ends, context released
   - Any git worktree is cleaned up (unless you keep it)
   - Main session continues with the agent's findings

Creating Sub-Agents

Sub-agents are defined in `.claude/agents/` directory:

```
.claude/
└── agents/
    ├── code-reviewer.md
    ├── security-auditor.md
    └── test-orchestrator.md
```

Each agent is a markdown file with frontmatter + instructions:

```
---
name: code-reviewer
description: Reviews code changes for bugs, style, and best practices
tools: Read, Edit, Bash
model: claude-opus-4-7
disabled: false
isolation: none
---

You are a senior code reviewer. Your job is to:

1. Read the proposed code changes
2. Identify bugs, edge cases, and improvements
3. Check for adherence to project conventions
4. Provide specific file:line feedback

Focus on:
- Logic errors and potential runtime failures
- Security vulnerabilities
- Performance issues
- Test coverage gaps
- Style/convention violations

Provide findings in this format:

## Findings

### Critical Issues (must fix)
- [file:line] — [Issue] — [Why it matters]

### Improvements (should consider)
- [file:line] — [Suggestion] — [Benefit]

### Passing checks
- Code follows all project conventions
- Error handling is consistent
- Tests cover happy path and edge cases
```

Frontmatter Fields

Field	Purpose	Example
`name`	Agent identifier	`code-reviewer`
`description`	What the agent does	`Reviews code for bugs and style`
`tools`	Allowed tools (comma-separated)	`Read, Edit, Bash, WebFetch`
`model`	Claude model to use	`claude-opus-4-7` (for complex tasks)
`disabled`	Is this agent disabled?	`false`
`isolation`	Worktree isolation?	`none` or `worktree`

Real Sub-Agent Examples

Agent 1: Code Reviewer (For Developers)

Developers benefit from fresh-perspective code review without blocking their own work.

````
---
name: code-reviewer
description: Reviews code changes for bugs, style, and best practices
tools: Read, Edit, Bash
model: claude-opus-4-7
isolation: none
---

# Code Reviewer Agent

You are a senior code reviewer conducting a thorough review of code changes.

## Your task

Read the current git diff and identify:

1. **Logic errors** — code that will fail at runtime or produce wrong results
2. **Security issues** — injection, auth bypass, data exposure
3. **Edge cases** — scenarios the code doesn't handle
4. **Style violations** — deviations from project conventions
5. **Test gaps** — scenarios not covered by tests

## How to proceed

```bash
git diff HEAD~1
````

For each changed file, review:

- Function/method signatures (unchanged contract?)
- Error handling (all failure paths covered?)
- Resource cleanup (memory, file handles, connections?)
- Concurrency (race conditions if async?)
- Type safety (TypeScript types sound?)

Report format

Code Review — [COMMIT_HASH]

Critical Issues (must fix before merge)

- [file:line]

  functionName()

  — [Issue] — [Why]

Improvements (should consider)

- [file:line] — [Suggestion] — [Benefit]

Passing checks

✓ No logic errors detected ✓ Error handling is comprehensive ✓ No security vulnerabilities ✓ All tests passing
````

````
### Agent 2: Test Orchestrator (For Testers)

Testers need to run parallel test suites, generate reports, and identify flaky tests.

```markdown
---
name: test-orchestrator
description: Runs parallel test suites, analyzes results, and identifies flaky tests
tools: Bash, Read, Edit
model: claude-sonnet-4-6
isolation: none
---

# Test Orchestrator Agent

You orchestrate test execution across multiple suites in parallel.

## Your task

Run all test suites and generate a comprehensive report:

```bash
npm test -- --coverage
npm run test:e2e
npm run test:integration
````

Analysis

For each test run:

1. Count passed/failed
2. Identify new failures vs previously failing
3. Check coverage trends
4. List flaky tests (tests that pass/fail inconsistently)

Report format

Test Report — [DATE]

Summary

- Total tests: XXX
- Passing: XXX (XX%)
- Failing: XXX
- Skipped: XXX

By suite

- Unit tests

  : XXX passed, XX% coverage

- E2E tests

  : XXX passed

- Integration

  : XXX passed

Failures

- [test name] — [reason] — [last 5 runs: pass/fail pattern]

Flaky tests detected

- [test] — fails 20% of the time — investigate: [suggestion]

Coverage gaps

- [file] — [uncovered lines] — [recommended tests]

````
### Agent 3: Infrastructure Auditor (For DevOps)

DevOps teams benefit from continuous infrastructure audits without blocking deployments.

```markdown
---
name: infra-auditor
description: Audits infrastructure for security, compliance, and cost optimization
tools: Bash, Read, WebFetch
model: claude-opus-4-7
isolation: none
---

# Infrastructure Auditor Agent

You audit infrastructure configurations for security, compliance, and operational health.

## Your task

Scan all infrastructure code and live resources:

```bash
# Scan Terraform
terraform plan -out=plan.json

# Scan Kubernetes
kubectl get all -A

# Check logs
kubectl logs -l app=myapp --tail=50
````

Security checks

- No hardcoded secrets (passwords, API keys, tokens)
- All storage encrypted at rest
- All traffic encrypted in transit (TLS 1.2+)
- IAM policies follow least-privilege
- Logging enabled on all sensitive operations
- No public-facing databases

- Backups automated and tested

Compliance checks

- Resources tagged appropriately
- Audit logging enabled
- Retention policies enforced
- Data residency compliant

Operational health

- Multi-AZ/replicas for critical services
- Resource limits configured
- Monitoring/alerting active
- Disaster recovery tested recently

Report format

Infrastructure Audit Report — [DATE]

Security Status: [✓ Secure | ⚠ Warnings | ✗ Critical]

Critical Issues (fix immediately)

- [Resource] — [Issue] — [Risk] — [Fix]

Warnings (plan fixes)

- [Resource] — [Issue] — [Recommendation]

Passing checks

✓ All secrets managed via HashiCorp Vault ✓ Multi-AZ on all critical services ✓ Encryption enabled across storage and transit ✓ Logging configured with 90-day retention

Cost optimization opportunities

- [Resource] — [Opportunity] — [Estimated savings]

````
---

## Invoking Sub-Agents

**Method 1: Simple delegation**

```bash
Use the code-reviewer agent to review the changes in this PR
````

Claude automatically creates the agent, gives it context about the PR, and runs it.

Method 2: Explicit task assignment
````

```
Run the test-orchestrator agent to execute all test suites and report results
```

Method 3: Programmatic via Claude Code SDK

For advanced workflows, spawn agents directly in code.

Sub-Agent Memory

Sub-agents maintain their own memory files separate from your main session:

```
~/.claude/projects/my-project/agents/
├── code-reviewer/
│   └── memory/
│       └── MEMORY.md
├── test-orchestrator/
│   └── memory/
│       └── MEMORY.md
└── infra-auditor/
    └── memory/
        └── MEMORY.md
```

Each agent learns from its own sessions:

- Code Reviewer

  learns project conventions, common mistakes

- Test Orchestrator

  learns test infrastructure, flaky patterns

- Infrastructure Auditor

  learns infrastructure decisions, compliance requirements

This specialization means agents improve at their tasks over time without cluttering main session memory.

Worktree Isolation

For agents that make code changes, use worktree isolation:

```
---
name: automated-fixer
isolation: worktree
---
```

This causes the agent to:

1. Create a git worktree (separate branch + folder)
2. Run in that isolated environment
3. Make changes without affecting your main branch
4. Open a PR when done
5. Clean up the worktree

Great for: automated fixes, refactoring, migrations.

Parallel Execution Patterns

Pattern 1: Review While You Code

```
# Main session: implementing a feature
You: "implement OAuth2 authentication"
Claude: [your session writes code]

# Meanwhile, delegate review
You: "have the code-reviewer agent review my changes as I go"
Agent: [runs in background, reports findings when done]
```

Pattern 2: Multi-Agent Audit

```
# Run security and compliance audits in parallel
You: "run security-auditor and compliance-auditor agents on infrastructure"

Agent 1 (Security): scans for vulnerabilities
Agent 2 (Compliance): checks regulatory requirements
Both: report findings simultaneously
```

Pattern 3: Test Orchestration

```
# Run different test suites in parallel
You: "run test-orchestrator to execute unit, e2e, and integration tests"

Agent:
  - Spins up 3 parallel test runners
  - Aggregates results
  - Identifies flaky tests
  - Generates report
```

When to Use Sub-Agents

Use a sub-agent when:

- Task requires reading many files (would pollute main context)
- You want a fresh perspective (not biased by main conversation)
- Task can run in parallel with your work
- Task needs specialized expertise (security audit, performance analysis)
- Task needs to make changes in isolation (use worktree)

Don't use a sub-agent when:

- The task is simple and fits in main context
- You need immediate feedback (agent spin-up adds latency)
- Task must read your real-time conversation context
- Task is one-time-only and not worth specialization

Tips for Effective Sub-Agents

1. Make agent scope clear and bounded

Good: “Review code for security vulnerabilities” Bad: “Do a general code review”

2. Specify exactly what tools they need

```
tools: Read, Bash(npm test), Bash(terraform plan)
```

Don’t give `Bash` without restrictions unless necessary.

3. Use the right model for the task

- claude-haiku-4-5

  — quick checks, simple analysis
- claude-sonnet-4-6

  — standard tasks, good balance
- claude-opus-4-7

  — complex reasoning, security, architecture

4. Include output format in instructions

Agents should produce output you can easily parse and act on. Structured format (markdown tables, JSON) beats prose.

5. Test the agent profile first

```
Use the code-reviewer agent to review src/auth.ts
```

Verify it produces useful output before relying on it.

Common Mistakes

1. Over-scoping agent responsibility

Agent becomes “do everything” instead of “do one thing well.”

2. Not specifying allowed tools

Agent gets too much access. Always restrict to necessary tools.

3. Agents with very different models

Don’t mix Haiku and Opus for the same type of task unless there’s a reason. Keep consistency.

4. Ignoring agent memory

Agents improve over time. Check their memory files to see what they’ve learned about your project.

5. Not using worktree isolation for changes

Agent makes code changes in your branch. Use `isolation: worktree` to prevent conflicts.

Sub-Agents in Teams

For team-wide specialists:

```
# Store agents in version control
.claude/agents/
├── security-reviewer.md     # team standard
├── performance-analyzer.md  # team standard
└── compliance-checker.md    # team standard
```

All team members get the same agent definitions. Agents learn individually but apply team standards.

Monitoring Agent Progress

When an agent is running, Claude Code shows:

```
[Sub-agent: code-reviewer] — Reading files...
[Sub-agent: code-reviewer] — Analyzing patterns...
[Sub-agent: code-reviewer] — Generating report...
[Sub-agent: code-reviewer] — Complete
```

Check `/agents` to see active and completed agents in current session.

Sub-agents unlock parallel work, fresh perspective, and specialization. Invest in well-defined agents and your team’s quality and speed both improve significantly.

Claude Code Session Management: Parallel Work and Persistent Context

Published 2026-06-08 | Source: https://blog.devops-monk.com/2026/06/claude-code-session-management/

Master session management in Claude Code — directory-scoped sessions, git worktrees, session resumption, and five developer habits that scale your productivity.

A Claude Code session is your working context — conversation history, file state, permissions, and memory. Understanding how sessions work and how to manage them unlocks the ability to parallelize work, switch contexts without losing state, and recover from mistakes instantly.

This guide covers session fundamentals, parallel work patterns, and five developer habits that compound over time.

What Is a Session?

A session is:

- Directory-scoped, not branch-scoped

  — tied to a folder location, not a git branch
- Persistent across context limits

  — survives

  /compact

  and

  /clear
- Shareable

  —

  /rename

  and

  /resume

  to switch between named sessions
- Isolated

  — each session has its own context window, permissions, memory

Critical distinction: Sessions follow your **directory**, not your **branch**. When you `git checkout feature-x`, the same session continues — Claude reads files from the new branch, but the conversation history stays. This is intentional and useful (you remember context), but it means you need separate sessions for parallel work on different branches.

Session Lifecycle

```
1. Start
   └→ claude                          [new session in current directory]
       └→ claude -c                   [resume last session]
       └→ claude --resume name        [resume by name]

2. Work
   └→ Type prompts
   └→ Claude reads/edits files from current branch
   └→ Memory is written automatically

3. Pause
   └→ /rename my-task                [name the session]
   └→ Cmd+D or exit                   [pause without closing]

4. Resume
   └→ claude -c                       [same session, full history]
   └→ claudeSession list              [see all sessions]

5. Cleanup
   └→ Cmd+D                           [close session]
   └→ Sessions auto-clean after 7 days of inactivity (configurable)
```

Directory-Scoped Sessions

Sessions are tied to **directories**, not branches. This matters:

```
cd ~/myrepo
git checkout main
claude                    # Session A starts, reads main branch

git checkout feature-x
# Same Session A continues, now reads feature-x branch

git checkout main
# Still Session A, now reads main branch again
```

Advantage: You remember context across branches **Disadvantage:** If you're working on multiple branches *simultaneously*, you need separate sessions

---

Git Worktrees: Parallel Sessions

For true parallel work on multiple branches, use git worktrees:

```
# Terminal 1: Main branch
cd ~/myrepo
claude                    # Session A for main branch

# Terminal 2: Feature branch (in a new worktree)
claude --worktree feature-auth     # Creates worktree + new session B
# New folder created: ~/myrepo/.claude/worktrees/feature-auth
# New branch created: worktree-feature-auth
# Session B opens inside that isolated folder

# Terminal 3: Bug fix branch (another worktree)
claude --worktree bugfix-api       # Creates worktree + new session C
```

Each worktree has:

- Separate folder

  — files are independent copies

- Separate branch

  — changes never conflict
- Separate session

  — different conversation history and context
- Same git history

  — all worktrees share the repository's git log and remotes

Workflow:

```
Main Branch (Terminal 1)          Feature Branch (Terminal 2)
│                                 │
├─ Session A                       ├─ Session B
│  ├─ Working on refactor         │  ├─ Building OAuth2
│  ├─ Running tests               │  ├─ Testing authentication
│  └─ Writing docs                └─ Opening PR (Session B)
│
└─ Ready to commit                 Ready to review
```

When you're done with a worktree:

```
# Inside the worktree session
exit                          # Closes the session
# Worktree is automatically cleaned up

# Or manually remove
git worktree remove .claude/worktrees/feature-auth
```

Session Resumption and Forking

Resume: Continue Where You Left Off

```
# Resume last session
claude -c

# Resume a named session
claude --resume my-task

# List all sessions
claudeSession list
```

When you resume:

- Full conversation history is restored
- Files are in the state they were when you paused
- Permissions reset (you re-approve them, like security passes expiring)
- Memory loads automatically

Fork: Branch Off Without Losing Original

```
# Fork current session into a copy
claude --continue --fork-session
```

This creates a **new session with the same conversation history**. The original stays untouched. Useful when:

- You want to try a different approach without losing context
- You're at a decision point and want to explore both paths
- You're debugging and want to revert to a stable point without losing history

Example:

```
Original Session A
├─ "Implement OAuth2"
├─ [writes code]
└─ [code review finds issues]
   │
   └─ Fork → Session A' (copy of conversation)
      ├─ "Try a different approach"
      ├─ [rewrites code differently]
      └─ [compare approaches later]
```

Session Naming

Name sessions immediately when starting meaningful work:

```
# At the start of a task
/rename oauth-implementation

# Later, resume by name
claude --resume oauth-implementation
```

Why this matters:

After a month of work, you'll have 50+ sessions. Without names, finding the right one is impossible. Named sessions appear in:

- claudeSession list

  — easy to find
- Session sidebar (desktop app) — visual organization
- /resume

  autocomplete — quick access

Naming convention: `feature-name` or `fix-description` or `task-type-description`

```
/rename payment-gateway-refactor
/rename fix-auth-race-condition
/rename security-audit-2026-06
```

Five Developer Habits That Scale

These habits are simple but compound dramatically over months:

Habit 1: Clear Between Tasks

When: After finishing one task, before starting the next **Command:** `/clear` **Why:** Previous conversations pollute new work

```
# Finished the auth feature
# Context is now filled with auth-related history

/clear

# Now starting the payment feature
# Fresh context, no auth noise
```

Cost impact: Saves ~100K tokens per task switch. On a 5-task day, that's 500K tokens saved (roughly 40% of daily cost).

Habit 2: Compact at Breakpoints

When: After finishing a subtask, before starting a new phase **Command:** `/compact keep only: [what matters]` **Why:** Preserves knowledge while freeing context

```
# Finished phase 1: schema design
# About to start phase 2: implementation

/compact keep only: final schema, API design decisions, performance notes

# Now 30% context used instead of 80%
```

Best practices:

- Compact

  before

  context reaches 80%, not after
- Give Claude guidance on what to preserve
- Run

  /context

  first to understand what's consuming space

Habit 3: Name Your Sessions

When: Immediately after starting work **Command:** `/rename descriptive-name` **Why:** Future you will thank present you

```
claude                          # Start session
# [do some work]
/rename redis-migration-phase1   # Name it now
```

After a week:

- Without names: "Which of these 12 unnamed sessions was the Redis work?"
- With names:

  /resume redis-migration-phase1

  (instant access)

Habit 4: Monitor Your Context

When: Periodically during work (every 30 minutes) **Command:** `/context` **Why:** Catch context creep before it blocks you

```
/context

# Output:
# Conversation history: 42% (300K tokens)
# Files loaded: 25% (180K tokens)
# Tool outputs: 18% (130K tokens)
# ---
# Total: 85% of context window
```

Actions:

- Below 60%: keep working, plenty of room
- 60-75%: consider compact if you have a pause point
- 75-85%: compact soon, plan next task carefully
- 85%+: compact immediately or start next task fresh

Habit 5: Document and Clear for Big Tasks

When: Task will consume most of context window **Pattern:** "Document → Clear → Resume" **Why:** Unlimited context by resetting strategically

```
# Phase 1: Architecture
[work through architecture design]
/compact keep architecture decisions in a file

# Ask Claude to write progress to a file
"Save our architecture decisions to docs/ARCHITECTURE.md"

/clear

# Phase 2: Implementation
"Read docs/ARCHITECTURE.md and implement based on this design"
[fresh context, full knowledge]

/clear

# Phase 3: Testing
"Read docs/ARCHITECTURE.md and write comprehensive tests"
[another fresh context, full knowledge]
```

Result: You never run out of context for large projects, and knowledge persists via git-committed documentation.

Parallel Workflows

Pattern 1: Developer + Reviewer

Terminal 1 (Writer):

```
cd ~/myrepo
claude --worktree feature-auth
# Implement OAuth2 authentication
```

Terminal 2 (Reviewer):

```
cd ~/myrepo
claude --worktree review-session
# In parallel, review code from feature-auth branch
# Use: "Review @../myrepo/.claude/worktrees/feature-auth/src/auth.ts"
```

Both work simultaneously:

- Writer implements, runs tests, iterates
- Reviewer provides feedback as code is written
- No context pollution (separate sessions)
- No merge conflicts (separate branches)

Pattern 2: Multiple Features in Parallel

```
# Terminal 1: OAuth Feature
claude --worktree oauth-feature
# [working on auth]

# Terminal 2: Payment Feature
claude --worktree payment-feature
# [working on payments]

# Terminal 3: DevOps Task
claude --worktree infra-update
# [working on infrastructure]
```

Three engineers (or one person's three sessions) working in parallel on different features. Commits never conflict because they're on different branches.

Pattern 3: Bug Fix While Continuing Main Work

```
# Main development
cd ~/myrepo
claude

# Bug reported
# Without closing main session:
claude --worktree bugfix-issue-234

# [fix the bug, commit, PR]
# Main session still running, unaffected
```

Session State and Permissions

Each session maintains state:

State	Resets on	Persists across
Conversation history	`/clear`	Resume (`claude -c`)
Context	Auto-compaction	Session-level settings
Permissions	Resume	Session-level allow/deny
Memory	Never	All sessions in directory
Allowed tools	Settings change	Session-level config

Important: When you resume with `claude -c`, permissions reset. This is intentional — like security passes expiring when you leave the building. You'll see permission prompts again on first tool use.

Session Storage and Cleanup

Sessions are stored in:

```
~/.claude/sessions/
├── <directory-hash>/
│   ├── current.jsonl        # Latest session conversation
│   ├── history/
│   │   └── <session-id>.jsonl
│   └── metadata.json
```

Auto-cleanup:

- Sessions older than 7 days of inactivity are deleted
- Configure with:

  sessionRetentionDays

  in

  .claude/settings.json

```
{
  "sessionRetentionDays": 30   # keep inactive sessions for 30 days
}
```

Tips Most Engineers Miss

1. Use `/rename` immediately

```
claude
/rename current-task
# Now it's findable later
```

2. Monitor context in real-time

Add status line to your shell prompt showing context percentage. When it climbs past 70%, you know it's time to compact.

```
/statusline
```

3. Use `--add-dir` for monorepos

```
claude --add-dir ../frontend ../shared-lib

# Now Claude can read/edit in all three folders
# Single session, multiple working areas
```

4. Worktrees for parallel work

Never do serial refactoring when you could do parallel work:

```
# Serial (slow): finish feature-a, then start feature-b
# Parallel (fast): start both in worktrees simultaneously
claude --worktree feature-a &
claude --worktree feature-b &
```

5. The session recovery pattern

Lost context mid-task?

```
# Don't: manually re-type context
# Do: resume the session
claude -c
```

6. `/compact` beats `/clear`

```
# Don't: /clear (lose all context)
# Do: /compact keep only: X
```

Compacting preserves what matters while freeing space.

7. Checkpoints are your safety net

```
/rewind
# See all checkpoints, restore code-only, conversation-only, or both
```

Checkpoints are free (just git internally). Use them without guilt.

Session Patterns for Teams

Pattern: Shared Session Log

For pair programming or async collaboration:

```
# Session started at 10am
/rename feature-oauth-[date]

# Throughout the day, both team members:
claude --resume feature-oauth-[date]

# Same conversation history, same code changes
# Like pair programming across time zones
```

Pattern: Session Handoff

Engineer A starts work, Engineer B continues:

```
# Engineer A
/rename fixing-db-query
[works for 2 hours]
/rename fixing-db-query-[assigned-to-B]
exit

# Engineer B
claude --resume fixing-db-query-[assigned-to-B]
# Full context! Reads the entire conversation, understands decisions
```

Common Mistakes

1. Not using worktrees for parallel work

Branches solve version control, but sessions are separate. Use worktrees for true parallelization.

2. Letting context fill to 100%

Monitor context. Compact at 75%. Never let it max out.

3. Forgetting to name sessions

Name sessions immediately. Unnamed sessions are unsearchable.

4. Clearing instead of compacting

`/compact` preserves knowledge. `/clear` deletes it. Choose wisely.

5. Long sessions without checkpoints

Checkpoints are free. Use them. Rewind instantly if something breaks.

Advanced: Custom Session Configuration

Per-session settings in `.claude/settings.json`:

```
{
  "sessionDefaults": {
    "autoMemoryEnabled": true,
    "defaultModel": "claude-sonnet-4-6",
    "permissionMode": "acceptEdits"
  },
  "sessionRetentionDays": 30
}
```

Session management is the foundation of productive Claude Code work. Master these five habits and you'll never lose context, never conflict on branches, and never struggle to find your previous work.

Claude Code Checkpoints: Your AI Safety Net for Code Changes

Published 2026-05-02 | Source: https://blog.devops-monk.com/2026/05/claude-code-checkpoints-guide/

Master Claude Code's checkpoint system for automatic code snapshots. Learn when to rewind, how checkpoints work, and best practices for protecting your codebase.

When Claude Code works on your project, it can touch dozens of files in a single prompt—refactoring a module, renaming methods across the codebase, restructuring folders. What happens when it goes wrong? Traditional developers rely on keyboard undo or Git commits, but neither fits an AI that operates at speed.

Claude Code's **checkpoints** are your safety net. They automatically capture the state of your code before each edit, letting you rewind in seconds if anything goes sideways.

What Are Checkpoints?

Checkpoints are **automatic snapshots** of your code state captured silently before Claude touches anything.

Key facts:

- Persist across sessions (close your terminal, they're still there)
- Don't disappear when you close Claude Code
- Expire after 30 days
- Only track direct file edits made through Claude's editing tools
- Do NOT track

  changes made by bash commands (

  rm file.txt

  ,

  mv old.txt new.txt

  ,

  cp source dest

)

Why Checkpoints Matter

Before Claude Code, developers had to choose between:

1. **Keyboard Undo** — Works fine for single-file edits, but when Claude updates 5 files simultaneously:
   - Which file changed first?

- How many times do you undo?
- Did you accidentally remove something important?
- Undo becomes guesswork.

2. **Git Commits** — Powerful but not designed for constant experimentation. Creating a safe point requires:
   - git add .
   - Write a commit message
   - Run

     git commit
   - Result: messy commits like “test”, “in-progress”, “ignore” cluttering history.
3. **Manual Backups** — Copying folders or making ZIPs:
   - Takes time
   - Breaks your concentration
   - Most people forget to do it
   - By the time something breaks, there’s no recent backup.

Checkpoints solve all three problems.

How Checkpoints Work

Every prompt you submit triggers a silent snapshot **before** Claude touches anything. No save button. No planning. Just automatic protection.

```
You: "Refactor the auth module"
↓
[Checkpoint created automatically]
↓
Claude modifies files
```

If Claude’s changes go sideways, you can rewind in seconds.

Checkpoints also **persist across sessions**, so you can close your terminal and still rewind later.

When Checkpoints Are Essential

Scenario 1: Exploring Risky Approaches

You want Claude to try a completely different architectural pattern, but you’re not 100% sure it’ll work.

Checkpoint gives you safety to experiment without fear.

Scenario 2: Claude Misunderstands the Prompt

You asked for a small refactor and Claude rewrote three modules. Before you even figure out what changed, checkpoint lets you rewind in seconds.

```
You: "Simplify the login form component"
Claude: *accidentally refactors entire auth flow*
You: Press Esc twice → /rewind → Select checkpoint → Done
```

Scenario 3: Context Gets Polluted

You've corrected Claude twice on the same issue and it keeps making the same mistake.

Rewind to before the first wrong attempt and try again with a better prompt—a **clean prompt almost always outperforms** a long session with accumulated corrections.

Scenario 4: Direction Changes Mid-Session

You were migrating to version 4 across multiple prompts, then requirements changed to use version 5.

Rewind both code and conversation to the pre-migration state and start fresh—without losing any work outside that scope.

How to Rewind

Press `Esc` twice or use:

```
/rewind
```

This opens the rewind menu. Pick a checkpoint and decide what to restore:

Restore Code Only

- Reverts file changes, keeps the conversation
- Good when: The code went wrong but the context is still useful

Restore Conversation Only

- Rewinds the chat, keeps the code
- Good when: You want to re-prompt differently without throwing away changes you liked

Restore Both

- Full reset — code and conversation go back together
- Good when: Everything needs a reset

Summarize from Here

- Doesn't restore anything
- Instead, condenses messages from that point forward while keeping earlier context intact
- A surgical alternative to

  /compact

  that only compresses the part eating up space

Critical Limitation: What Checkpoints DON'T Track

This is crucial: **Checkpointing does NOT track files modified by bash commands.**

If Claude runs:

```
rm file.txt
mv old.txt new.txt
cp source.txt dest.txt
```

These modifications **cannot be undone through rewind**. Only direct file edits made through Claude's file editing tools are tracked.

Workaround: If you need to protect files from bash modification, use hooks (see Hooks guide) to prevent dangerous commands or use Git commits before risky operations.

Checkpoints vs Git: Complementary, Not Competitive

Think of them this way:

- Checkpoints

  = local undo (frictionless, session-scoped, 30-day lifespan)

- Git

  = permanent history (travels with your repo, team-shareable, CI/CD traceable)

They work together, not against each other.

Before a Large Task: Commit Your Working State

```
git checkout -b feature/refactor-auth
git add .
git commit -m "before AI refactor"
```

If everything breaks, `git reset --hard` brings you back instantly. Unlike checkpoints, this:

- Survives beyond 30 days
- Travels with your repo
- Works in team environments
- Provides permanent history

Best Practices

Situation	Use Checkpoints	Use Git
Quick experiments	Yes	No
Mid-session corrections	Yes	No
Before large refactors	No	Yes
Sharing work with team	No	Yes
Long-term history	No	Yes
Exploring approaches	Yes	No

Common Mistakes

Mistake: Relying on checkpoints for everything **Fix:** Use Git for large tasks, checkpoints for experimentation

Mistake: Thinking checkpoints track bash commands **Fix:** Remember—only direct Claude file edits are tracked

Mistake: Assuming checkpoints last forever **Fix:** They expire after 30 days; use Git for permanent history

Next Steps

- Session Management Guide:

  Learn how to organize long sessions effectively

- Hooks Guide:

  Automate actions before/after Claude's work

- Git Integration Guide:

  Use Git alongside Claude Code for powerful workflows

Part 3: Extending Claude Code

MCP Servers Worth Installing: For Developers, Testers, and DevOps Teams

Published 2026-04-26 | Source: https://blog.devops-monk.com/2026/04/mcp-servers-for-devops-teams/

A role-based guide to MCP servers that actually save time — curated lists for developers, QA testers, and DevOps/platform engineers, with configuration examples and a framework for evaluating new servers.

The MCP (Model Context Protocol) ecosystem now has over 14,000 servers. Most of them you do not need. Installing too many slows responses, inflates your context window with tool definitions that never get used, and turns debugging into a guessing game across a dozen integrations.

This guide applies a simple filter: **only install a server if it replaces a daily copy-paste workflow**. It is organised by role — developers, QA testers, and DevOps/platform engineers — because the right stack is different for each.

How MCP Fits Into Your Workflow

```
flowchart TD
    User[Developer / Tester / DevOps] -->|natural language query| Claude[Claude Code]
    Claude -->|tool call| MCP[MCP Server]
    MCP -->|authenticated request| Service[External Service\nGitHub / Playwright / AWS etc]
    Service -->|structured data| MCP
    MCP -->|response| Claude
    Claude -->|answer + action| User
```

Without MCP, you copy-paste data into Claude. With MCP, Claude pulls it directly — no context switching, no stale information. The same applies whether you are querying a GitHub PR, running a browser test, or reading a Kubernetes pod log.

For Developers

These servers eliminate the most common interruptions in a coding session: switching to the browser to check docs, copy-pasting database output, and manually verifying what your code actually does at runtime.

1. GitHub MCP

The single most-used MCP server across all roles. GitHub MCP lets Claude search your entire codebase, create and review pull requests, read issue comments, and check CI status — without leaving the editor.

```
claude mcp add --transport http github https://api.github.com/mcp/ \
  --header "Authorization: Bearer ghp_YOUR_TOKEN"
```

Useful queries:

```
What PRs need my review today?
Search the codebase for all uses of the old auth middleware
Summarise the failing CI checks on PR #342
Create an issue for the memory leak we discussed — assign it to me
What did we merge in the last 3 days?
```

2. Figma MCP (Dev Mode)

Figma's official Dev Mode MCP server exposes the live structure of the selected layer — hierarchy, auto-layout, variants, text styles, and token references — so Claude can generate code against the real design instead of a screenshot.

Before this existed, the workflow was: screenshot the Figma frame → paste into Claude → get approximate code → iterate. Now Claude reads the actual design spec.

```
{
  "mcp": {
    "servers": {
      "figma": {
        "transport": "http",
        "url": "https://api.figma.com/v1/mcp",
        "headers": {
          "Authorization": "Bearer YOUR_FIGMA_TOKEN"
        }
      }
    }
  }
}
```

Useful queries:

```
Generate a React component for the selected card in Figma
What spacing tokens does this layout use?
Implement the checkout form exactly as designed — use our existing design system tokens
```

3. E2B MCP (Sandboxed Code Execution)

E2B gives Claude a secure cloud sandbox to run Python or JavaScript, execute shell commands, install packages, and inspect outputs — all inside an isolated microVM. This is the right answer to "I want Claude to verify this migration script before it touches the real database."

```
{
  "mcp": {
    "servers": {
      "e2b": {
        "transport": "http",
        "url": "https://api.e2b.dev/mcp",
        "headers": {
          "Authorization": "Bearer YOUR_E2B_KEY"
        }
      }
    }
  }
}
```

Useful queries:

```
Run this data transformation script on the sample CSV and show me the output
Test this regex against these 20 edge case strings
Execute the migration dry-run and tell me what it would change
```

4. Database MCP (Match to Your Stack)

Pick one. All of these follow the same pattern: Claude translates natural language into queries, executes them, and explains the results.

Stack	Server	Best for
PostgreSQL	`mcp-postgres`	Most production backends
Prisma + Postgres	`prisma-mcp`	TypeScript teams — also manages schema migrations
Supabase	`supabase-mcp`	Respects Row Level Security
SQLite	`mcp-sqlite`	Local development and prototyping
MongoDB	`mcp-mongodb`	Document store queries and backfills

Important: Point database MCP at a **read replica with a read-only user**. Never give Claude write access to a production database through MCP — destructive queries do happen.

```
{
  "mcp": {
    "servers": {
      "postgres": {
        "transport": "stdio",
        "command": "mcp-postgres",
        "args": [
          "--connection-string",
          "postgresql://readonly_user:pass@read-replica.example.com/db"
        ]
      }
    }
  }
}
```

5. Context7 / Docs MCP

Claude's training data has a cutoff. For libraries that change frequently — React Router, Next.js, Prisma — Claude might suggest deprecated APIs, miss new features, or give you code that doesn't compile against the current version. This is not carelessness; it is how LLMs work.

Context7 solves it. It indexes official documentation for thousands of libraries, keeps them continuously updated, and serves only the relevant section — in a format optimised for LLMs, not humans browsing a docs site.

```
claude mcp add --transport http context7 --scope project https://mcp.context7.com/mcp
```

Note the transport: this is a **remote HTTP MCP server**, not a local npx process. No installation needed — Claude reaches out over HTTP.

Real-world example: your app eagerly imports all page components at startup. Users who never visit admin pages still pay the bundle cost. The fix is React Router v7 lazy loading — but the API changed from v6 and you don't want to guess.

```
Using context7, look up the React Router DOM v7 docs for lazy-loading routes
with React.lazy() and Suspense. Then refactor src/App.jsx to lazy-load all
page components instead of eagerly importing them. Keep provider nesting and
error boundaries intact. Add a simple fallback (spinner or "Loading..." text).
```

Context7 fetches the exact current API, not a cached version from training data.

Other useful queries:

```
Show me the current React Query v5 useQuery signature
What changed in Next.js 15 App Router between RC and stable?
Give me the Prisma docs for nested transactions
```

For Testers

These servers give Claude the ability to actually *run* your application, observe its behaviour, and report what it finds — rather than reason about what *should* happen.

6. Playwright MCP

The most important testing MCP. Playwright MCP drives a real browser using Claude — it navigates, clicks, fills forms, and reads the resulting DOM and accessibility tree. Claude can verify its own code changes by actually running the app.

```
npx @playwright/mcp@latest
```

Or configure in `.claude/settings.json`:

```
{
  "mcp": {
    "servers": {
      "playwright": {
        "transport": "stdio",
        "command": "npx",
        "args": ["@playwright/mcp@latest", "--headless"]
      }
    }
  }
}
```

Once connected, Claude can navigate, click, fill forms, intercept network requests, take screenshots, and generate Playwright test files from its own actions. It runs across Chrome, Firefox, and Safari.

Ready-to-use prompts:

```
"A user reported that clicking 'Add to Cart' sometimes shows a 500 error.
Open the app, test with real data, figure out what's actually happening,
and tell me what's broken."
```

```
"Navigate through the main user flows at localhost:3000 (signup, login,
dashboard, settings). Collect every console error and warning. Propose
fixes ranked by severity."
```

```
"Take screenshots at 1920×1080, 768×768, and 375×667. Compare against
./baseline/. Tell me which pages have visual regressions and what changed."
```

```
"Walk through my checkout flow as a real user would. Record what you do,
then generate a Playwright test file covering this flow with proper
assertions. Save it to tests/e2e/checkout.spec.ts"
```

```
"The tests in tests/e2e/login.spec.ts are failing. Run them, inspect the
DOM at /login, figure out whether the app or the test is broken, and fix
whichever side is actually broken."
```

```
"Crawl every page linked from my homepage (up to 2 levels deep). For each
page capture: title, meta description, H1, word count, broken links, load
time. Give me a table sorted by problems found."
```

```
"Test the signup form with: empty fields, SQL injection strings, extremely
long inputs, unicode/emoji, mismatched passwords, already-taken emails.
For each, report what happened and whether the app handled it gracefully.
Fix any validation issues you find."
```

Playwright MCP uses the **accessibility tree** rather than screenshots — which means tests are more stable, not tied to visual layout, and explain *why* something is failing rather than just that it is.

7. Chrome DevTools MCP

Where Playwright MCP drives the browser, Chrome DevTools MCP *inspects* it. You can read console errors, examine network requests, check performance metrics, and debug JavaScript — all through Claude.

```
{
  "mcp": {
    "servers": {
      "chrome-devtools": {
        "transport": "stdio",
        "command": "mcp-chrome-devtools",
        "args": ["--port", "9222"]
      }
    }
  }
}
```

Useful queries:

```
What JavaScript errors appeared on the checkout page during my last test run?
Are there any failed network requests when I load the dashboard?
What is the Largest Contentful Paint score on the homepage?
Which API calls are taking more than 500ms?
```

8. BrowserStack MCP

For cross-browser and cross-device testing without maintaining local device labs. BrowserStack MCP runs your tests on real browsers (Safari, Firefox, Edge) and real mobile devices.

```
{
  "mcp": {
    "servers": {
      "browserstack": {
        "transport": "http",
        "url": "https://api.browserstack.com/mcp",
        "headers": {
          "Authorization": "Basic BASE64_USERNAME_ACCESSKEY"
        }
      }
    }
  }
}
```

Useful queries:

```
Run the login test on Safari iOS 17 and tell me if it passes
Check whether the payment form renders correctly on Samsung Galaxy S23
What is the test pass rate across browsers this week?
```

9. Sentry MCP

Sentry MCP bridges the gap between a test failure and its production origin. When a test reproduces a bug, Claude can immediately check whether the same error pattern has appeared in production and how many users it affects.

```
{
  "mcp": {
    "servers": {
      "sentry": {
        "transport": "stdio",
        "command": "mcp-sentry",
        "env": {
          "SENTRY_AUTH_TOKEN": "YOUR_TOKEN",
          "SENTRY_ORG": "your-org"
        }
      }
    }
  }
}
```

Useful queries:

```
Has this NullPointerException appeared in production? How many users hit it?
Show me the full stack trace for the checkout error from yesterday
What are the top 5 errors in production right now?
```

For DevOps and Platform Engineers

These are the servers that eliminate the toil: swapping between dashboards during incidents, writing Kubernetes debug commands from memory, and manually correlating deployment times with error spikes.

10. Kubernetes MCP

The most valuable infrastructure MCP for teams running Kubernetes. Natural language diagnostics replace kubectl command lookup for the 80% of debugging you do every week.

```
{
  "mcp": {
    "servers": {
      "kubernetes": {
        "transport": "stdio",
        "command": "kubectl-mcp-server",
        "args": ["--kubeconfig", "~/.kube/config"]
      }
    }
  }
}
```

Useful queries:

```
Why is the nginx pod crashing? Check events, logs, and resource limits
Which pods are not in RUNNING state across all namespaces?
What changed in the last deployment to the payments service?
Scale the api deployment to 5 replicas
Show me nodes with high CPU or memory pressure
```

11. AWS MCP (Read-Only)

Read-only access to CloudWatch logs, EC2 status, ECS task definitions, S3 bucket policies, and cost data covers 80% of what you need during debugging and incident response. Keep it read-only — destructive AWS operations belong in Terraform or the console.

```
{
  "mcp": {
    "servers": {
      "aws": {
        "transport": "stdio",
        "command": "mcp-aws",
        "args": ["--profile", "readonly", "--region", "us-east-1"]
      }
    }
  }
}
```

Useful queries:

```
What errors appeared in CloudWatch for the payments service in the last hour?
List ECS tasks that are not in RUNNING state
What is the current CPU utilisation on our RDS cluster?
Show me the cost breakdown for this month vs last month
Which S3 buckets have public access enabled?
```

12. Terraform MCP

Manages Terraform Cloud workspaces, triggers runs, inspects state, and performs cost estimations — without leaving Claude Code.

```
{
  "mcp": {
    "servers": {
      "terraform": {
        "transport": "http",
        "url": "https://app.terraform.io/api/v2/mcp",
        "headers": {
          "Authorization": "Bearer YOUR_TF_TOKEN"
        }
      }
    }
  }
}
```

Useful queries:

```
What would change if I apply the latest run in the staging workspace?
Show me the current state of the production VPC resources
Which workspaces have pending runs?
What is the cost estimate for the new EKS node group?
```

13. Grafana MCP

Queries dashboards, inspects data sources, and retrieves incident details — the natural language layer on top of your existing Grafana setup.

```
{
  "mcp": {
    "servers": {
      "grafana": {
        "transport": "stdio",
        "command": "uvx",
        "args": ["mcp-grafana"],
        "env": {
          "GRAFANA_URL": "http://your-grafana:3000",
          "GRAFANA_SERVICE_ACCOUNT_TOKEN": "YOUR_TOKEN"
        }
      }
    }
  }
}
```

Useful queries:

```
What does the API latency look like over the last 2 hours?
Show me the error rate for the payments service since the last deploy
Are there any active alerts right now?
```

14. PagerDuty MCP

During incidents, PagerDuty MCP turns Claude into an effective co-pilot. It reads the timeline, affected services, escalation policy, and past resolution notes without you switching tabs.

```
{
  "mcp": {
    "servers": {
      "pagerduty": {
        "transport": "stdio",
        "command": "mcp-pagerduty",
        "args": ["--api-key", "YOUR_PD_KEY"]
      }
    }
  }
}
```

Useful queries:

```
What services are currently in alert state?
Who is on-call right now for the payments team?
Summarise the last 5 incidents for the checkout service — what were the causes?
Create an incident for the database latency spike
```

15. Trivy MCP (Security Scanning)

Trivy scans container images and infrastructure code for CVEs, misconfigurations, and exposed secrets. With the MCP server, Claude can run scans and explain findings in context.

```
{
  "mcp": {
    "servers": {
      "trivy": {
        "transport": "stdio",
        "command": "mcp-trivy",
        "args": []
      }
    }
  }
}
```

Useful queries:

```
Scan the payments-service:latest image for critical vulnerabilities
Are there any HIGH or CRITICAL CVEs in our base image?
Check the Terraform configs in infra/ for misconfigurations
Scan for exposed secrets in the repo
```

16. Linear MCP (or Jira MCP)

Converting Slack threads and incident retrospectives into properly structured tickets is a time sink. With Linear MCP, Claude creates, labels, and assigns tickets from natural language.

```
claude mcp add linear --api-key lin_YOUR_KEY
```

Useful queries:

```
Create a ticket for the memory leak in the auth service — P2, platform team
What tickets are blocking the next release?
Mark PLAT-234 as done and add a comment with the fix summary
Create a post-mortem ticket for this incident with the timeline we discussed
```

Role-Based Starter Stacks

```
flowchart LR
    subgraph Dev[Developer]
        D1[GitHub MCP]
        D2[Figma MCP]
        D3[Database MCP]
        D4[Context7 Docs]
        D5[E2B Sandbox]
    end
    subgraph Test[QA Tester]
        T1[Playwright MCP]
        T2[Chrome DevTools MCP]
        T3[BrowserStack MCP]
        T4[Sentry MCP]
    end
    subgraph Ops[DevOps / Platform]
        O1[GitHub MCP]
        O2[Kubernetes MCP]
        O3[AWS MCP read-only]
        O4[Grafana MCP]
        O5[PagerDuty MCP]
        O6[Linear MCP]
    end
```

Developer starter pack (in priority order):

1. GitHub MCP — daily use for every developer
2. Database MCP matching your stack — debugging and exploration
3. Context7 — stop hallucinated API signatures
4. Figma MCP — only if you implement designs regularly

Tester starter pack:

1. Playwright MCP — core verification tool
2. Chrome DevTools MCP — runtime error inspection
3. Sentry MCP — bridge tests to production error patterns

DevOps starter pack:

1. GitHub MCP — shared with developers, essential for everyone
2. Kubernetes MCP — if you run k8s, this pays for itself in the first week
3. AWS MCP (read-only) — CloudWatch and cost visibility
4. PagerDuty MCP — incident co-pilot
5. Grafana or Prometheus MCP — observability layer

Servers to Skip (and Why)

```
flowchart LR
    subgraph Skip[Skip or Limit These]
        N[Notion MCP\n3-5s latency per call]
        P[Postgres MCP on prod\nrisk of destructive queries]
        S[Slack MCP inbound\nbetter just to read Slack]
        FS[Filesystem MCP with broad paths\nred flag for security]
    end
```

Postgres MCP against production — point it at a read replica with a read-only user instead. Claude does issue destructive queries occasionally.

Notion MCP — high latency, low return for most teams. Export relevant pages to markdown and use `@` imports in CLAUDE.md instead.

Slack MCP for reading — slower than reading Slack yourself. Use outbound webhooks so Claude can *post* to Slack; skip inbound retrieval.

Filesystem MCP with broad paths — granting Claude access to your entire home directory or `/` creates unnecessary risk. Scope it to specific project directories only.

Project-Level vs Global Configuration

```
~/.claude/settings.json           ← personal servers (your GitHub token, your Figma token)
.claude/settings.json             ← project servers (team Linear workspace, project DB)
.claude/settings.local.json       ← personal project overrides (gitignored)
```

Put shared servers in `.claude/settings.json` (committed to git) so the whole team gets them. Put personal credentials in `~/.claude/settings.json`.

Check what is actually being used:

```
claude mcp list          # see all configured servers
/mcp                     # check server status and context cost inside a session
/mcp disable <name>      # disable a server you are not using right now
```

Every enabled server loads its tool definitions into context on every message — whether you use it or not. If you are not actively using a server in a session, disable it with `/mcp disable`. This is one of the highest-impact token optimisations available.

Multi-MCP Workflow Scenarios

The real leverage comes from combining servers. Here are four cross-tool workflows that eliminate entire categories of manual work:

Scenario	Problem solved	Key MCP servers	Sample prompt
Ticket → Code → PR	Manual Jira → coding → GitHub switching kills velocity	Jira (or Linear) + GitHub + Filesystem	"Implement the feature from JIRA issue ENG-4521. Read the full ticket, write the code, run tests, commit, and open a PR with a description linking back to the ticket."
Production Data → Code Insights	Developers guess user impact because they cannot query the prod DB	Postgres + Filesystem	"Query our Postgres DB for the last 30 days of usage data for feature X. Cross-reference with the relevant code modules. Suggest 2 optimisations based on the data and commit the changes."
Monitoring → Auto-Fix	Bug triage from Sentry is slow and error-prone	Sentry + GitHub	"Check recent Sentry errors in the auth module. Analyse the stack traces against our codebase. Fix the root cause, run tests, and create a PR with before/after metrics."
Cross-Tool Automation	Team updates scattered across Notion, Slack, and code	Notion + Slack + GitHub + Filesystem	"Read the latest product spec from Notion. Update the README and relevant code files. Post a summary to #engineering on Slack. Create a tracking issue on GitHub."

Each of these requires no manual copy-paste between tools. Claude reads from the source, acts on the data, and writes back to the destination — across all four systems in one prompt.

The Evaluation Framework

Before installing any new MCP server, answer three questions:

1 Does it replace a daily copy-paste workflow?

   If yes, install it. If it saves time once a week, skip it.

2. Can it make a destructive change?

   If yes, either restrict to read-only or skip it entirely.

3. Will Claude actually reach for it, or will I forget it exists?

   If unsure, skip it for now and revisit in a month.

Security checklist for any server you install:

- Use a dedicated service account, not your personal credentials
- Grant the minimum permissions needed (read-only where possible)
- Rotate credentials on the same schedule as your other service accounts
- Review what data the server can access before connecting it to Claude

Five well-chosen MCP servers outperform twenty poorly chosen ones every time. Start with one stack that matches your role, observe which tools Claude actually uses, then trim the rest.

MCP in Claude Code: Connect to External Tools, Databases & APIs

Published 2026-05-04 | Source: https://blog.devops-monk.com/2026/05/claude-code-mcp-protocol-guide/

Master the Model Context Protocol (MCP) to give Claude Code access to your databases, GitHub, AWS, Jira, and custom tools. Includes Chrome DevTools, Playwright, and Context7 examples.

Claude Code can read and write files, execute shell commands, and search your codebase. But here's the problem: it lives in isolation. It can't talk to your database. It can't query your Jira board. It can't inspect network requests in your running app. It can't access your AWS infrastructure.

Without MCP, Claude Code is smart but limited.

With **MCP (Model Context Protocol)**, Claude Code becomes a full participant in your development ecosystem—talking to databases, APIs, monitoring systems, and custom tools in real-time.

The Problem Without MCP

Imagine you hire the world's smartest developer. But that desk has:

- No internet
- No database access
- No Jira board
- No Slack
- No GitHub issues
- No internal documentation
- No AWS console
- No monitoring dashboard

How useful are they? Still smart, but **frustratingly limited.**

That's Claude Code without MCP.

What Is MCP?

MCP (Model Context Protocol) is an open standard that lets Claude connect to external tools, databases, APIs, and services in **real-time**—not through copy-paste, not through uploaded files, but through **live two-way connections.**

Think of it as **USB for AI.** Just like USB lets you plug any device into any computer, MCP lets you plug any tool/service into Claude Code.

Key insight: MCP servers are extensions—extra tools you hand to Claude. Once connected, Claude doesn't just *know about* your database—it can actually *talk to it.*

Three Perspectives on MCP

Developers: Supercharge Your Workflow

Before MCP: You describe a bug in chat, Claude guesses what's wrong based on source code.

With MCP: Claude opens your browser, reproduces the bug, reads console errors, inspects network requests, then fixes the code.

Before MCP: You ask Claude to implement a feature from a Jira ticket.

With MCP: Claude fetches the ticket, reads the full requirements, explores the codebase, implements the feature, and creates a PR.

Testers: Automated Testing at Speed

Before MCP: You describe test scenarios in text, Claude writes tests blind.

With MCP (Playwright MCP): Claude:

- Navigates your app at localhost:5173
- Clicks buttons, fills forms with real data
- Takes screenshots at each step
- Reads console errors and warnings
- Generates Playwright test files from its own actions
- Runs tests across Chrome, Firefox, Safari

You get **end-to-end tests that actually work**, generated by an AI that can see your app.

DevOps: Infrastructure & Automation

Before MCP: You describe infrastructure needs in chat, Claude writes Terraform blind.

With MCP: Claude:

- Queries your AWS, Kubernetes, or Terraform state
- Sees current infrastructure status
- Validates changes against real configs
- Runs deployments and monitors results
- Reads logs from production systems

You get **infrastructure changes that are proven to work** before they touch production.

MCP in Action: Three Real Examples

Example 1: Chrome DevTools MCP (Developers & Testers)

One command:

```
claude mcp add chrome-devtools --scope project --npx chrome-devtools-mcp@latest
```

What happens:

- Claude connects to Chrome DevTools via MCP
- Scopes it to this project only
- Runs the MCP server automatically (no install needed)

Verify it's working:

```
/mcp
```

Developer scenario:

You: "My React app at localhost:5173 has a login form that always fails. Open it, test with sample data, figure out what's broken, and fix it."

Claude:

1. Opens the browser
2. Fills the login form with test data
3. Clicks Submit
4. Reads console errors and network responses
5. Traces the bug back to source code
6. Fixes it

Without MCP: Claude reads source files, makes educated guesses. You go back and forth.

With MCP: Claude sees the actual error, fixes it first try.

Example 2: Playwright MCP (Testers & Developers)

One command:

```
claude mcp add playwright --scope project npx @playwright/mcp@latest
```

What Playwright does:

- Microsoft's open-source browser automation
- Controls Chrome, Firefox, Safari
- Clicks, fills forms, waits for elements
- Takes screenshots, intercepts network
- Battle-tested at enterprise scale

Tester scenario:

You: "Walk through my entire user flow (signup → login → dashboard → settings). Capture every console error. Then look at source code and propose fixes ranked by severity."

Claude walks through your app, collects all errors, reads the code, and proposes fixes with specific file/line numbers.

Tester scenario 2:

You: “Test my checkout flow with edge cases: empty fields, SQL injection strings, extremely long inputs, unicode/emoji, mismatched passwords, already-taken emails. Report what happened for each case.”

Claude tests each edge case, reports which ones the app handled gracefully, which ones failed. Developers get a detailed bug report.

Developer scenario:

You: “Our form submission is timing out in production. Open localhost:5173/checkout, monitor network requests, and tell me which API call is slow.”

Claude opens the form, submits it, watches network requests in real-time, identifies the slow endpoint with timing data.

Example 3: Context7 MCP (Developers & DevOps)

The problem: Claude was trained on data up to a cutoff. When you ask about React 19, it might know React 18 but miss new APIs.

The solution: Context7 indexes official documentation for thousands of libraries and serves docs optimized for AI.

One command:

```
claude mcp add --transport http context7 --scope project https://mcp.context7.com/mcp
```

(Note: This is a remote server, different from local npx.)

Developer scenario:

Your app eagerly imports all 18 page components at startup. Everyone pays the load cost, even job seekers who never visit admin pages.

You: “Using context7, look up React Router v7 docs for lazy-loading routes. Then refactor src/App.jsx to lazy-load all pages instead of eager imports.”

Claude:

1 Fetches the exact React Router v7 lazy-loading API (not v6, not your memory)

2 Refactors with the current best practice

3 Adds proper error boundaries and fallbacks

DevOps scenario:

You: “Our staging environment uses a deprecated API endpoint. Query the API docs via context7, identify all callers in the codebase, and refactor them to the new endpoint.”

Claude gets live documentation, finds all callers, refactors them correctly.

Common MCP Scenarios & Outcomes

Scenario	Without MCP	With MCP	Audience
Frontend bug	Claude guesses from code	Opens browser, sees error, fixes it	Dev/Tester

Scenario	Without MCP	With MCP	Audience
Test generation	Tests written blind	Tests from actual app behavior	Tester
API integration	Educated guess	Fetches live API docs, integrates	Dev
DB query optimization	Code review only	Queries actual data, proposes indexes	DevOps
Jira feature impl	Manual reading	Fetches ticket, implements, creates PR	Dev
Prod monitoring	Logs pasted in chat	Queries Sentry/DataDog, fixes bugs live	DevOps
Deploy validation	Terraform review	Queries AWS state, validates, deploys	DevOps

---

MCP Tool Search & Context Window Efficiency

By default, Claude scans lightweight tool names from all MCP servers. It pulls full schemas only when needed.

Why this matters:

- You can connect 10+ MCP servers without blowing up context
- Idle tools cost almost nothing
- Context only grows when a tool is actually used
- ~95% less context consumption vs loading all schemas upfront

---

Setting Up MCP for Your Team

For Developers: Chrome DevTools + Context7

```
# Browser automation + always-fresh docs
claude mcp add chrome-devtools --scope project --npx chrome-devtools-mcp@latest
claude mcp add --transport http context7 --scope project https://mcp.context7.com/mcp
```

For Testers: Playwright + GitHub

```
# Browser testing + issue tracking
claude mcp add playwright --scope project npx @playwright/mcp@latest
claude mcp add github --scope project npx @github-mcp/server@latest
```

For DevOps: AWS + Databases + Monitoring

```
# Infrastructure + data + alerts
claude mcp add aws --scope project npx @aws-sdk/client-s3@latest
claude mcp add postgres --scope project npx @postgres-mcp/server@latest
claude mcp add datadog --scope project npx @datadog-mcp/server@latest
```

Real-World Workflow Examples

Developer Workflow: Feature Implementation

1. You: “Implement the feature described in Jira ticket ENG-4521”
2. Claude (via MCP):
   - Fetches Jira ticket details
   - Reads full requirements
   - Explores codebase
   - Implements feature
   - Creates PR with description
3. You review PR and merge

Tester Workflow: Cross-Browser Testing

1. You: “Test my app across Chrome, Firefox, Safari. Capture screenshots at 1920x1080, 768x768, and 375x667. Compare against baseline/. Report visual regressions.”
2. Claude (via Playwright MCP):
   - Opens app in three browsers
   - Takes screenshots at three resolutions
   - Compares to baseline images
   - Reports differences with visual details

DevOps Workflow: Production Troubleshooting

1. You: “Check recent Sentry errors in the auth module. Analyze stack traces. Fix the root cause and create a PR with before/after metrics.”
2. Claude (via Sentry MCP):
   - Queries Sentry for recent errors
   - Reads stack traces
   - Finds root cause in codebase
   - Implements fix
   - Compares error rates before/after

Building Custom MCP Servers

If your tool isn’t in the marketplace, you can build your own MCP server.

Example: Internal documentation system

```
---
name: internal-docs-mcp
description: Query internal documentation from our wiki
tools: Read, Glob
---

You are an MCP server that queries our internal documentation.
When asked about company policies, architecture decisions, or runbooks,
fetch from our internal wiki at wiki.company.com and serve the results.
```

This lets Claude access your company's internal knowledge just like external APIs.

Comparison: MCP vs Traditional Integrations

Aspect	Copy-Paste	File Upload	MCP
Real-time data	No	No	Yes
Live execution	No	No	Yes
Bandwidth	Manual	One-time	Efficient
Security	Risky	Risky	Scoped
Setup time	Minutes	Minutes	One command

Security & Scoping

MCP servers are scoped:

```
# Project scope - only available in this project
claude mcp add chrome-devtools --scope project npx chrome-devtools-mcp@latest

# User scope - available in all projects
claude mcp add context7 --scope user https://mcp.context7.com/mcp
```

Principle of least privilege: Only give Claude the tools it needs for the current task.

Next Steps

- Skills Guide:

  Create reusable playbooks that use MCP tools
- Sub-Agents Guide:

  Delegate MCP work to specialized agents
- Rules Guide:

  Define when and how Claude can use MCP tools
- GitHub Actions Bot:

  Automate MCP-powered workflows in CI/CD

Resources

- MCP Protocol:

https://modelcontextprotocol.io

- Claude Code Docs:
  https://code.claude.com/docs
- Available MCP Servers:
  https://smithery.ai

Claude Code Plugins: The Complete Guide to Building and Sharing Extensions

Published 2026-05-17 | Source: https://blog.devops-monk.com/2026/05/claude-code-plugins-guide/

How Claude Code plugins work, when to use them over standalone config, and a step-by-step walkthrough building a real DevOps plugin with skills, hooks, and an MCP server.

Skills, agents, and hooks you add to `.claude/` are powerful — but they are locked to one project. Every time you start a new repo you copy the same files, maintain them in multiple places, and drift out of sync. Claude Code plugins solve this: a plugin is a shareable, versioned package that carries all your customisations and can be installed in any project with one command.

This post covers what plugins are, when to use them, and how to build a real one from scratch — a DevOps helper that ships a deployment skill, a pre-deploy safety hook, and an MCP server connection to your Kubernetes cluster.

Plugins vs Standalone Configuration

Claude Code supports two ways to extend it:

Approach	Skill names	Best for
Standalone (`.claude/` directory)	`/deploy`	One project, personal workflows, quick experiments
Plugins (separate directory with manifest)	`/devops-helper:deploy`	Sharing with team, multiple projects, versioned releases, marketplace distribution

Start with standalone when you are iterating fast on a single project. **Convert to a plugin** when you want to share the same setup across projects or with teammates — or publish to the Anthropic marketplace.

The main trade-off is skill namespacing. Plugin skills are always prefixed with the plugin name (`/devops-helper:deploy`) to prevent conflicts when multiple plugins are installed. Standalone skills have no prefix (`/deploy`).

Plugin Structure

Every plugin is a directory with this layout:

```
devops-helper/
├── .claude-plugin/
│   └── plugin.json          ← manifest (only file that goes in .claude-plugin/)
├── skills/
│   └── deploy/
│       └── SKILL.md         ← skill definition
├── agents/
│   └── incident-responder.md
├── hooks/
│   └── hooks.json           ← event hooks
├── .mcp.json                ← MCP server config
├── monitors/
│   └── monitors.json        ← background monitors
├── bin/                     ← executables added to PATH
├── settings.json            ← default settings applied when plugin is enabled
└── README.md
```

The critical rule: **only** `plugin.json` **goes inside** `.claude-plugin/`. Everything else — skills, agents, hooks — sits at the plugin root. This is the most common structural mistake.

What each component does

Component	What it is
`skills/`	Skills Claude invokes automatically or you call with `/plugin:skill-name`
`agents/`	Subagent definitions for specialised parallel tasks
`hooks/`	Shell scripts that fire on lifecycle events (PreToolUse, PostToolUse, etc.)
`.mcp.json`	MCP server config connecting Claude to external APIs and databases
`monitors/`	Background watchers (log tails, file watchers) that notify Claude as events arrive
`bin/`	Executables added to the Bash tool's PATH during the session
`settings.json`	Default settings applied when the plugin is enabled

The Plugin Manifest

The manifest at `.claude-plugin/plugin.json` is the only required file:

```
{
  "name": "devops-helper",
  "description": "Deployment, Kubernetes, and incident-response tools for DevOps teams",
  "version": "1.0.0",
  "author": {
    "name": "Your Name",
    "email": "you@company.com"
  },
  "homepage": "https://github.com/your-org/devops-helper",
  "repository": "https://github.com/your-org/devops-helper",
  "license": "MIT"
}
```

Field	Purpose
`name`	Plugin identifier and skill namespace prefix
`description`	Shown in the plugin manager when browsing

Field	Purpose
`version`	Bump to trigger updates for installers. Omit to use git SHA
`author`	Attribution — useful for marketplace listings
`homepage` / `repository`	Links shown in the plugin manager

Building a Real Plugin: DevOps Helper

Let's build a plugin that DevOps teams can install across all their projects. It will include:

1. A deploy skill — runs deployment with pre-flight checks
2. A pre-deploy hook — blocks deploys to production without explicit confirmation
3. A Kubernetes MCP server — gives Claude live cluster access
4. A background log monitor — streams app errors to Claude automatically

Step 1: Create the structure

```
mkdir -p devops-helper/.claude-plugin
mkdir -p devops-helper/skills/deploy
mkdir -p devops-helper/skills/rollback
mkdir -p devops-helper/skills/incident-summary
mkdir -p devops-helper/hooks
mkdir -p devops-helper/agents
mkdir -p devops-helper/monitors
```

Step 2: Write the manifest

```
{
  "name": "devops-helper",
  "description": "Deployment safety, Kubernetes access, and incident response for DevOps teams",
  "version": "1.0.0",
  "author": {
    "name": "DevOps Team"
  }
}
```

Step 3: Create the deploy skill

```
<!-- devops-helper/skills/deploy/SKILL.md -->
---
description: >
  Runs a deployment with pre-flight checks and rollback instructions.
  Auto-invokes when the user asks to deploy, release, or push to production.
user-invocable: true
allowed-tools:
  - Bash
  - Read
---

# Deploy Skill

When the user asks to deploy, follow this sequence:

## 1. Read deployment config
Read `deploy.yaml` or `Makefile` to understand the deployment command. If neither exists, ask the user.

## 2. Pre-flight checks
Run these before any deploy:
```bash
# Check current branch
git branch --show-current

# Check for uncommitted changes
git status --porcelain

# Run tests
npm test --silent 2>&1 | tail -10
```

If tests fail or there are uncommitted changes, stop and report. Do not deploy.

3. Identify target environment

Parse $ARGUMENTS for the environment name (staging, production, etc.).

- Default to

  staging

  if none specified
- If

  production

  is specified, ask the user to confirm: “Deploying to production. Type YES to confirm.”

4. Deploy

Run the deployment command. Stream output. Capture exit code.

5. Verify

After deploy, verify the service is healthy:

```
# Check rollout status (Kubernetes)
kubectl rollout status deployment/$SERVICE_NAME -n $NAMESPACE --timeout=120s

# Or check the health endpoint
curl -sf $HEALTH_URL || echo "Health check failed"
```
```

6. Report

Summarise: what was deployed, to which environment, any warnings, and the health check result. If deploy failed: run the rollback skill immediately and report what happened.

````
### Step 4: Create the rollback skill

```markdown
<!-- devops-helper/skills/rollback/SKILL.md -->
---
description: Rolls back the last deployment. Use when a deploy fails or causes issues.
user-invocable: true
allowed-tools:
  - Bash
---

# Rollback Skill

## Steps

1. Identify the last stable deployment:
```bash
kubectl rollout history deployment/$SERVICE_NAME -n $NAMESPACE
````

1 Roll back:

```
kubectl rollout undo deployment/$SERVICE_NAME -n $NAMESPACE
```

1 Wait and verify:

```
kubectl rollout status deployment/$SERVICE_NAME -n $NAMESPACE --timeout=60s
```

1 Report: whether rollback succeeded, which revision is now active, and recommend investigating what caused the failed deploy.

````
### Step 5: Create the incident summary skill

```markdown
<!-- devops-helper/skills/incident-summary/SKILL.md -->
---
description: >
  Generates an incident summary from logs and recent changes.
  Auto-invokes when the user mentions an incident, outage, or asks what went wrong.
user-invocable: true
allowed-tools:
  - Bash
  - Read
---

# Incident Summary Skill

Generate a structured incident summary. Collect:

1. **Timeline**: Run `git log --since="6 hours ago" --oneline` — what changed recently?
2. **Errors**: Read recent error logs from `logs/error.log` or run `kubectl logs --tail=100 -l
app=$SERVICE`
3. **Impact**: Ask the user to describe impact if not obvious from logs

Write a summary in this format:

## Incident Summary — [DATE]
````
````

Status: Ongoing / Resolved

Impact: [What broke, how many users affected]

Timeline:

- HH:MM — [event]
- HH:MM — [event]

Root cause: [One sentence]

Fix applied: [What was done]

Action items:

- [Follow-up task]

```
Keep it factual, under 300 words, suitable for a post-mortem or Slack channel update.
```

Step 6: Add a pre-deploy safety hook

The hook blocks deploys to production unless the user explicitly confirms. It intercepts the `Bash` tool and checks if the command matches a deploy pattern targeting production:

```
// devops-helper/hooks/hooks.json
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "devops-helper/hooks/pre-deploy-check.sh"
          }
        ]
      }
    ],
    "PostToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "devops-helper/hooks/log-commands.sh"
          }
        ]
      }
    ]
  }
}
```

```
#!/bin/bash
# devops-helper/hooks/pre-deploy-check.sh
input=$(cat)
cmd=$(echo "$input" | jq -r '.tool_input.command // ""')

# Check for production deploy patterns
PROD_PATTERNS="prod|production|--env=prod"

if echo "$cmd" | grep -qiE "(deploy|release|kubectl apply|helm upgrade)" && \
   echo "$cmd" | grep -qiE "$PROD_PATTERNS"; then

  jq -n '{
    hookSpecificOutput: {
      hookEventName: "PreToolUse",
      permissionDecision: "ask",
      permissionDecisionReason: "Production deployment detected. Confirm this is intentional before
proceeding."
    }
  }'
  exit 0
fi

exit 0
```

```
#!/bin/bash
# devops-helper/hooks/log-commands.sh
# Append every Bash command to an audit log
input=$(cat)
cmd=$(echo "$input" | jq -r '.tool_input.command // ""')
ts=$(date -u +"%Y-%m-%dT%H:%M:%SZ")

echo "[$ts] $cmd" >> "${AUDIT_LOG:-/tmp/claude-commands.log}"
exit 0
```

Step 7: Add a Kubernetes MCP server

The `.mcp.json` file at the plugin root connects Claude to your cluster:

```
{
  "mcpServers": {
    "kubernetes": {
      "command": "npx",
      "args": ["-y", "@anthropic/mcp-server-kubernetes"],
      "env": {
        "KUBECONFIG": "${KUBECONFIG:-~/.kube/config}"
      }
    },
    "pagerduty": {
      "command": "npx",
      "args": ["-y", "@anthropic/mcp-server-pagerduty"],
      "env": {
        "PAGERDUTY_API_KEY": "${PAGERDUTY_API_KEY}"
      }
    }
  }
}
```

With this in place, Claude can run `kubectl` commands, read pod logs, and check deployment status via MCP tools — without you manually setting up shell access.

Step 8: Add a background error log monitor

The monitor streams new error log lines to Claude during the session:

```
// devops-helper/monitors/monitors.json
[
  {
    "name": "app-errors",
    "command": "tail -F logs/error.log 2>/dev/null || echo 'No error.log found'",
    "description": "Application error log — new lines are delivered to Claude automatically"
  }
]
```

Claude receives each new log line as a notification, letting it flag new errors without you having to ask.

Final plugin structure

```
devops-helper/
├── .claude-plugin/
│   └── plugin.json
├── skills/
│   ├── deploy/
│   │   └── SKILL.md
│   ├── rollback/
│   │   └── SKILL.md
│   └── incident-summary/
│       └── SKILL.md
├── hooks/
│   ├── hooks.json
│   ├── pre-deploy-check.sh
│   └── log-commands.sh
├── monitors/
│   └── monitors.json
├── .mcp.json
└── README.md
```

Testing Your Plugin

Test locally without installing using `--plugin-dir`:

```
# Load from directory
claude --plugin-dir ./devops-helper

# Load from a zip archive (Claude Code v2.1.128+)
claude --plugin-dir ./devops-helper.zip

# Load multiple plugins at once
claude --plugin-dir ./devops-helper --plugin-dir ./other-plugin
```

Inside the session, verify everything loaded:

```
# List skills
/help

# Test the deploy skill
/devops-helper:deploy staging

# Test the rollback skill
/devops-helper:rollback

# Check agents are visible
/agents
```

After making changes, reload without restarting:

```
/reload-plugins
```

If something does not work, check the structure first — the most common issue is placing `skills/` or `hooks/` inside `.claude-plugin/` instead of the plugin root.

Installing Plugins

Once the plugin is in a git repository, anyone can install it:

```
# From GitHub
claude plugin add github:your-org/devops-helper

# From the official Anthropic marketplace
claude plugin add @anthropic/deploy-helper

# Project-scoped install (only active in this repo)
claude plugin add github:your-org/devops-helper --project
```

Managing installed plugins:

```
claude plugin list                            # see what's installed
claude plugin update github:your-org/devops-helper
claude plugin update --all                    # update everything
claude plugin remove github:your-org/devops-helper
```

Or use the in-app plugin manager:

```
/plugin
```

The `/plugin` command opens a browser with Discover, Installed, and Updates tabs.

Converting Existing Config to a Plugin

If you have skills and hooks in `.claude/` already, migration is straightforward:

```
# 1. Create plugin structure
mkdir -p my-plugin/.claude-plugin

# 2. Write manifest
cat > my-plugin/.claude-plugin/plugin.json << 'EOF'
{
  "name": "my-plugin",
  "description": "Migrated from standalone .claude/ config",
  "version": "1.0.0"
}
EOF

# 3. Copy existing components
cp -r .claude/skills  my-plugin/
cp -r .claude/agents  my-plugin/
cp -r .claude/commands my-plugin/

# 4. Migrate hooks
mkdir my-plugin/hooks
# Copy the hooks object from .claude/settings.json into my-plugin/hooks/hooks.json
```

What changes after migration:

Before (standalone)	After (plugin)
`/deploy`	`/my-plugin:deploy`
Copy files per project	`claude plugin add github:you/my-plugin`
Manual updates	`claude plugin update --all`
Only in one project	Available everywhere

Publishing to the Anthropic Marketplace

When your plugin is ready to share publicly:

1. Push to a public GitHub repository
2. Add a

   README.md

   with installation instructions and a usage guide
3. Submit via the in-app form:
   - Claude.ai:

     Settings → Plugins → Submit
   - Platform console:

     platform.claude.com/plugins/submit

Once listed, users can install with `/plugin install your-plugin-name` or `claude plugin add @your-org/your-plugin`.

For team-internal distribution, host the plugin in a private GitHub repo and configure the team marketplace URL in project settings — no public listing required.

When to Build a Plugin

```
flowchart TD
    Q1{Do you need this\nacross multiple projects?} -->|Yes| Q2{Share with\nothers?}
    Q1 -->|No| Standalone[Standalone .claude/ config]
    Q2 -->|Yes| Plugin[Build a plugin]
    Q2 -->|No| Q3{More than one\nproject?}
    Q3 -->|Yes| Plugin
    Q3 -->|No| Standalone
```

Build a plugin when:

- You want the same skills and hooks in every project without copying files
- You are building tools for a team and want them to install with one command
- You want version control over your tooling (bump

  version

  to push updates)
- You want to publish to the Anthropic marketplace for the community

Stick with standalone when:

- The configuration is specific to one project and will never be reused
- You are experimenting and not sure if the skill is worth keeping
- You want short skill names without namespace prefixes

The practical path: build standalone first, convert to a plugin once the skill proves its value.

Plugin Ideas to Build Next

Plugin	What it does
`git-helper`	PR description writer, commit message generator, branch namer
`test-enforcer`	Auto-run tests on edit, enforce coverage thresholds, generate missing tests
`security-scanner`	IDOR checker, injection auditor, pre-commit security review
`doc-writer`	Generate README, API docs, changelogs from code
`infra-ops`	Terraform plan reviewer, cost estimator, drift detector
`incident-bot`	Post-mortem template filler, on-call summary, PagerDuty integration

Each of these follows the same pattern: a few skill files, an optional hook, and an MCP server connection if external data is needed.

GitHub Actions Bot: Automated @claude Integration

The `@claude` bot on GitHub allows you to tag Claude directly in pull requests and issues. The bot runs in GitHub Actions, reads full context, and either comments with analysis or opens commits and PRs with fixes.

What @claude Can Do

Tag `@claude` in any PR comment, issue comment, or review comment:

```
@claude please review this PR for security issues
```

The bot:

1 Reads context

— PR diff, issue description, conversation history

2 Runs Claude

— in a GitHub Actions environment with full tool access

3 Responds

— posts findings as a comment, or opens a PR with fixes

Examples of @claude Usage

Code review:

```
@claude review this PR for bugs and security issues
```

Bug fix:

```
@claude this test is failing — can you fix it and push the fix?
```

Feature implementation:

```
@claude look at issue #234 and implement the described feature
```

Documentation:

```
@claude generate API documentation for this module
```

Performance analysis:

```
@claude analyze this code for performance issues and suggest optimizations
```

Installation

Option 1: One-Command Setup (Easiest)

From within a Claude Code session in your repo:

```
/install-github-app
```

This:

1. Installs the Claude GitHub App to your repository
2. Creates a PR containing .github/workflows/claude.yml
3. Prompts you to add ANTHROPIC_API_KEY as a GitHub Secret
4. Merges the PR — setup is complete

Option 2: Manual Setup

1. Go to github.com/apps/claude and install the app
2. Create .github/workflows/claude.yml manually:

```
name: Claude Bot
on:
  issue_comment:
    types: [created, edited]
  pull_request_review_comment:
    types: [created, edited]

jobs:
  claude:
    if: contains(github.event.comment.body, '@claude')
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Claude Bot
        run: |
          claude -p "$(cat <<'PROMPT'
          You are the Claude bot on GitHub. Read the issue/PR context and the user's request.

          User request: ${{ github.event.comment.body }}

          Respond with:
          - Analysis of the problem
          - Specific recommendations or fixes
          - Any follow-up questions
          PROMPT
          )" \
            --max-budget-usd 5.00
        env:
          ANTHROPIC_API_KEY: ${{ secrets.ANTHROPIC_API_KEY }}

      - name: Post Comment
        uses: actions/github-script@v6
        with:
          github-token: ${{ secrets.GITHUB_TOKEN }}
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: 'Claude review complete. See output above.'
            })
```

1 Add

ANTHROPIC_API_KEY

as a repository secret:

- Go to

  Settings → Secrets and variables → Actions

- Click

  New repository secret

- Name:

  ANTHROPIC_API_KEY

- Value: your API key

How @claude Works in Practice

PR Review Workflow:

```
Developer: Opens PR #42

@claude please review for security issues

GitHub Actions triggers → Claude runs in isolated environment → Claude reads PR diff → Claude posts
findings as comment
```

Example bot response:

```
## Claude Security Review

### Findings

**Critical Issues:**
- [src/auth.ts:45] — SQL injection vulnerability in query construction. Use parameterised queries instead
 of string concatenation.
- [src/api.ts:123] — API key logged to console in error handler. Remove before production.

**Warnings:**
- [src/db.ts:67] — Password hashing algorithm is weak. Upgrade to bcrypt with 12+ rounds.

**Passing checks:**
✓ No hardcoded secrets
✓ CORS properly configured
✓ Input validation on all endpoints

### Next steps
1. Fix the SQL injection immediately
2. Remove the API key logging
3. Update password hashing
```

Running @claude Automatically on Every PR

Create a workflow that runs without needing to tag `@claude`:

```
name: Automatic Claude Review
on:
  pull_request:
    types: [opened, synchronize]

jobs:
  claude-review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Review PR
        run: |
          claude -p "Review this PR diff for:
          1. Bugs and logic errors
          2. Security vulnerabilities
          3. Performance issues
          4. Best practice violations

          Be specific with file:line references." \
            --max-budget-usd 5.00 > review-output.txt
        env:
          ANTHROPIC_API_KEY: ${{ secrets.ANTHROPIC_API_KEY }}

      - name: Comment Review
        uses: actions/github-script@v6
        with:
          github-token: ${{ secrets.GITHUB_TOKEN }}
          script: |
            const fs = require('fs');
            const review = fs.readFileSync('review-output.txt', 'utf8');
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: review
            })
```

Bot with Auto-Fixes

For simple issues, the bot can open a PR with fixes:

```
name: Claude Auto-Fix
on:
  issue_comment:
    types: [created]

jobs:
  auto-fix:
    if: contains(github.event.comment.body, '@claude fix')
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          token: ${{ secrets.GITHUB_TOKEN }}

      - name: Create Fix Branch
        run: git checkout -b claude-fix-${{ github.event.issue.number }}

      - name: Run Claude
        run: |
          claude -p "Fix the issue described. Make minimal, focused changes." \
            --max-budget-usd 3.00
        env:
          ANTHROPIC_API_KEY: ${{ secrets.ANTHROPIC_API_KEY }}

      - name: Commit and Push
        run: |
          git add -A
          git commit -m "Claude auto-fix for issue #${{ github.event.issue.number }}"
          git push origin claude-fix-${{ github.event.issue.number }}

      - name: Open PR
        uses: actions/github-script@v6
        with:
          github-token: ${{ secrets.GITHUB_TOKEN }}
          script: |
            github.rest.pulls.create({
              owner: context.repo.owner,
              repo: context.repo.repo,
              title: 'Claude auto-fix for issue #${{ github.event.issue.number }}',
              body: 'Automated fix generated by Claude bot',
              head: 'claude-fix-${{ github.event.issue.number }}',
              base: 'main'
            })
```

Limiting Bot Costs

Each run can cost $0.50 to $5 depending on PR size. Control costs with:

```
- name: Claude Review
  run: |
    claude -p "..." \
      --max-budget-usd 2.00  # stops if cost exceeds $2
```

Or disable the bot for large PRs:

```
jobs:
  claude-review:
    if: github.event.pull_request.additions < 500  # only run if < 500 lines added
    runs-on: ubuntu-latest
```

Bot Permissions and Limits

The bot runs with:

- GitHub token
  — reads issues, PRs, opens PRs, posts comments
- Anthropic API key
  — makes Claude requests
- Read access
  — can read all files in the repo
- Write access
  — can create commits and PRs on
  claude-*
  branches

The bot cannot:

- Force-push
- Delete branches
- Modify GitHub Actions workflows (security protection)
- Access secrets (only
  ANTHROPIC_API_KEY
  and
  GITHUB_TOKEN
)

Troubleshooting @claude Bot

Bot doesn’t respond:

- Check that
  ANTHROPIC_API_KEY
  is set in Settings → Secrets
- Verify the app is installed: Settings → Installed GitHub Apps
- Check workflow logs: Actions tab → Claude Bot workflow

Bot makes unwanted changes:

- Use branch protection: Settings → Branches → add rule for
  claude-*
  to require PR review
- Limit bot access: disable auto-fix workflows, require manual approval

High costs:

- Set

--max-budget-usd

to a reasonable limit ($2–$5 per run)

- Run on PRs < 500 lines only: add

  if:

  condition
- Schedule runs for critical reviews only, not every PR

The Claude Code /loop Command: In-Session Automation Explained

Published 2026-04-26 | Source: https://blog.devops-monk.com/2026/04/claude-code-loop-command/

What the /loop command actually does, when it is useful, its hard limits, and how it compares to cron jobs for persistent automation.

The `/loop` command in Claude Code lets you schedule a prompt to repeat at a regular interval within an active session. It is one of the less-documented features, and it is also one of the most misunderstood — particularly around what it cannot do.

This post explains what `/loop` actually is, the three scenarios where it genuinely earns its keep, and where cron jobs remain the better choice.

What /loop Does

```
/loop 5m check if the Vite dev server is still running and report any new errors
```

This runs the prompt immediately, then repeats it every 5 minutes for the duration of your session. The interval can be in seconds (`30s`), minutes (`5m`), or hours (`2h`).

Dynamic (self-paced) mode — omit the interval and Claude decides when to run again based on what it observed:

```
/loop check the build and report when it changes state
```

In dynamic mode, Claude wakes up, checks the condition, and schedules its next check based on what it found — polling faster when something is actively changing, backing off when nothing has changed. This is useful for monitoring tasks where the right polling interval depends on the current state rather than a fixed schedule.

Running `/loop` with no arguments activates a built-in maintenance prompt — Claude becomes your project caretaker:

- Continues any unfinished work from the conversation
- Tends to the current branch's PR — review comments, failed CI, merge conflicts
- Runs cleanup passes (bug hunts, simplifications) when nothing else is pending

Claude will not start new initiatives or push/delete anything unless the conversation already authorised it.

Customising the maintenance prompt: drop a `loop.md` file to define your own default:

```
.claude/loop.md       ← project-level (takes precedence if both exist)
~/.claude/loop.md     ← user-level (applies to every project)
```

Write it in plain markdown — whatever instructions you put there replace the built-in maintenance prompt when you run `/loop` with no arguments.

Hard limits:

- Maximum 50 iterations per loop
- Auto-expires after 7 days (one final fire, then gone)
- Dies when you close the Claude Code session

That last point is the most important one. `/loop` is session-scoped. The moment you close the terminal or the session times out, the loop stops. It is not a daemon. It is not a cron job. It is a within-session automation tool.

```
flowchart LR
    subgraph Session[Active Claude Code Session]
        Loop[loop command] -->|every N minutes| Prompt[Runs prompt]
        Prompt --> Result[Produces result]
    end
    Close[Session closes] -->|loop dies| End[Automation stops]
    Session --> Close
```

When It Is Actually Useful

1. Build and CI monitoring during active development

You are in the middle of a refactor and you want to know immediately if the build breaks — without switching tabs every few minutes.

```
/loop 3m check if our TypeScript build is passing and summarise any new errors
```

Claude runs `tsc --noEmit` or reads your CI output, reports back, and you stay in flow. The moment you stop working and close the session, the loop stops — which is exactly what you want.

2. Accumulating code review notes across a long session

```
/loop 20m /review
```

Every 20 minutes Claude reviews the current state of your changes and adds new observations to its running notes. By the end of the session you have a comprehensive review built up incrementally rather than one big scan at the end.

3. Monitoring a deployment in progress

```
/loop 1m check kubectl rollout status deployment/api-server and report when complete
```

Claude watches the deployment, reports progress every minute, and you stay informed without polling yourself. Once the deployment finishes (or you close the session), the loop ends naturally.

/loop vs Cron: Choosing the Right Tool

```
flowchart TD
    Task[Automation task] --> Q1{Runs while you\nare actively working?}
    Q1 -->|Yes| Q2{Session-bound\nis acceptable?}
    Q2 -->|Yes| Loop[Use /loop\nSimple, no setup]
    Q2 -->|No| Cron[Use cron\nPersists beyond session]
    Q1 -->|No - runs overnight\nor on schedule| Cron
```

Scenario	Use /loop	Use cron
Monitor build during active work	✅	
Morning briefing while you sleep		✅
Watch a 30-minute deployment	✅	
Weekly infrastructure audit		✅
Accumulate review notes in a session	✅	
Daily Slack summary		✅

The rule is simple: if the task needs to run while you are actively working in Claude Code, use `/loop`. If it needs to run without you present, use cron.

The /schedule Skill: Persistent Loops Beyond the Session

The `/schedule` skill bridges the gap between `/loop` and a full cron setup. Unlike `/loop` (which dies when the session closes), `/schedule` creates **routines** — recurring remote agents that run on a cron schedule and survive session close. They run even when you are not at your desk.

```
# Schedule a daily 9am standup summary (persists beyond session)
/schedule daily-standup "0 9 * * 1-5" "Read git log from yesterday, write a 3-bullet standup update, and
post to Slack"

# List all scheduled routines
/schedule list

# Remove a routine
/schedule delete daily-standup
```

Tool	Survives session close?	Needs your own server?	Best for
`/loop`	No	No	Active monitoring while you work
`/schedule`	Yes	No (Anthropic-hosted)	Recurring background automation
cron + `claude -p`	Yes	Yes	Custom tools, your own infrastructure

Use `/schedule` when you want the persistence of cron without managing infrastructure. Use `/loop` when the automation only makes sense while you are actively working.

Combining /loop with Hooks

`/loop` becomes more powerful when combined with hooks. A hook can run tests after every file edit; `/loop` can then summarise the test results every few minutes:

```
# While hooks auto-run tests on every edit...
/loop 10m summarise the test results from the last 10 minutes and tell me if anything is consistently
failing
```

This gives you a periodic human-readable summary on top of the deterministic hook behaviour — the hook catches individual failures, the loop gives you the bigger picture.

Practical Usage Tips

Be specific about what to check. “Check for errors” produces shallow results. “Check if the API server is responding to health checks and report the response time” produces actionable output.

Set an interval that matches the task. Build checks every 30 seconds is excessive. Deployment status every 5 minutes means you wait longer than needed. Match the interval to how fast the underlying state changes.

Use `/loop` to bridge gaps in your workflow. The best uses are for things you would otherwise manually check every few minutes — not for creating new workflows that require their own infrastructure.

Stop a loop with `/skip`. If you want to cancel the current loop before it completes all iterations, use `/skip` to stop the next scheduled run.

What /loop Is Not

- It is not a replacement for CI/CD monitoring (use PagerDuty or CI notifications)
- It is not a persistent agent (use cron +

  claude -p

  for that)
- It is not suitable for tasks that need to run when you are not at your desk
- It is not infinitely scalable — 50 iterations maximum

Think of `/loop` as an in-session assistant that watches something so you do not have to, for as long as you are actively working. For everything else, cron is the right answer.

Part 4: Performance and Cost

Stop Burning Tokens: A Practical Guide to Claude Code Cost Optimization

Published 2026-04-26 | Source: https://blog.devops-monk.com/2026/04/claude-code-token-optimization/

Five concrete habits that cut Claude Code token usage by up to 50%. Covers CLAUDE.md trimming, model tiering, targeted queries, front-loaded prompts, and subagent scoping.

Token usage with Claude Code follows a frustrating pattern: costs are not spread evenly — they cluster around a handful of bad habits. Most developers using Claude Code daily are burning 40–60% more tokens than they need to, simply because of how they phrase prompts, what they put in `CLAUDE.md`, and which model they reach for by default.

This guide covers five concrete changes that make an immediate difference.

Why Tokens Are Worth Caring About

Every message you send in a Claude Code session includes:

- The full conversation history up to that point
- Every file Claude has read during the session
- The content of your

  CLAUDE.md

  files
- Tool outputs from previous commands
- MCP tool definitions for configured servers

That means a long session is not just expensive on the last message — it gets more expensive on every single message, because history accumulates. A 3-hour debugging session where Claude reads ten files can cost 10x what the same work would cost if you cleared context between tasks.

```
flowchart LR
    subgraph SessionCost[What Goes Into Every Message]
        H[Conversation history]
        F[Files Claude read]
        C[CLAUDE.md content]
        T[Tool outputs]
        M[MCP definitions]
    end
    SessionCost -->|multiplied by| MSG[Every new message]
    MSG --> COST[Token cost compounds over time]
```

Habit 1: Trim Your CLAUDE.md

`CLAUDE.md` loads into every context window, every session, on every message. If your file is 400 lines, you are paying for 400 lines of context on every single message — even when 300 of those lines are things Claude already knows.

Common bloat to cut:

- Framework documentation ("React is a library for building UIs") — Claude knows this
- Obvious best practices ("write readable code", "use meaningful variable names")
- Package version lists Claude can read from

  package.json
- Architectural diagrams that are already in your README

What to keep:

- Commands Claude cannot guess:

  npm run deploy:staging

  ,

  ./scripts/seed-db.sh
- Team conventions that differ from defaults: "we use 4-space indentation, not 2"
- Project-specific gotchas: "the auth service requires

  NODE_ENV=production

  to initialise"
- Model routing rules (see Habit 3 below)

Target: under 150 lines. Studies of Claude's instruction-following show adherence drops noticeably beyond that length — so a shorter, sharper file is both cheaper and more effective.

Habit 2: Ask for Specific Lines, Not Whole Files

When Claude reads a file, the entire file enters the context window. If you ask "what does this function in auth.ts do?" and auth.ts is 800 lines, all 800 lines are loaded — even if the function is at line 45.

Instead, be specific:

```
Check lines 40 to 65 in src/auth/token-service.ts — why does the refresh logic fail when the user is logged in on two devices?
```

This is especially important for:

- Configuration files (often large, usually only partially relevant)
- Migration files (you usually care about one migration)
- Log files (you need the last 50 lines, not 10,000)

```
# Instead of: cat app.log | claude -p "debug this"
tail -100 app.log | claude -p "what errors appear here and why?"
```

Habit 3: Route Tasks to the Right Model

The default behaviour in Claude Code is to use a capable model for everything. That is correct for complex work but wildly over-specified for simple tasks.

```
flowchart TD
    Task[New task] --> Q1{Requires deep reasoning or architecture?}
    Q1 -->|Yes| Opus[Claude Opus\nArchitecture decisions\nComplex refactors\nDeep debugging]
    Q1 -->|No| Q2{Standard implementation or debugging?}
    Q2 -->|Yes| Sonnet[Claude Sonnet\nCode implementation\nBug fixes\nCode review\nTest writing]
    Q2 -->|No| Haiku[Claude Haiku\nRenamings\nFormatting\nSimple lookups\nSummaries]
```

Add this routing table to your `CLAUDE.md` so Claude applies it automatically:

```
## Model Routing

Use the cheapest model that can handle the task:

- **Haiku:** renaming, formatting, simple lookups, one-line changes, summarising output
- **Sonnet:** standard implementation, debugging, code review, writing tests
- **Opus:** architecture decisions, complex multi-file refactors, security analysis
```

You can also switch mid-session:

```
/model claude-haiku-4-5    # switch to Haiku for a simple task
/model claude-sonnet-4-6   # back to Sonnet for implementation
```

Habit 4: Front-Load Your Prompts

The most expensive pattern in any Claude Code session is back-and-forth clarification. Each exchange reloads the full context. A prompt that generates three rounds of clarification costs roughly 4x as much as a prompt that generates one complete answer.

Expensive pattern:

```
You: "Refactor the payment flow"
Claude: "Which part? The card form or the API layer?"
You: "The API layer"
Claude: "What framework? What's the error?"
You: "Express, and it times out on large orders"
```

That is three context loads before any work begins.

Cheaper pattern:

```
Before writing any code, confirm your understanding:
- I want to refactor the payment API layer in src/api/payments/
- It uses Express and times out on orders with more than 50 line items
- I need the fix to not change the public API contract
- Tell me if anything is ambiguous before you start
```

One message establishes full context. Claude confirms, you approve, work begins. One context load.

```
sequenceDiagram
    participant Dev as Developer
    participant Claude as Claude Code

    Note over Dev,Claude: Expensive pattern (4x context loads)
    Dev->>Claude: Refactor the payment flow
    Claude-->>Dev: Which part?
    Dev->>Claude: The API layer
    Claude-->>Dev: What framework?
    Dev->>Claude: Express, it times out on large orders
    Claude-->>Dev: OK, making changes...

    Note over Dev,Claude: Cheap pattern (2x context loads)
    Dev->>Claude: Refactor payment API in src/api/payments/ - Express - times out on 50+ line items -
keep public API unchanged - confirm understanding first
    Claude-->>Dev: Confirmed. Making changes...
```

Habit 5: Scope Your Subagent Briefs

When Claude spawns subagents to investigate your codebase, vague instructions cause expensive exploration. A subagent told to “look into the auth system” may read dozens of files searching for context it does not need.

Vague (expensive):

```
Use a subagent to investigate why login is slow
```

Scoped (cheap):

```
Use a subagent to investigate login latency.
Read only: src/auth/login-handler.ts, src/middleware/session.ts, and src/db/user-queries.ts
Report: which function takes the longest and why
Do not read any other files
```

Tight scoping reduces subagent context to just the relevant files, cutting costs dramatically on large codebases.

Habit 6: Disable Unused MCP Servers

Every MCP server you have enabled loads its full tool definition into context on every message — whether you use that server or not. Five connected MCP servers can add thousands of tokens per message as a constant overhead.

```
# Check what MCP servers are loaded and their context cost
/mcp

# Disable servers you are not using in this session
/mcp disable google-drive
/mcp disable slack

# Re-enable when needed
/mcp enable slack
```

The practical rule: **if you have not used an MCP server in the last two hours, disable it**. The savings are proportional to how many servers you have connected and how verbose their tool schemas are.

Bonus: The Advisor Tool Pattern

For long agentic runs where you need high-quality decision-making but most of the work is routine execution, the **Advisor Tool** is a powerful cost lever. A fast, cheap executor model (Haiku) does the file reading, editing, and command running. When it hits a decision point requiring judgment, it calls an intelligent advisor model (Opus 4.7) for a short plan.

The advisor generates a plan of roughly 400–700 tokens. The executor then carries out the full implementation at its much lower rate. You get near-Opus quality for the hard decisions at a fraction of the cost of running Opus throughout.

This is particularly effective for large refactors and long dependency audits — tasks with many mechanical steps punctuated by a few genuinely hard decisions.

```
import anthropic

client = anthropic.Anthropic()

response = client.messages.create(
    model="claude-haiku-4-5-20251001",   # cheap executor
    max_tokens=4096,
    extra_headers={"anthropic-beta": "advisor-tool-2026-03-01"},
    tools=[
        {
            "type": "advisor",
            "advisor_model": "claude-opus-4-7",  # smart advisor
        }
    ],
    messages=[
        {"role": "user", "content": "Refactor our test suite to use async patterns throughout"}
    ]
)
```

Tracking Your Usage

Make it a habit to check costs before ending a session:

```
/cost      # estimated session cost so far
/usage     # detailed breakdown by message
/context   # see what is consuming context space
```

The `/context` command is particularly useful — it shows you exactly what is taking up space, so you can target the right optimisation.

Quick Reference

Habit	Impact	Effort
Trim CLAUDE.md to under 150 lines	High	One-time
Use line ranges instead of full files	High	Low per-prompt
Route tasks to Haiku/Sonnet/Opus	Medium	One-time setup
Front-load context in prompts	High	Medium habit change
Scope subagent file access	Medium	Low per-task
Run `/clear` between unrelated tasks	High	Low habit

Habit	Impact	Effort
Disable unused MCP servers	Medium	Low habit

The biggest gains come from CLAUDE.md trimming and front-loading. Do those two first and check your usage dashboard in a week — you will see the difference immediately.

Claude Prompt Caching: Cut Your API Costs by 90%

Published 2026-05-03 | Source: https://blog.devops-monk.com/2026/05/claude-prompt-caching-guide/

A practical guide to implementing prompt caching with the Claude API — including ephemeral vs. persistent caching, automatic caching, and combining it with the Batch API for up to 95% cost reduction.

If you are calling the Claude API repeatedly with a large system prompt, a big document, or a long codebase context — and you are not using prompt caching — you are paying full price every time for content that has not changed. Prompt caching stores a prefix of your prompt server-side and charges 90% less to read it back on every subsequent request.

For applications that repeatedly process the same context, this is the single highest-impact API optimisation available.

How It Works

Every Claude API request has two cost components: input tokens and output tokens. Input tokens include your system prompt, any documents you include, and the conversation history. If you include the same 50,000-token codebase context on every request, you pay for all 50,000 tokens every time.

Prompt caching changes this. You mark a specific point in your prompt as a cache breakpoint. Anthropic stores everything up to that point server-side. On the next request that hits the same prefix, you pay the cache read rate instead of the full input rate.

```
sequenceDiagram
    participant App as Your App
    participant API as Claude API

    Note over App,API: First request — cache write
    App->>API: System prompt (50k tokens) + question [cache_control: ephemeral]
    API->>API: Store prefix in cache
    API-->>App: Response. Charged: 1.25x input rate for cached tokens

    Note over App,API: Second request — cache hit
    App->>API: Same prefix + new question
    API->>API: Found in cache
    API-->>App: Response. Charged: 0.1x input rate for cached tokens
```

Cache pricing:

	Write	Read
Ephemeral (5-min TTL)	1.25x input rate	0.1x input rate
Persistent (1-hour TTL)	2x input rate	0.1x input rate

The write cost is slightly higher than a normal request, but every subsequent read within the TTL costs 90% less. On any prefix you use more than twice within the TTL window, caching pays for itself.

Basic Implementation

Add a `cache_control` field to any content block you want to cache. The cache breakpoint sits at the end of that block.

Python

```
import anthropic

client = anthropic.Anthropic()

# Load your large context once
with open("codebase-summary.txt") as f:
    codebase_context = f.read()

def ask_about_codebase(question: str) -> str:
    response = client.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=1024,
        system=[
            {
                "type": "text",
                "text": "You are an expert software engineer helping with this codebase.",
            },
            {
                "type": "text",
                "text": codebase_context,
                "cache_control": {"type": "ephemeral"}  # cache everything up to here
            }
        ],
        messages=[{"role": "user", "content": question}]
    )
    return response.content[0].text

# First call: cache write (1.25x rate for the large context)
answer1 = ask_about_codebase("Why is the auth service slow?")

# Second call: cache hit (0.1x rate for the large context)
answer2 = ask_about_codebase("How does session management work?")

# The cache hit saves ~89% on the large context tokens
```

TypeScript

```
import Anthropic from "@anthropic-ai/sdk";
import { readFileSync } from "fs";

const client = new Anthropic();
const codebaseContext = readFileSync("codebase-summary.txt", "utf-8");

async function askAboutCodebase(question: string): Promise<string> {
  const response = await client.messages.create({
    model: "claude-sonnet-4-6",
    max_tokens: 1024,
    system: [
      {
        type: "text",
        text: "You are an expert software engineer helping with this codebase.",
      },
      {
        type: "text",
        text: codebaseContext,
        cache_control: { type: "ephemeral" },
      },
    ],
    messages: [{ role: "user", content: question }],
  });

  return (response.content[0] as { text: string }).text;
}
```

Cache TTL: Ephemeral vs. Persistent

Ephemeral (5-minute TTL) is the default. It is the right choice when:

- You are running an interactive session where questions come rapidly
- You are processing a document and asking many questions in quick succession
- Cost per write is a concern (1.25x vs 2x)

Persistent (1-hour TTL) costs more to write but keeps the cache alive longer. It is the right choice when:

- Requests come less frequently (every few minutes rather than every few seconds)
- You have a shared context used by multiple users (e.g., a documentation assistant)
- The write cost is small compared to the number of reads you expect

```
# Ephemeral (5-min TTL) — default
"cache_control": {"type": "ephemeral"}

# Persistent (1-hour TTL) — higher write cost, longer window
"cache_control": {"type": "persistent"}
```

Multi-Turn Conversations with Caching

For multi-turn conversations, you want to cache both the stable system context and the growing conversation history. The key is re-sending the full conversation with `cache_control` on the most recent complete exchange.

```
import anthropic

client = anthropic.Anthropic()

system_prompt = "You are a DevOps expert. Help the user debug their infrastructure."
large_runbook = open("runbook.md").read()

conversation_history = []

def chat(user_message: str) -> str:
    conversation_history.append({"role": "user", "content": user_message})

    # Mark the last user message as a cache point so history
    # is cached up to this point for the next turn
    messages_with_cache = []
    for i, msg in enumerate(conversation_history):
        if i == len(conversation_history) - 1 and msg["role"] == "user":
            messages_with_cache.append({
                "role": msg["role"],
                "content": [
                    {
                        "type": "text",
                        "text": msg["content"],
                        "cache_control": {"type": "ephemeral"}
                    }
                ]
            })
        else:
            messages_with_cache.append(msg)

    response = client.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=2048,
        system=[
            {"type": "text", "text": system_prompt},
            {
                "type": "text",
                "text": large_runbook,
                "cache_control": {"type": "ephemeral"}  # always cached
            }
        ],
        messages=messages_with_cache
    )

    assistant_message = response.content[0].text
    conversation_history.append({"role": "assistant", "content": assistant_message})
    return assistant_message
```

Automatic Caching

As of February 2026, Anthropic offers **automatic caching** — add one `cache_control` field to your system prompt and the infrastructure automatically advances the cache breakpoint as your context grows. You do not need to manually track which turns to cache.

```
response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    system=[
        {
            "type": "text",
            "text": large_context,
            "cache_control": {"type": "ephemeral"}  # that's it — auto-managed from here
        }
    ],
    messages=conversation_history
)
```

With automatic caching, the server detects the longest common prefix between your current request and recent cached requests, and serves from cache accordingly. This is the recommended approach for new integrations.

Workspace-level isolation (February 5, 2026): Prompt caches are now scoped to the workspace that created them rather than the entire organisation. If you share an Anthropic organisation across multiple teams or environments (e.g., staging and production workspaces), each workspace maintains its own independent cache. A cache write in your staging workspace will not be read by your production workspace — which is the correct behaviour for data separation, but worth understanding if you were relying on cross-workspace cache sharing.

Checking Cache Performance

The API response includes cache usage metrics so you can verify your caching is working:

```
response = client.messages.create(...)

usage = response.usage
print(f"Input tokens:          {usage.input_tokens}")
print(f"Cache write tokens:    {usage.cache_creation_input_tokens}")
print(f"Cache read tokens:     {usage.cache_read_input_tokens}")

# Cache hit rate
total = usage.input_tokens + usage.cache_creation_input_tokens + usage.cache_read_input_tokens
if total > 0:
    hit_rate = usage.cache_read_input_tokens / total
    print(f"Cache hit rate:        {hit_rate:.1%}")
```

A well-tuned implementation targeting a large stable prefix should show 80–95% of tokens as cache reads after the first request.

Why Your Cache Might Not Be Hitting

If `cache_read_input_tokens` stays at 0 across repeated requests, one of these is usually the cause:

- Under the minimum threshold

  — Cache breakpoints require at least ~1,024 tokens before the marker. If your system prompt and documents add up to less than that,

  cache_control

  is silently ignored. Pad or combine content to cross the threshold.

- TTL expired between requests

— The 5-minute ephemeral TTL resets if requests come more than 5 minutes apart. You always pay the write rate on a cold cache. Switch to

"type": "persistent"

(1-hour TTL) if your request cadence is slower than one per few minutes.

- Dynamic content before the cache marker

  — A single character difference before the

  cache_control

  block invalidates the prefix match. Timestamps, request IDs, or per-user text inserted before the cached block break the cache on every request. Move all dynamic content

  after

  the cache marker.

- Cross-workspace isolation

  — Since February 5, 2026, caches are scoped per workspace within an organisation. A cache written in your staging workspace is invisible to your production workspace. This is correct behaviour for data separation, but will break any cross-environment cache sharing you relied on before that date.

```
# Check cache performance in the API response
print(f"Cache read tokens:  {response.usage.cache_read_input_tokens}")   # want this > 0
print(f"Cache write tokens: {response.usage.cache_creation_input_tokens}")
# If cache_read is always 0, check the four causes above
```

DevOps Use Cases

Caching a Codebase for a Review Session

```
import anthropic
import subprocess

client = anthropic.Anthropic()

# Get all TypeScript files
ts_files = subprocess.check_output(
    ["find", "src", "-name", "*.ts", "-not", "-path", "*/node_modules/*"],
    text=True
).strip().split("\n")

# Build context from all files
codebase = []
for path in ts_files:
    try:
        content = open(path).read()
        codebase.append(f"=== {path} ===\n{content}")
    except Exception:
        pass

full_context = "\n\n".join(codebase)

def review_pr_changes(diff: str) -> str:
    """Review a git diff against cached codebase context."""
    response = client.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=2048,
        system=[
            {
                "type": "text",
                "text": "You are a senior engineer reviewing code changes. "
                        "You have full context of the codebase below.",
            },
            {
                "type": "text",
                "text": full_context,
                "cache_control": {"type": "ephemeral"}
            }
        ],
        messages=[{
            "role": "user",
            "content": f"Review this diff for bugs, security issues, and improvements:\n\n{diff}"
        }]
    )
    return response.content[0].text
```

Caching Documentation for a Support Bot

```
docs_content = ""
for doc_file in Path("docs").rglob("*.md"):
    docs_content += f"\n\n## {doc_file}\n\n{doc_file.read_text()}"

def answer_support_question(question: str) -> str:
    response = client.messages.create(
        model="claude-haiku-4-5-20251001",  # Haiku for fast, cheap support responses
        max_tokens=512,
        system=[
            {"type": "text", "text": "Answer questions based on our documentation."},
            {
                "type": "text",
                "text": docs_content,
                "cache_control": {"type": "persistent"}  # 1-hour TTL — questions come slowly
            }
        ],
        messages=[{"role": "user", "content": question}]
    )
    return response.content[0].text
```

Combining Caching with the Batch API

For maximum cost reduction on non-urgent work, combine prompt caching with the Batch API:

- Batch API: 50% discount on all requests
- Prompt caching: 90% discount on cached input tokens
- Combined: up to 95% reduction versus standard per-request pricing

```
import anthropic

client = anthropic.Anthropic()
large_context = open("large-context.txt").read()

# Create a batch of 100 requests — all reusing the same cached context
batch = client.messages.batches.create(
    requests=[
        {
            "custom_id": f"task-{i}",
            "params": {
                "model": "claude-sonnet-4-6",
                "max_tokens": 1024,
                "system": [
                    {"type": "text", "text": "You are a code reviewer."},
                    {
                        "type": "text",
                        "text": large_context,
                        "cache_control": {"type": "ephemeral"}
                    }
                ],
                "messages": [
                    {"role": "user", "content": f"Review change #{i} for security issues."}
                ]
            }
        }
        for i in range(100)
    ]
)

print(f"Batch submitted: {batch.id}")
print("Will complete within 24 hours.")
```

What Can and Cannot Be Cached

Can be cached:

- System prompt content
- User message content
- Tool definitions
- Documents and file content included in messages

Cannot be cached:

- Output tokens — you always pay full output rate
- The last message in a conversation (it needs to be complete to cache)
- Images (currently; text-based content only)

Minimum cacheable size: The minimum size for a cache breakpoint is approximately 1,024 tokens. Caching very short prefixes is not supported — the content needs to be substantial enough that the savings justify the infrastructure overhead.

Summary: When to Use Each Approach

Scenario	Approach
Interactive session with a large codebase	Ephemeral caching, automatic mode
Documentation assistant with shared users	Persistent caching (1-hour TTL)
Batch code review across 100+ PRs	Batch API + ephemeral caching
Support bot answering questions slowly	Persistent caching + Haiku model
Real-time chat with conversation history	Ephemeral caching on last user turn
One-off request, no repetition	Skip caching — no benefit

The minimum effective use of caching is a single large document that you query more than once in a 5-minute window. The maximum benefit is a large stable context used across hundreds of requests per hour — there, the savings are dramatic.

Prompt caching is available on all current Claude models: Opus 4.7, Sonnet 4.6, and Haiku 4.5.

Part 5: Advanced Capabilities

Voice Dictation in Claude Code: Speak Instead of Type

Published 2026-05-03 | Source: https://blog.devops-monk.com/2026/05/claude-code-voice-dictation/

Use Claude Code's /voice command for push-to-talk dictation. Perfect for tired hands, complex explanations, and hands-free coding workflows.

You've been coding for three hours straight. Your fingers are tired. Your wrists are starting to complain. You have a complex prompt to type out—explaining the bug, the context, what you've already tried, what you want next.

You're tempted to just write "Fix this" and hit enter. But Claude needs real context to help you effectively.

Here's the problem: **Humans speak about 3x faster than they type.** You can describe a problem in 30 seconds of talking that would take 2 minutes to type properly.

Claude Code's `/voice` command solves this with built-in push-to-talk dictation.

The `/voice` Command

```
/voice
```

How it works:

1. Hold Space
2. Speak naturally
3. Release Space
4. Your words appear in the prompt as you say them

No installation. No configuration. Just push-to-talk.

How Voice Dictation Works Behind the Scenes

Cloud-powered, not on-device — your voice streams to Anthropic's servers, gets transcribed there, returns as text. Nothing runs locally.

Claude.ai login required — only works when signed in with a Claude.ai account.

Not available if Claude Code is wired to:

- Anthropic API keys directly
- Amazon Bedrock
- Google Vertex AI
- Microsoft Foundry

It's free — transcription doesn't consume messages/tokens and won't count toward `/usage` limits. Speak as much as you want.

When Voice Dictation Shines

Complex Problem Descriptions

Instead of typing:

```
"So the payment module has a race condition where if two requests
come in simultaneously for the same user ID, the database write
happens twice instead of being idempotent. I've tried adding a
unique constraint but Postgres is throwing a conflict error.
The weird part is it only happens on high-load days."
```

Just talk it out naturally. Speaking captures nuance that typed text misses—inflection, pauses, emphasis.

Extended Explanations

Architecture decisions, debugging sessions, requirements clarification. Voice naturally handles long-form explanation better than typing.

Tired Hands

After 3+ hours of coding, your wrists hurt. Voice gives your hands a break while staying productive.

Hands-Free Workflows

When your hands are occupied (adjusting code on monitor, reading docs) but your brain is ready to direct Claude.

Real-World Example

Typed prompt (hard to compose, 2 minutes):

```
"We're getting timeout errors in production on the auth service.
The logs show requests taking 15+ seconds. I've checked CPU and
memory—both normal. The database connections look fine. But when
I checked the connection pool settings, they seem low. Should I
increase them? Also, we're doing N+1 queries on login."
```

Spoken prompt (30 seconds):

```
"Our auth service is timing out in production. CPU and memory
are fine. Database connections look normal too. But I'm wondering
if the connection pool size is too small. And I noticed we're
doing N+1 queries on login—is that the culprit?"
```

Both describe the problem, but the second came 4x faster and required zero typing effort.

Requirements

- Claude Code CLI

  (installed and running)

- Claude.ai account
  (signed in)
- Not
  using direct API key, Bedrock, Vertex AI, or Foundry setup
- Microphone
  (any device with audio input)

Best Practices

Do:

- Use for complex explanations that benefit from natural speech
- Combine voice for prompts, then refine with keyboard if needed
- Speak clearly but don't over-articulate
- Use voice for initial context-setting, then switch to typing for precise instructions

Don't:

- Rely on voice for code snippets (accuracy matters there)
- Expect perfect transcription for highly technical jargon
- Use for password or sensitive data (it's still going to Anthropic's servers)

Cost Implications

Zero. Voice transcription is completely free and doesn't:

- Consume your messages quota
- Count toward token limits
- Cost anything beyond your Claude Code subscription

This is one of the rare cases where a powerful feature has no usage cost.

Troubleshooting

Voice not working:

- Verify you're signed in with Claude.ai account (
  /who
  shows login status)
- Check microphone is accessible
- Ensure Claude Code isn't using API keys directly

Transcription accuracy issues:

- Speak more naturally, don’t over-enunciate
- Take pauses between sentences
- For technical terms, spell them out: “P-Q-S, or Priority Queue Structure”

Nothing being recognized:

- Check microphone volume
- Make sure you held Space long enough
- Try a short test: just say “hello”

Comparison: Voice vs Typed Prompts

Scenario	Voice	Typed
Complex explanation	Better	Slower
Tired hands	Essential	Painful
Code snippets	Worse	Better
Initial problem setup	Much faster	Slower
Precise instructions	Typed better	Essential
Brainstorming	Natural	Awkward

Workflow Tip: Hybrid Approach

Many power users combine both:

1. Initial voice prompt
   — Set context, explain the problem naturally
2. Switch to typing
   — Give precise instructions, code snippets
3. Back to voice
   — Ask follow-up questions, explain results

This gets you the speed of voice for explanation + accuracy of typing for details.

Future-Looking: Voice Features

As Claude Code evolves, expect:

- Offline voice recognition (coming soon to supported platforms)
- Voice responses (Claude speaks its output)
- Custom voice profiles for better accuracy
- Multi-language voice support

For now, cloud-powered transcription is the standard.

Next Steps

- Session Management:

  Combine voice prompts with clean session habits
- Status Line Guide:

  Monitor your work while using hands-free voice
- Hooks & Automation:

  Set up automated workflows triggered by voice prompts

Claude Extended and Adaptive Thinking: Making Claude Reason Before It Answers

Published 2026-05-03 | Source: https://blog.devops-monk.com/2026/05/claude-adaptive-thinking-guide/

A practical guide to extended thinking and adaptive thinking in Claude — what they are, when to enable them, how to implement them in Python and TypeScript, and the breaking change in Opus 4.7.

By default, Claude generates its response token by token without any deliberate planning step. For most tasks — answering a question, writing a function, explaining a concept — this is fine. The response comes quickly and it is good.

For some tasks, it is not enough. Complex multi-step reasoning problems, ambiguous architecture decisions, intricate security analyses — these benefit from Claude thinking through the problem before committing to an answer. That is what extended thinking and adaptive thinking provide.

Extended Thinking vs. Adaptive Thinking

These are two distinct mechanisms, and which one you use depends on the model:

Extended Thinking (manual control, Sonnet 4.6 and Haiku 4.5):

- You explicitly enable thinking and set a token budget
- Claude uses up to that many tokens thinking through the problem
- You see the thinking blocks in the response (or choose to hide them)
- Available as

  thinking: {type: "enabled", budget_tokens: N}

Adaptive Thinking (model-controlled, Sonnet 4.6 and Opus 4.7):

- The model decides when and how much to think
- You control the effort level (

  low

  ,

  medium

  ,

  high

  ,

  max

)

- Available as

  thinking: {type: "adaptive"}, effort: "high"

Important: Opus 4.7 only supports adaptive thinking. If you pass `type: "enabled"` with a budget on Opus 4.7, the API returns an error. This is a breaking change from Opus 4.6.

How Thinking Works Under the Hood

```
sequenceDiagram
    participant You as Developer
    participant Claude as Claude API

    Note over You,Claude: Standard response (no thinking)
    You->>Claude: "What's the best database for this use case?"
    Claude-->>You: [answer] — instant, no deliberation

    Note over You,Claude: With extended/adaptive thinking
    You->>Claude: "What's the best database for this use case?" + thinking enabled
    Claude->>Claude: [thinking block — internal reasoning]
    Note right of Claude: Considers options, trade-offs,\nweighs constraints...
    Claude-->>You: [thinking block] + [answer] — deliberated response
```

When thinking is enabled, Claude generates a thinking block before its response. This is real deliberation — Claude working through the problem, considering alternatives, and catching errors in its own reasoning. The final answer is grounded in that reasoning process.

You are billed for thinking tokens. On models that return **summarised thinking** (Claude 4 and newer), you pay for the full internal token count but receive a summary — Anthropic returns a condensed version to prevent misuse of the raw reasoning trace.

Extended Thinking — Python

> ***Deprecation notice:*** `budget_tokens` *is deprecated on Sonnet 4.6 and Opus 4.6 and will be removed in a future release. For new code, prefer adaptive thinking with the* `effort` *parameter. For Opus 4.7, adaptive thinking is the only option — passing* `type: "enabled"` *returns a 400 error.*

```
import anthropic

client = anthropic.Anthropic()

response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=16000,
    thinking={
        "type": "enabled",
        "budget_tokens": 10000  # up to 10k tokens for thinking
    },
    messages=[
        {
            "role": "user",
            "content": """
            We are designing a data pipeline that needs to:
            - Ingest 50,000 events per second from IoT sensors
            - Store raw events for 90 days
            - Support real-time aggregation queries (last 5 minutes, 1 hour, 24 hours)
            - Support historical queries going back 90 days
            - Budget: ~$500/month infrastructure

            Recommend an architecture with specific technologies and explain your reasoning.
            """
        }
    ]
)

# Response contains thinking blocks and text blocks
for block in response.content:
    if block.type == "thinking":
        print("=== Claude's reasoning ===")
        print(block.thinking)
        print()
    elif block.type == "text":
        print("=== Final answer ===")
        print(block.text)
```

Controlling the Budget

The `budget_tokens` parameter sets an upper limit — Claude will use as many tokens as the problem requires, up to the budget. Setting it too low forces Claude to cut reasoning short on hard problems.

Task difficulty	Suggested budget
Simple reasoning, few steps	2,000–5,000
Standard complex problem	8,000–12,000
Deep architectural analysis	15,000–20,000
Maximum depth	Up to 100,000

```
# For a simple trade-off question
thinking={"type": "enabled", "budget_tokens": 3000}

# For a multi-step security audit
thinking={"type": "enabled", "budget_tokens": 15000}
```

Hiding Thinking from the Response

If you want the quality benefit without showing users the raw reasoning:

```
response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=8000,
    thinking={
        "type": "enabled",
        "budget_tokens": 8000,
        "display": "omitted"  # thinking still happens, just not returned
    },
    messages=[...]
)

# Only text blocks in response.content — thinking is omitted but still happened
```

This is useful in production where you want improved answer quality without streaming or storing thinking content.

Adaptive Thinking — Python

```
import anthropic

client = anthropic.Anthropic()

response = client.messages.create(
    model="claude-opus-4-7",       # or claude-sonnet-4-6
    max_tokens=16000,
    thinking={"type": "adaptive"}, # model decides when to think
    effort="high",                 # low | medium | high (default) | max
    messages=[
        {
            "role": "user",
            "content": """
            Review this Terraform module for security issues.
            Focus on IAM policies, S3 bucket configurations, and network ACLs.
            Identify specific CVE patterns if applicable.
            """
        }
    ]
)

for block in response.content:
    if block.type == "thinking":
        print(f"[Reasoning — {len(block.thinking)} chars]")
    elif block.type == "text":
        print(block.text)
```

Effort Levels

Effort	Behaviour	Cost
`low`	Minimal reasoning — fast, cheap. Thinks only when unavoidable.	Lowest
`medium`	Moderate reasoning. Thinks on moderately complex problems.	Moderate
`high`	Default. Reasons on any problem that benefits from it.	Higher
`max`	Maximum reasoning. Applies deep thinking to almost every response.	Highest

Start with `high` (the default). Drop to `medium` or `low` for tasks where speed matters more than depth. Use `max` only when you genuinely need the absolute best answer on a hard problem — the cost scales accordingly.

Adaptive Thinking — TypeScript

```
import Anthropic from "@anthropic-ai/sdk";

const client = new Anthropic();

async function reviewArchitecture(description: string): Promise<string> {
  const response = await client.messages.create({
    model: "claude-opus-4-7",
    max_tokens: 16000,
    thinking: { type: "adaptive" },
    effort: "high",
    messages: [
      {
        role: "user",
        content: `Review this architecture for scalability and reliability issues:\n\n${description}`,
      },
    ],
  });

  // Collect only text blocks for the final response
  return response.content
    .filter((block) => block.type === "text")
    .map((block) => (block as { text: string }).text)
    .join("\n");
}
```

Multi-Turn Conversations with Thinking

When building multi-turn conversations, you must pass thinking blocks back in subsequent turns to preserve the model’s reasoning chain. Dropping them causes errors.

```
import anthropic

client = anthropic.Anthropic()

conversation = []

def chat_with_thinking(user_message: str) -> str:
    conversation.append({"role": "user", "content": user_message})

    response = client.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=8000,
        thinking={"type": "enabled", "budget_tokens": 5000},
        messages=conversation
    )

    # Add the full response (including thinking blocks) back to history
    # This is required — you cannot drop thinking blocks from history
    conversation.append({
        "role": "assistant",
        "content": response.content  # includes both thinking and text blocks
    })

    # Return only the text to the user
    text_blocks = [b.text for b in response.content if b.type == "text"]
    return "\n".join(text_blocks)

# Multi-turn example
answer1 = chat_with_thinking("What database should I use for a high-write IoT workload?")
print(answer1)

answer2 = chat_with_thinking("What if we also need ACID transactions on some of those writes?")
print(answer2)  # Claude has full context including previous reasoning
```

When to Enable Thinking

Thinking adds latency and cost. It is not always the right choice.

Enable thinking for:

- Architecture decisions with multiple valid approaches and real trade-offs
- Security analysis requiring understanding of multi-step attack chains
- Complex debugging where the cause is not obvious from the symptoms
- Mathematical or algorithmic problems with non-trivial solutions
- Any task where you have tried without thinking and found the output lacking

Skip thinking for:

- Simple factual questions (“what does this flag do?”)
- Formatting, renaming, or structural refactoring tasks
- Summarisation of content that is already clear
- Tasks where speed is more important than depth
- High-volume production endpoints where cost is a constraint

A useful heuristic: if a senior engineer would spend more than 5 minutes thinking before answering, enable thinking. If they would answer immediately, skip it.

Interleaved Thinking in Agentic Workflows

Newer Claude 4 models support **interleaved thinking** — Claude reasons not just before the final response, but between tool calls during a multi-step agent run. This is particularly valuable for agentic coding tasks where the model needs to plan, check the result, re-plan, and continue.

```
import anthropic

client = anthropic.Anthropic()

tools = [
    {
        "name": "run_bash",
        "description": "Execute a bash command and return output",
        "input_schema": {
            "type": "object",
            "properties": {
                "command": {"type": "string", "description": "The bash command to run"}
            },
            "required": ["command"]
        }
    },
    {
        "name": "read_file",
        "description": "Read a file from the filesystem",
        "input_schema": {
            "type": "object",
            "properties": {
                "path": {"type": "string"}
            },
            "required": ["path"]
        }
    }
]

response = client.messages.create(
    model="claude-opus-4-7",  # interleaved thinking automatic on Opus 4.7
    max_tokens=32000,
    thinking={"type": "adaptive"},
    effort="high",
    tools=tools,
    messages=[
        {
            "role": "user",
            "content": "Debug why our test suite is taking 12 minutes instead of the expected 3. Find the
 bottleneck."
        }
    ]
)

# Claude will think, call tools, think again based on results,
# call more tools, and reason through the full investigation
for block in response.content:
    if block.type == "thinking":
        print(f"[Claude reasoning: {len(block.thinking)} chars]")
    elif block.type == "tool_use":
        print(f"[Tool call: {block.name}({block.input})]")
    elif block.type == "text":
        print(block.text)
```

With interleaved thinking, Claude does not just think once at the start — it reasons between every step of the tool-calling loop. This is what makes it effective at open-ended investigation tasks.

Migrating from Opus 4.6 to Opus 4.7

If you are using extended thinking with Opus 4.6, this migration is required when moving to Opus 4.7:

```
# Opus 4.6 — extended thinking (manual budget)
response = client.messages.create(
    model="claude-opus-4-6",
    max_tokens=16000,
    thinking={
        "type": "enabled",
        "budget_tokens": 10000
    },
    messages=[...]
)

# Opus 4.7 — adaptive thinking (model-controlled)
response = client.messages.create(
    model="claude-opus-4-7",
    max_tokens=16000,
    thinking={"type": "adaptive"},  # NOT "enabled"
    effort="high",                  # replaces budget_tokens
    messages=[...]
)
```

The `effort` parameter is the practical replacement for `budget_tokens`. If you were using a large budget (15,000+) for complex tasks, use `effort: "max"`. For standard budgets (5,000–10,000), use `effort: "high"` (the default).

Cost Estimation

Thinking tokens are billed as input tokens. Estimate costs before enabling at scale:

```
import anthropic

client = anthropic.Anthropic()

# Check what thinking actually costs on your specific task
response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=8000,
    thinking={"type": "enabled", "budget_tokens": 8000},
    messages=[{"role": "user", "content": your_task}]
)

print(f"Input tokens (non-thinking): {response.usage.input_tokens}")
print(f"Thinking tokens:             {response.usage.thinking_tokens if hasattr(response.usage,
'thinking_tokens') else 'n/a'}")
print(f"Output tokens:               {response.usage.output_tokens}")
```

As a rough guide, a 5,000-token thinking budget on a complex task with Sonnet 4.6 costs about $0.015 in thinking tokens alone (5,000 × $3/MTok). At that rate, it is cost-effective for decisions that matter but negligible for tasks you run once. For high-volume production workloads, skip thinking unless the task genuinely requires it.

Quick Reference

Scenario	Model	Thinking config
Architecture decision	Opus 4.7 or Sonnet 4.6	Adaptive, effort: high

Scenario	Model	Thinking config
Security audit	Opus 4.7	Adaptive, effort: max
Complex debugging	Sonnet 4.6	Adaptive, effort: high
Simple code review	Sonnet 4.6	None
Feature implementation	Sonnet 4.6	None (or low)
Agentic multi-step task	Opus 4.7	Adaptive, effort: high
High-volume production	Haiku 4.5	Adaptive, effort: low

The goal is not to enable thinking everywhere — it is to apply it where it makes a meaningful difference. On tasks that are hard, ambiguous, or high-stakes, it substantially improves output quality. On tasks that are straightforward, it is wasted cost and latency.

Claude Computer Use for DevOps: When to Use It and When to Use an API

Published 2026-04-26 | Source:
https://blog.devops-monk.com/2026/04/claude-computer-use-devops-workflows/

Claude's computer use capability lets it interact with any UI on your screen. Here is a practical guide to where it fits in DevOps workflows versus structured API integrations.

Claude's computer use capability — the ability to see your screen and interact with applications via mouse clicks, keyboard input, and scrolling — is one of the most discussed features in the AI space. It is also one of the most misapplied.

The correct mental model is not "Claude can now automate everything on my screen." It is "Claude now has a flexible fallback layer for tasks that do not have a structured API integration."

How Computer Use Actually Works

```
sequenceDiagram
    participant You as Developer
    participant Claude as Claude
    participant Screen as Your Screen

    You->>Claude: Check the Vercel deploy and send me the URL if green
    Claude->>Screen: Take screenshot
    Screen-->>Claude: Current screen state
    Claude->>Screen: Navigate to Vercel dashboard
    Screen-->>Claude: Dashboard screenshot
    Claude->>Screen: Read deploy status
    Claude-->>You: Deploy is green, URL is https://app.example.com
```

Claude receives screenshots of your screen and sends back coordinates to click, text to type, or scroll instructions. It operates like a remote control for your desktop.

This is fundamentally different from an API integration: it is slower, less reliable when UI layouts change, and has inherent overhead from screenshot processing. But it works on *anything with a UI* — including tools that have no API, legacy internal dashboards, and one-off tasks that do not justify building a dedicated integration.

The Two-Layer Tool Model

The right architecture for AI-powered DevOps workflows uses two layers:

```
flowchart TD
    Task[Automation task] --> Q1{Does a reliable API\nor MCP server exist?}
    Q1 -->|Yes| API[Use API or MCP\nFast, reliable, structured]
    Q1 -->|No| Q2{Is this task\nrepeated daily?}
    Q2 -->|Yes| Build[Build an API integration\nWorth the investment]
    Q2 -->|No - one-off or occasional| CU[Use computer use\nFlexible fallback]
```

Layer 1 — Structured integrations (APIs and MCP servers): GitHub, PagerDuty, AWS CloudWatch, Linear, Sentry. These are fast, deterministic, and the right choice for anything you do every day.

Layer 2 — Computer use (UI fallback): The legacy internal tool that has no API. The vendor dashboard you use twice a month. The one-off task that would take longer to write a script for than to just do with computer use.

Practical DevOps Use Cases

Verify a deployment across multiple dashboards

You want to confirm a deploy is healthy: the CI job passed, the deployment is running, the error rate did not spike. Three different tools, none of which share data directly.

```
After this deploy completes, check:
1. GitHub Actions — confirm the deploy workflow shows success
2. DataDog — confirm error rate is below 0.1% for the payments service
3. Our internal status page — confirm all services show green

Message me on Slack with a summary of what you find.
```

Without computer use, this is three manual checks across three tabs. With computer use, it is a single delegated task.

Interact with a legacy internal tool

Many engineering organisations have internal tools built years ago with no REST API. Computer use makes Claude an interface to those tools without requiring a rewrite or a custom integration.

```
Go to the internal deployment portal at deploy.internal.company.com,
find the batch job that failed at 14:30 UTC, and restart it.
```

Screenshot-based design review

When a designer shares a Figma link and you want Claude's analysis without building a Figma MCP integration:

```
Open this Figma URL, screenshot the checkout flow components,
and tell me if the spacing is consistent with our design system guidelines.
```

The Dispatch Pattern

The most productive way to use computer use is asynchronously — you send the task from your phone or another device while Claude works on your desktop in the background.

Anthropic launched **Dispatch** alongside computer use — a companion mobile app that lets you assign tasks to Claude from your phone. You open Dispatch, describe the task, and Claude executes it on your desktop while you step away. You receive a notification when it completes. This is the canonical workflow for the async delegation pattern: send from phone → Claude works on desktop → notification on phone.

```
sequenceDiagram
    participant Phone as Your Phone
    participant Claude as Claude (on desktop)
    participant Desktop as Desktop Tools

    Phone->>Claude: Check if the staging deployment is healthy and Slack me
    Note over Claude,Desktop: You go into a meeting
    Claude->>Desktop: Opens CI dashboard
    Claude->>Desktop: Opens DataDog
    Claude->>Desktop: Checks Sentry for new errors
    Claude->>Desktop: Posts summary to Slack
    Note over Phone: You receive Slack message during meeting
```

The task is done before you return. This works particularly well for:

- Pre-meeting checks you would otherwise rush through
- Deployment verification that does not need your full attention
- Status gathering across multiple tools before a standup

When Not to Use Computer Use

For daily repeated tasks — If you are checking the same three dashboards every morning, build a proper integration or use MCP. Computer use has enough overhead and fragility that repeating it daily will eventually break.

For write operations on critical systems — Computer use makes mistakes on complex multi-step workflows more often than API-based approaches. Do not use it to apply Terraform or run database migrations.

When a UI changes frequently — Computer use instructions reference UI elements by position and appearance. If the vendor redesigns their dashboard, the instructions break. API integrations are immune to visual redesigns.

For anything requiring speed — API calls take milliseconds. Computer use takes seconds per step. For latency-sensitive automation, API is the only option.

Getting Started

Computer use in Claude Code is available in the desktop app. To enable it for a task:

```
Use computer use to: [your task description]
```

Be specific about what success looks like and what tools to use. Vague instructions produce slow, wandering behaviour. Clear instructions with specific tool names and expected outcomes produce fast, accurate results.

Start with low-risk, read-only tasks: check a status, read a dashboard, screenshot a UI. Build confidence with those before delegating write operations.

Claude Design: A Complementary Visual Tool

Anthropic Labs launched **Claude Design** alongside the broader computer use improvements — a separate product that lets you collaborate with Claude to create visual outputs: designs,

prototypes, slides, and one-pagers. Rather than Claude navigating your screen to interact with Figma, Claude Design gives Claude a native environment for producing visual artifacts.

The distinction is worth understanding:

Tool	What it does
Computer use	Claude interacts with existing UIs on your screen
Claude Design	Claude creates new visual artifacts in a purpose-built canvas

For DevOps use cases, Claude Design is relevant when you need to produce architecture diagrams, runbook one-pagers, or slide decks summarising infrastructure changes — faster than asking Claude to navigate presentation software via computer use.

Summary

Computer use is a powerful capability best understood as a flexible fallback rather than a primary automation strategy. The decision framework is straightforward:

- Use APIs and MCP servers for structured, daily, reliable workflows
- Use computer use for UI-only tools, one-off tasks, and asynchronous delegation
- Use Claude Design for creating new visual artifacts rather than interacting with existing visual tools
- Do not use computer use for anything that needs to run reliably at scale

The teams getting the most value from it are those who have already set up their core integrations properly and are using computer use to fill the gaps — not those trying to replace structured automation with screenshot-based interactions.

Claude Code as a Security Scanner: Beyond Pattern Matching

Published 2026-04-26 | Source: https://blog.devops-monk.com/2026/04/claude-code-security-scanner/

How to use Claude Code for security audits that traditional linters miss — including business logic flaws, IDOR vulnerabilities, and multi-step workflow exploits.

Tools like ESLint, Semgrep, and Bandit catch what they are programmed to find: known patterns, common injection strings, deprecated API calls. They are fast, reliable, and deterministic. They are also blind to anything that requires understanding *what your code is supposed to do*.

Claude Code operates differently. It reads code the way a human security researcher would — tracing data flows across files, understanding business logic, and reasoning about what could go wrong given the specific context of your application. This makes it effective at a class of vulnerabilities that pattern-matching tools routinely miss.

What Traditional Scanners Cannot Find

```
flowchart TD
    subgraph Traditional[Traditional Static Analysis]
        P1[Known injection patterns]
        P2[Deprecated API calls]
        P3[Common misconfigurations]
        P4[Syntax-level issues]
    end
    subgraph Claude[Claude Code Analysis]
        C1[Business logic flaws]
        C2[IDOR vulnerabilities]
        C3[Multi-step workflow exploits]
        C4[Cross-file data flow issues]
        C5[Auth state inconsistencies]
    end
    Traditional -->|misses| Claude
```

Business logic flaws are the hardest category to catch with automation. Consider a scenario where a discount code is validated at checkout but stored in the session — and the session persists across authentication state changes. A scanner looking for SQL injection will not find this. Claude, reading the checkout flow end-to-end, can trace the discount code from validation through storage through redemption and flag that the code survives logout.

IDOR vulnerabilities (Insecure Direct Object References) require understanding the relationship between your URL parameters and your data model. A route that accepts `/api/orders/:id` without checking that the authenticated user owns that order is exploitable — but only if the scanner understands what "owns" means in your domain.

Multi-step workflow exploits — broken password resets, email verification bypasses, payment state mismatches — require following a complete user journey across multiple endpoints, often across multiple files.

How to Run a Security Audit

Option 1: Direct prompt

For a specific file or component:

```
Audit src/api/checkout.ts for security vulnerabilities.

Trace data flows end to end — follow the cart data from the request through
validation, pricing, discount application, and payment processing.

Look specifically for:
- Auth and authorization gaps (can an unauthenticated user reach any step?)
- State that persists across user sessions incorrectly
- Places where user-supplied values touch database queries or file paths
- Business logic that could be exploited (price manipulation, discount reuse)

For each finding: severity (critical/high/medium/low), exact location,
and why it is exploitable in this specific codebase.
```

Option 2: Custom slash command

Create `.claude/commands/security-audit.md` in your project:

```
Review $ARGUMENTS for security vulnerabilities.

Trace data flows end-to-end across related files — not just the file provided.

Check for:
- Injection risks: SQL, command injection, path traversal
- Authentication and authorisation gaps
- Sensitive data exposure in logs or responses
- Business logic flaws specific to this application's domain
- Insecure Direct Object References
- Multi-step workflow vulnerabilities

Rate each finding: critical / high / medium / low
Include: file path, line number, description, and why it is exploitable.
Do not make any code changes — report only.
```

Then run it:

```
/project:security-audit src/api/payments/
```

The Audit Workflow

```
sequenceDiagram
    participant Dev as Developer
    participant Claude as Claude Code
    participant Files as Codebase

    Dev->>Claude: security-audit src/api/checkout.ts
    Claude->>Files: Read checkout.ts
    Claude->>Files: Trace imports - read auth middleware
    Claude->>Files: Trace imports - read cart service
    Claude->>Files: Read payment processor integration
    Claude-->>Dev: 3 findings: 1 high, 2 medium
    Dev->>Dev: Review each finding
    Dev->>Claude: Fix the high severity finding in checkout.ts
    Claude->>Files: Edit checkout.ts
    Claude-->>Dev: Fix applied - here is what changed and why
    Dev->>Dev: Review diff, approve change
```

The human-in-the-loop is deliberate. Claude reports findings and explains them but does not automatically patch them. Security decisions should stay human-controlled — an automated fix to a security issue can introduce a different vulnerability if the surrounding context is not fully understood.

What Claude Catches vs What It Misses

Category	Claude	Semgrep / ESLint
SQL injection (parameterised)	Good	Good
Business logic flaws	Excellent	Poor
IDOR	Good	Poor
Multi-step exploits	Good	Very poor
Known CVE patterns	Limited	Excellent
Supply chain issues	Poor	Good (with plugins)
Performance	No	No

The right approach is both. Run your standard linters in CI for fast, deterministic coverage of known patterns. Use Claude Code for deeper audits before major releases, after significant changes to auth or payment flows, or when onboarding to an unfamiliar codebase.

Practical Tips

Be specific about what to audit. “Audit the whole repo” produces shallow results across everything. “Audit the checkout flow” produces deep results where it matters.

Provide business context. The more Claude understands about what your application is *supposed* to do, the better it can identify where it does something *unintended*. Include a brief description of the feature you are auditing.

Iterate on findings. Ask Claude to explain its reasoning. “Why is this IDOR exploitable specifically?” often reveals whether it is a real finding or a false positive, faster than manually tracing the code yourself.

Run audits on a clean context. Start a fresh session for security audits. Stale context from unrelated work can reduce the quality of analysis.

Use Opus 4.7 for high-stakes audits. For payment flows, authentication systems, and anything customer-data-adjacent, switch to Opus 4.7 (`/model claude-opus-4-7`) and enable adaptive thinking at max effort. The improved reasoning depth is particularly valuable for tracing multi-step attack chains that span several files. For routine PR security reviews, Sonnet 4.6 is sufficient and much cheaper.

Integration with CI

For teams wanting automated security feedback on pull requests, Claude can run as part of a GitHub Actions workflow:

```
name: Security Review
on: [pull_request]

jobs:
  security:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Claude Security Audit
        run: |
          changed=$(git diff --name-only origin/main...HEAD | grep -E '\.(ts|js|py|go)$')
          if [ -n "$changed" ]; then
            echo "$changed" | xargs -I{} claude -p \
              "Security audit of {}: check for auth gaps, injection risks, and business logic flaws.
Report findings with severity." \
              --output-format json
          fi
        env:
          ANTHROPIC_API_KEY: ${{ secrets.ANTHROPIC_API_KEY }}
```

This runs a targeted audit on only the files changed in the PR — keeping it fast and focused.

Claude Code Security: The Managed Product

Everything described in this post is a DIY approach — you bring the API key, write the prompts, and wire up the CI workflow. Anthropic also ships a managed version.

Claude Code Security launched in public beta on April 30, 2026, available to Enterprise customers. No API integration, no custom agent, no prompt engineering required. Connect your repository and it handles the rest.

In a February 2026 research preview, it found over **500 previously undetected vulnerabilities** in production open-source codebases — bugs that had survived years of expert review and conventional scanning.

How it differs from the manual approach: The managed product provides a purpose-built UI, continuously scans your entire repository, and integrates directly with your PR workflow — findings appear as PR comments with no configuration. The manual Claude Code approach gives you more control: you decide what to audit, how to frame the question, and what context to include. That matters for custom deep dives, component-specific audits, and teams not on Enterprise.

Nothing is applied without human approval. Findings are surfaced for your team to review and act on.

When to use which:

- Use

  claude.ai/security

  for Enterprise teams wanting continuous automated scanning with zero setup
- Use the

  manual Claude Code approach

  for custom audits, targeted investigations, or teams not on Enterprise

Part 6: Building AI Systems

Claude Managed Agents: Deploy AI Agents Without Managing Infrastructure

Published 2026-05-03 | Source: https://blog.devops-monk.com/2026/05/claude-managed-agents-guide/

A practical guide to Claude Managed Agents — the fully hosted REST API that handles agent infrastructure, sandboxed execution, long-running sessions, and memory. Ideal for DevOps teams who want agents without the ops overhead.

Building an AI agent that runs autonomously — browses the web, executes code, reads and writes files, persists memory across sessions — requires infrastructure. You need a sandbox, a process that can run for hours without your web server timing out, and a way to resume from where you left off after a network hiccup.

Claude Managed Agents, launched in public beta in April 2026, offloads all of that to Anthropic. You send a task to a REST endpoint, and Anthropic runs the agent loop in a secure sandboxed container. The agent can run for hours, survives disconnections, and maintains memory across sessions.

This article covers when to use it, how to call it, and what it costs.

Managed Agents vs. the Agent SDK: What to Choose

Before diving into the API, here is the decision framework:

	Managed Agents	Agent SDK
Runtime	Anthropic-hosted	You host
Agent loop	Anthropic manages	You implement
Infrastructure	Fully managed	You manage
Sandbox	Built-in, secure	You set up
Long-running sessions	Natively supported	You handle timeouts
Memory	Built-in (public beta)	You implement
Customisation	Less control	Full control
Time to first agent	Minutes	Days to weeks
Runtime pricing	$0.08/session-hour	Your infra cost

Use Managed Agents when:

- You want to ship an agent quickly without infrastructure work
- Tasks run for hours and you need reliable resumption
- You do not need custom tool integrations or unusual control flow
- Your team does not have capacity to maintain agent infrastructure

Use the Agent SDK when:

- You need custom tool integrations or non-standard architecture
- Compliance requires your infrastructure (not Anthropic's)
- You have specific performance requirements the managed service cannot meet
- You are building a product where the agent runtime is a core differentiator

---

How It Works

```
sequenceDiagram
    participant App as Your Application
    participant API as Managed Agents API
    participant Container as Secure Sandbox

    App->>API: POST /agents/sessions {task, tools, model}
    API->>Container: Provision sandbox container
    Container->>Container: Agent loop begins
    Note over Container: Agent uses web search, code execution,\nfile management autonomously
    App->>API: GET /agents/sessions/{id}/stream (SSE)
    Container-->>App: Stream: thinking steps, tool calls, progress
    Container->>Container: Task completes
    Container-->>App: Stream: final result + session summary
```

Every agent run gets its own isolated container. The container has access to built-in tools (web search, code execution, file management) and runs with the model you specify. The session persists even if your connection drops — you can reconnect and resume streaming from where you left off.

Creating Your First Agent Session

All Managed Agents requests require the beta header: `anthropic-beta: managed-agents-2026-04-01`

Python

```
import anthropic
import json

client = anthropic.Anthropic()

# Create an agent session
session = client.beta.managed_agents.sessions.create(
    model="claude-sonnet-4-6",
    max_tokens=8192,
    tools=[
        {"type": "web_search"},
        {"type": "code_execution"}
    ],
    messages=[
        {
            "role": "user",
            "content": """
            Research the top 5 Kubernetes security vulnerabilities reported in 2026,
            write a concise summary for each, and produce a markdown report
            with a risk rating and recommended mitigations.
            Save the final report as 'k8s-security-2026.md'.
            """
        }
    ],
    betas=["managed-agents-2026-04-01"]
)

print(f"Session started: {session.id}")
print(f"Status: {session.status}")
```

TypeScript

```
import Anthropic from "@anthropic-ai/sdk";

const client = new Anthropic();

async function runResearchAgent(task: string): Promise<string> {
  const session = await client.beta.managedAgents.sessions.create({
    model: "claude-sonnet-4-6",
    max_tokens: 8192,
    tools: [
      { type: "web_search" },
      { type: "code_execution" },
      { type: "file_management" }
    ],
    messages: [{ role: "user", content: task }],
    betas: ["managed-agents-2026-04-01"]
  });

  return session.id;
}
```

Streaming Progress with Server-Sent Events

Agent tasks can take minutes or hours. Use Server-Sent Events to stream progress as it happens:

```
import anthropic

client = anthropic.Anthropic()

# Create a session
session = client.beta.managed_agents.sessions.create(
    model="claude-opus-4-7",
    max_tokens=16384,
    tools=[
        {"type": "web_search"},
        {"type": "code_execution"}
    ],
    messages=[
        {
            "role": "user",
            "content": "Analyse our GitHub Actions CI logs from the last 7 days and identify the top 3
most common failure patterns."
        }
    ],
    betas=["managed-agents-2026-04-01"]
)

# Stream the agent's progress
with client.beta.managed_agents.sessions.stream(session.id) as stream:
    for event in stream:
        if event.type == "content_block_delta":
            # Agent is writing a response
            print(event.delta.text, end="", flush=True)
        elif event.type == "tool_use":
            # Agent is using a tool
            print(f"\n[Tool: {event.name}] {json.dumps(event.input)[:100]}...")
        elif event.type == "message_stop":
            print("\n\nAgent completed.")
            break
```

Reconnecting to a Running Session

If your connection drops — or you want to check on a long-running session later — reconnect by session ID:

```
import anthropic

client = anthropic.Anthropic()

session_id = "session_01ABC..."  # saved from when you created the session

# Check current status
session = client.beta.managed_agents.sessions.retrieve(
    session_id,
    betas=["managed-agents-2026-04-01"]
)

print(f"Status: {session.status}")  # running, completed, failed

# If still running, reconnect to the stream
if session.status == "running":
    with client.beta.managed_agents.sessions.stream(session_id) as stream:
        for event in stream:
            print(event)
```

Memory Across Sessions

As of April 2026, Managed Agents support persistent memory. Claude remembers facts from previous sessions and uses them automatically in new ones.

```
import anthropic

client = anthropic.Anthropic()

# First session — Claude learns your environment
session1 = client.beta.managed_agents.sessions.create(
    model="claude-sonnet-4-6",
    max_tokens=4096,
    tools=[{"type": "web_search"}, {"type": "code_execution"}],
    memory={"enabled": True, "scope": "user"},  # persist memory for this user
    messages=[
        {
            "role": "user",
            "content": "Our production cluster runs Kubernetes 1.29 on AWS EKS in eu-west-1. "
                       "Our staging cluster is in us-east-1. Remember this for future sessions."
        }
    ],
    betas=["managed-agents-2026-04-01"]
)

# Later — a separate session, Claude remembers
session2 = client.beta.managed_agents.sessions.create(
    model="claude-sonnet-4-6",
    max_tokens=4096,
    tools=[{"type": "web_search"}],
    memory={"enabled": True, "scope": "user"},
    messages=[
        {
            "role": "user",
            "content": "Is there a known CVE affecting our Kubernetes version?"
        }
        # Claude knows from memory that you run K8s 1.29
    ],
    betas=["managed-agents-2026-04-01"]
)
```

Scheduling Recurring Agent Runs

Managed Agents supports **routines** — cron-scheduled agent tasks that run automatically without your application triggering them:

```
import anthropic

client = anthropic.Anthropic()

# Run a security scan every Monday at 08:00 UTC
routine = client.beta.managed_agents.routines.create(
    name="weekly-security-scan",
    schedule="0 8 * * 1",  # cron expression
    model="claude-sonnet-4-6",
    max_tokens=8192,
    tools=[
        {"type": "web_search"},
        {"type": "code_execution"}
    ],
    messages=[
        {
            "role": "user",
            "content": """
            Scan our GitHub repository for:
            1. Dependencies with known CVEs published this week
            2. Any new Dependabot alerts
            3. Failed security checks in recent CI runs
            Produce a markdown summary and post it to our Slack webhook.
            """
        }
    ],
    webhook_url="https://your-app.com/webhooks/agent-results",
    betas=["managed-agents-2026-04-01"]
)

print(f"Routine created: {routine.id}")
print(f"Next run: {routine.next_run_at}")
```

The webhook receives the completed session result when the task finishes.

DevOps Use Cases

Automated Incident Triage

```
import anthropic
import os

client = anthropic.Anthropic()

def triage_incident(alert_body: str) -> str:
    """
    Given a PagerDuty alert body, spin up an agent to investigate
    and return a preliminary diagnosis.
    """
    session = client.beta.managed_agents.sessions.create(
        model="claude-opus-4-7",
        max_tokens=16384,
        tools=[
            {"type": "web_search"},
            {"type": "code_execution"}
        ],
        system="You are an on-call SRE. Investigate the incident methodically. "
               "Check logs, metrics, and recent deployments. "
               "Produce a diagnosis and recommended immediate actions.",
        messages=[
            {"role": "user", "content": f"Incoming incident alert:\n\n{alert_body}"}
        ],
        betas=["managed-agents-2026-04-01"]
    )

    # Collect the full result
    result_text = ""
    with client.beta.managed_agents.sessions.stream(session.id) as stream:
        for event in stream:
            if event.type == "content_block_delta":
                result_text += event.delta.text

    return result_text


# Called by your PagerDuty webhook handler
incident_alert = """
CRITICAL: High error rate on payment-service
Time: 2026-05-03 14:32 UTC
Error rate: 12.4% (normal: 0.1%)
Affected endpoints: /api/v2/payments/create, /api/v2/payments/confirm
Recent deploy: payment-service v2.3.1 at 14:15 UTC
"""

diagnosis = triage_incident(incident_alert)
print(diagnosis)
```

Weekly Dependency Audit

```
import anthropic

client = anthropic.Anthropic()

# Run once; schedule as a routine for weekly automation
session = client.beta.managed_agents.sessions.create(
    model="claude-sonnet-4-6",
    max_tokens=8192,
    tools=[
        {"type": "web_search"},
        {"type": "code_execution"},
        {"type": "file_management"}
    ],
    messages=[
        {
            "role": "user",
            "content": """
            Audit our Node.js dependencies:
            1. Read package.json from the repo root
            2. For each direct dependency, check the npm registry for:
               - Current latest version vs what we pin
               - Known security advisories (npm audit equivalent)
            3. Produce a prioritised list: security issues first, then major version gaps
            4. Write the report to dependency-audit.md
            """
        }
    ],
    betas=["managed-agents-2026-04-01"]
)

print(f"Audit running: {session.id}")
```

Infrastructure Documentation Generator

```
import anthropic

client = anthropic.Anthropic()

session = client.beta.managed_agents.sessions.create(
    model="claude-opus-4-7",
    max_tokens=16384,
    tools=[
        {"type": "code_execution"},
        {"type": "file_management"}
    ],
    messages=[
        {
            "role": "user",
            "content": """
            Generate architecture documentation for our infrastructure:
            1. Read all Terraform files in infra/
            2. Read all Kubernetes manifests in k8s/
            3. Read all GitHub Actions workflows in .github/workflows/
            4. Produce a comprehensive architecture document covering:
               - AWS resources and their purposes
               - Service dependencies and data flows
               - CI/CD pipeline stages
               - Network boundaries and security groups
            5. Save as docs/architecture.md with Mermaid diagrams
            """
        }
    ],
    betas=["managed-agents-2026-04-01"]
)
```

Pricing

Managed Agents uses standard Claude API token pricing (same rates as direct API calls) plus a runtime charge for the sandboxed container:

Component	Price
Input/output tokens	Same as Claude API
Container runtime	**$0.08 per session-hour**
Web search	$10 per 1,000 searches
Code execution (with search)	Free

For a task that runs for 20 minutes and uses Sonnet 4.6 with 100k tokens total:

- Tokens: ~$0.45 (100k input at $3/MTok + some output)
- Container: ~$0.027 (20 min at $0.08/hr)
- Total: roughly $0.48

Compare that to the engineering time to build and maintain the equivalent infrastructure.

New in May 2026: Dreaming, Outcomes, and Multiagent Orchestration

On May 6, 2026, Anthropic shipped three significant additions to Managed Agents. All three are available now.

> *Harvey reported a 6x jump in task completion rates after enabling multiagent orchestration. — Anthropic, May 6, 2026*

Dreaming (Research Preview)

Dreaming is a scheduled background process that reviews your agent's past sessions and memory stores, extracts patterns, and curates memories so your agents improve over time.

Standard memory lets agents recall specific facts from earlier sessions. Dreaming goes further: it surfaces patterns that a single session cannot see — recurring mistakes, workflows the agent converges on across many runs, preferences shared across a team. The result is an agent that gets measurably better at your specific workflows the more you use it.

You control how much autonomy dreaming has via two modes:

- Auto mode

  (default) — dreaming updates memory automatically based on session patterns

- Review mode

  — dreaming surfaces proposed memory changes for human approval before they land. This is the right choice for regulated industries or teams that want oversight of what the agent "learns".

```
memory={
    "enabled": True,
    "scope": "user",
    "dreaming": {
        "enabled": True,
        "mode": "review"  # "auto" (default) or "review" (human approves changes)
    }
}
```

To enable dreaming on a session, add the `dreaming` configuration:

```
session = client.beta.managed_agents.sessions.create(
    model="claude-sonnet-4-6",
    max_tokens=8192,
    tools=[{"type": "web_search"}, {"type": "code_execution"}],
    memory={"enabled": True, "scope": "user", "dreaming": {"enabled": True}},
    messages=[{"role": "user", "content": your_task}],
    betas=["managed-agents-2026-04-01"]
)
```

Outcomes (Public Beta)

Outcomes let you define what success looks like, and the agent works toward your criteria rather than just completing a task description.

You write a rubric describing success. A separate grader evaluates the agent's output against your criteria in its own context window — isolated from the agent's reasoning so it cannot be influenced by how the agent justified its own choices.

```
session = client.beta.managed_agents.sessions.create(
    model="claude-opus-4-7",
    max_tokens=16384,
    tools=[{"type": "web_search"}, {"type": "code_execution"}],
    outcomes={
        "rubric": """
        The dependency audit report is successful if:
        1. All direct dependencies are listed with their current and pinned versions
        2. Every CVE found has a severity rating (critical/high/medium/low)
        3. The top 3 most urgent issues are clearly identified with fix instructions
        4. The report is actionable by a developer with no prior context
        """,
        "grader_model": "claude-sonnet-4-6"
    },
    messages=[
        {"role": "user", "content": "Audit our Node.js dependencies for security issues."}
    ],
    betas=["managed-agents-2026-04-01"]
)
```

The grader result is returned alongside the session output, so you can programmatically gate on quality — retry if the rubric score is below threshold, or escalate to a human review queue.

Multiagent Orchestration (Public Beta)

Multiagent orchestration lets a lead agent break a complex job into pieces and delegate each to a specialist agent with its own model, prompt, and tools. Specialists work in parallel on a shared filesystem and contribute to the lead agent's overall context.

Unlike spawning subagents from Claude Code CLI (which share a session), Managed Agents orchestration runs each specialist in its own fully isolated container. Specialists can check back with the lead agent mid-workflow because events are persistent and every agent remembers

what it has done.

```
session = client.beta.managed_agents.sessions.create(
    model="claude-opus-4-7",
    max_tokens=32768,
    tools=[
        {"type": "web_search"},
        {"type": "code_execution"},
        {"type": "file_management"}
    ],
    orchestration={
        "enabled": True,
        "specialists": [
            {
                "name": "security-reviewer",
                "model": "claude-opus-4-7",
                "prompt": "You are a security specialist. Review code for vulnerabilities only.",
                "tools": [{"type": "code_execution"}]
            },
            {
                "name": "dependency-checker",
                "model": "claude-sonnet-4-6",
                "prompt": "You are a dependency specialist. Check npm audit and CVE databases.",
                "tools": [{"type": "web_search"}]
            },
            {
                "name": "report-writer",
                "model": "claude-sonnet-4-6",
                "prompt": "You write clear technical reports from findings provided to you.",
                "tools": [{"type": "file_management"}]
            }
        ]
    },
    messages=[
        {
            "role": "user",
            "content": """
            Run a comprehensive security audit:
            1. Review source code for vulnerabilities
            2. Check all dependencies for known CVEs
            3. Compile findings into a prioritised report
            Do these in parallel, then combine results.
            """
        }
    ],
    betas=["managed-agents-2026-04-01"]
)
```

The lead agent plans the work, delegates to specialists, and integrates results — you get parallelism without managing orchestration logic yourself.

Limits and Constraints (Beta)

Managed Agents is in public beta as of May 2026. Current constraints:

- Maximum session duration: 4 hours per session
- Built-in tools: web search, code execution, file management (custom MCP tools: roadmap)
- Memory scope: user-level or session-level (org-level: roadmap)
- Streaming: SSE only (WebSockets: roadmap)
- Concurrency: rate limits apply per workspace

These will expand as the beta matures. The beta header (`managed-agents-2026-04-01`) is required on all requests — when the feature becomes generally available, the header will no longer be required.

The ant CLI

Alongside Managed Agents, Anthropic launched the `ant` CLI — a command-line client for interacting with the Claude API, including managed agents, without writing code:

```
# Install
npm install -g @anthropic/ant

# Run a one-off agent task
ant run --model claude-sonnet-4-6 --tools web_search,code_execution \
  "Research Kubernetes 1.31 release notes and summarise breaking changes"

# List running sessions
ant sessions list

# Stream output from a running session
ant sessions stream session_01ABC...

# Create a routine
ant routines create --schedule "0 9 * * 1" --name "weekly-audit" \
  "Audit npm dependencies for security issues and write a report"
```

The `ant` CLI is useful for one-off agent tasks and for testing prompts before wiring them into an application.

Managed Agents is the fastest path from “I want an autonomous agent” to “I have a running autonomous agent.” The infrastructure decisions — sandboxing, session persistence, memory — are handled. You focus on the task.

For teams already running Claude Code or using the Claude API, adding Managed Agents for long-running autonomous tasks is a natural extension of the same toolchain.

Building a Personal AI Assistant with Claude Agent SDK and Bun

Published 2026-04-26 | Source:
https://blog.devops-monk.com/2026/04/building-personal-ai-assistant-claude/

How to build an autonomous AI assistant that researches, writes, and delivers weekly briefings — using Bun, the Claude Agent SDK, and a cron schedule instead of a chatbot interface.

Most AI assistants are chatbots. You ask, they answer, the interaction ends. The interesting shift happening right now is treating AI as an *autonomous worker* — something that runs on a schedule, produces real artifacts, and delivers results without you being in the loop.

This post walks through building that kind of assistant: a background agent that runs weekly, researches a set of topics relevant to your work, and delivers a structured briefing via Telegram or email. No chatbot UI. No manual prompting. Just a scheduled process that does real work.

The Architecture

```
flowchart TD
    Cron[Cron schedule\nSunday 8am] --> Script[index.ts\nBun runtime]
    Script --> Memory[Read last week's report\nmemory/weekly.md]
    Script --> Agent[Claude Agent SDK\nquery loop]
    Agent --> Tools{Available tools}
    Tools --> Web[Web search]
    Tools --> Files[Read and write files]
    Tools --> API[API calls]
    Agent --> Report[Write report to\nmemory/weekly.md]
    Report --> Notify[Send via Telegram\nor email]
```

Three design principles make this work:

1. Schedule-driven, not prompt-driven
   — runs autonomously via cron, not when you ask
2. File-based memory
   — reads last week's output before generating this week's, avoiding repetition
3. Real artifacts
   — produces a markdown report file, not just a chat response

Why Bun

Bun starts faster than Node.js — important for scheduled processes that run and exit rather than staying alive as a server. It also supports TypeScript natively without a compilation step, making the code simpler.

For a process that runs once a week, the performance difference matters less than the development experience: one file, no build step, native TypeScript.

Setting Up

```
curl -fsSL https://bun.sh/install | bash
mkdir my-assistant && cd my-assistant
bun init -y
bun add @anthropic-ai/claude-code
```

Create the project structure:

```
my-assistant/
├── index.ts              ← main entry point
├── memory/
│   └── weekly.md         ← previous report (auto-managed)
├── prompts/
│   └── weekly-brief.md   ← prompt template
└── .env                  ← ANTHROPIC_API_KEY, TELEGRAM_TOKEN
```

The Agent Loop

The Claude Agent SDK handles tool orchestration automatically. You provide tools and a prompt; the SDK manages the reasoning loop until the task is complete.

```
import { query } from "@anthropic-ai/claude-code";
import { readFile, writeFile } from "fs/promises";

async function runWeeklyBriefing() {
  // Load previous report to avoid repetition
  const lastReport = await readFile("memory/weekly.md", "utf-8")
    .catch(() => "No previous report.");

  // Load the prompt template
  const template = await readFile("prompts/weekly-brief.md", "utf-8");
  const today = new Date().toISOString().split("T")[0];
  const prompt = template
    .replace("{{DATE}}", today)
    .replace("{{LAST_REPORT}}", lastReport);

  let finalReport = "";

  // Run the agent
  for await (const message of query({
    prompt,
    options: {
      maxTurns: 20,
    },
  })) {
    if (message.type === "result") {
      finalReport = message.result;
    }
  }

  // Persist this week's report
  await writeFile("memory/weekly.md", finalReport);
  return finalReport;
}
```

Agent SDK V2: Simpler Multi-Turn Conversations

The V2 Claude Agent TypeScript SDK (available as a preview) removes the need for async generators and yield coordination. Each conversation turn becomes a separate `send()`/`stream()` call instead of a managed generator state.

V1 style (current — stable):

```
for await (const message of query({ prompt, options: { maxTurns: 20 } })) {
  if (message.type === "result") {
    finalReport = message.result;
  }
}
```

V2 style (preview):

```
import { createAgent } from "@anthropic-ai/claude-agent-sdk/v2";

const agent = createAgent({ maxTurns: 20 });
const result = await agent.send(prompt);
// No generator state to manage across turns
```

For new projects, the V2 style is simpler to reason about. Existing code on the V1 generator API continues to work — hold off on migrating until V2 reaches GA.

The Prompt Template

The prompt is where you shape what the agent does. Keep it specific — vague prompts produce generic output.

```
# Weekly Briefing - {{DATE}}

You are researching DevOps and platform engineering trends for a weekly briefing.

## Last Week's Report
{{LAST_REPORT}}

## Your Task

Research and write a briefing covering:

1. **New tools or releases** — Kubernetes, Terraform, Docker, or major cloud providers
2. **Incidents worth learning from** — public post-mortems published this week
3. **Practical tips** — one concrete technique a DevOps engineer can apply immediately
4. **Community discussion** — one interesting thread from Hacker News or Reddit

Rules:
- Do not repeat topics covered in last week's report (shown above)
- Each section should be 2-3 paragraphs, not bullet lists
- Cite sources with URLs
- Write for a senior engineer — skip basics

Save the final report to memory/weekly.md before finishing.
```

Memory: Avoiding Repetition

The file-based memory pattern is simple but effective. Before each run, the agent reads the previous report. The prompt explicitly instructs it to avoid repeating covered topics.

This is enough for a weekly cadence. You do not need a vector database or a sophisticated memory system — a single markdown file captures enough context to produce non-repetitive output across months of runs.

```
// The memory pattern in three lines
const lastReport = await readFile("memory/weekly.md", "utf-8").catch(() => "");
const prompt = `Previous report:\n${lastReport}\n\nDo not repeat these topics:\n${prompt}`;
// After the run:
await writeFile("memory/weekly.md", newReport);
```

Delivery via Telegram

Telegram is the easiest delivery mechanism — free, API-first, and instant on mobile.

```
async function sendToTelegram(report: string) {
  const { TELEGRAM_TOKEN, TELEGRAM_CHAT_ID } = process.env;

  // Telegram has a 4096 character limit per message
  const chunks = report.match(/.{1,4000}/gs) ?? [report];

  for (const chunk of chunks) {
    await fetch(`https://api.telegram.org/bot${TELEGRAM_TOKEN}/sendMessage`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        chat_id: TELEGRAM_CHAT_ID,
        text: chunk,
        parse_mode: "Markdown",
      }),
    });
  }
}
```

Get your chat ID by messaging `@userinfobot` on Telegram.

Scheduling with Cron

```
# Edit crontab
crontab -e

# Run every Sunday at 8 AM
0 8 * * 0  cd /home/user/my-assistant && bun run index.ts >> logs/assistant.log 2>&1
```

Create the logs directory first:

```
mkdir -p /home/user/my-assistant/logs
```

The Full Entry Point

```
import { runWeeklyBriefing } from "./agent";
import { sendToTelegram } from "./telegram";

async function main() {
  console.log(`[${new Date().toISOString()}] Starting weekly briefing`);

  try {
    const report = await runWeeklyBriefing();
    await sendToTelegram(report);
    console.log("Briefing delivered successfully");
  } catch (error) {
    console.error("Briefing failed:", error);
    process.exit(1);
  }
}

main();
```

Extending the Pattern

Once this works, the pattern extends naturally:

```
flowchart LR
    Base[Weekly briefing\nbase pattern] --> R1[Daily standup\nfrom git log]
    Base --> R2[Monthly cost\nreport from AWS]
    Base --> R3[Incident summary\nfrom PagerDuty]
    Base --> R4[Dependency audit\nfrom npm audit]
    Base --> R5[Team newsletter\nfrom Linear tickets]
```

Each variant changes the prompt template and the data source. The scheduling, memory, and delivery patterns stay the same.

The key shift in thinking: instead of asking Claude a question when you need an answer, design a workflow that delivers answers before you know you need them. The assistant does the work while you sleep and the briefing is waiting when you arrive.

When to Use Managed Agents Instead

The pattern above runs on infrastructure you control — a machine with Bun and a cron job. That is the right choice when you want full ownership of the execution environment.

If you want to skip the infrastructure entirely, **Claude Managed Agents** (launched April 2026) is the alternative. You POST a task to a REST endpoint, and Anthropic runs the agent loop in a secure sandboxed container that can stay alive for hours. With the **routines** feature, you can schedule recurring agent tasks without maintaining a cron job at all:

```
# Create a routine that runs your weekly briefing automatically
ant routines create \
  --schedule "0 8 * * 0" \
  --name "weekly-devops-brief" \
  --tools web_search,code_execution \
  "Research DevOps trends this week. Cover new tool releases, post-mortems, and one practical tip. Write
for a senior engineer."
```

The trade-off is control vs. convenience: the self-hosted Bun pattern gives you custom tool integrations, local file access, and full control over the runtime. Managed Agents gives you zero infrastructure but constrains you to the built-in tool set (web search, code execution, file

management).

For a weekly briefing that mainly needs web search and file writing, either works. If your briefing needs to pull from internal databases, private APIs, or local files on your machine, the self-hosted approach remains the right choice.

You Don't Need a Framework to Build an AI Assistant

Published 2026-04-26 | Source:
https://blog.devops-monk.com/2026/04/build-ai-assistant-without-framework/

200 lines of shell scripts, a cron job, and Claude Code headless mode are all you need for a powerful personal AI assistant. No frameworks, no servers, no complexity.

There is a tendency in the AI tooling space to reach for frameworks — LangChain, AutoGen, CrewAI, OpenClaw — the moment you want an AI that does more than answer one question at a time. Most of the time, that is the wrong move. The framework adds complexity, dependencies, and debugging surface area for problems that a few shell scripts and cron jobs solve perfectly well.

Claude Code's headless mode (`-p` flag) plus a markdown file for personality plus cron scheduling is a complete AI assistant stack. Here is how to build it.

The Architecture

```
flowchart TD
    Cron[Cron schedule] --> Script[Bash script]
    Script --> Template[Load prompt template\nfrom prompts/]
    Script --> Memory[Read memory files\nfrom memories/]
    Script --> Claude[claude -p with combined prompt]
    Claude --> Output[Write output to file]
    Output --> Notify[Send to Telegram or Slack]

    subgraph Assistant Directory
        SOUL[SOUL.md\nPersonality and principles]
        Memories[memories/\nPersistent context files]
        Prompts[prompts/\nTask templates]
        Scripts[scripts/\nCron-invoked shell scripts]
    end
```

No server. No message queue. No framework. The entire system is files and shell scripts.

The SOUL.md File

The first thing to create is a personality file. This defines how the assistant approaches its work — its tone, its priorities, its operating principles. Without this, outputs feel generic.

```
# Assistant: Devika

## Role
You are Devika, a research and briefing assistant for a DevOps engineering team.
Your job is to save engineers time by surfacing relevant information before they know they need it.

## Operating Principles
- Be specific and technical — this audience is senior engineers, not beginners
- Cite sources — do not make claims without a URL or reference
- Be concise — a tight 200-word section beats a rambling 500-word one
- Surface actionable insights — "here is something you can try today" is more valuable than "here is an
interesting trend"
- Do not repeat yourself across runs — check what was covered last time

## Tone
Professional but conversational. Not corporate. Not casual.
Write like a knowledgeable colleague who is good at summarising.

## Output Format
- Use Markdown headings
- Include a TL;DR at the top (3 bullet points maximum)
- Keep total length under 800 words
```

The Memory System

Memory is just markdown files in a directory. Each file captures a category of persistent context:

```
memories/
├── last_weekly.md          ← previous weekly briefing (auto-updated)
├── topics_covered.md       ← running list of covered topics
├── team_context.md         ← what this team works on (static)
└── feedback.md             ← lessons from past runs (manually updated)
```

The team_context.md is static — write it once:

```
# Team Context

This team runs a platform engineering function at a mid-size SaaS company.
Current focus areas: Kubernetes migration, Terraform module standardisation,
developer experience improvements.

Tools in use: AWS, Kubernetes (EKS), Terraform, GitHub Actions, PagerDuty, Datadog.
Stack: Node.js microservices, PostgreSQL, Redis.

Do not brief on topics already in regular team discussions.
```

Prompt Templates

Templates are markdown files with `{{VARIABLE}}` placeholders. Each task gets its own template:

```
# Weekly Engineering Briefing - {{DATE}}

Read the following context files before starting:
- Team context: {{TEAM_CONTEXT}}
- Last briefing: {{LAST_BRIEFING}}
- Topics already covered: {{TOPICS_COVERED}}

## Your Task

Research and write a briefing for the platform engineering team covering:

1. New releases in our toolchain (Kubernetes, Terraform, AWS services we use)
2. One notable incident post-mortem published this week — what happened and what we can learn
3. One practical technique or tool worth trying — include a concrete example
4. One community discussion worth reading — with a brief summary and why it matters

Constraints:
- Do not cover topics listed in {{TOPICS_COVERED}}
- Maximum 800 words total
- All claims must have sources
- Save a summary of topics covered to memories/topics_covered.md before finishing
```

The Shell Script

The script loads the soul, memory, and template — assembles them into a complete prompt — and runs `claude -p`:

```
#!/bin/bash
# scripts/weekly-brief.sh

set -e

BASE_DIR="$(cd "$(dirname "$0")/.." && pwd)"
DATE=$(date +%Y-%m-%d)
LOG="$BASE_DIR/logs/$(date +%Y-%m-%d).log"

log() { echo "[$(date +%H:%M:%S)] $1" | tee -a "$LOG"; }

log "Starting weekly briefing"

# Load memory files
TEAM_CONTEXT=$(cat "$BASE_DIR/memories/team_context.md" 2>/dev/null || echo "")
LAST_BRIEFING=$(cat "$BASE_DIR/memories/last_weekly.md" 2>/dev/null || echo "No previous briefing.")
TOPICS_COVERED=$(cat "$BASE_DIR/memories/topics_covered.md" 2>/dev/null || echo "None yet.")

# Assemble prompt from template
PROMPT=$(cat "$BASE_DIR/prompts/weekly-brief.md")
PROMPT="${PROMPT//\{\{DATE\}\}/$DATE}"
PROMPT="${PROMPT//\{\{TEAM_CONTEXT\}\}/$TEAM_CONTEXT}"
PROMPT="${PROMPT//\{\{LAST_BRIEFING\}\}/$LAST_BRIEFING}"
PROMPT="${PROMPT//\{\{TOPICS_COVERED\}\}/$TOPICS_COVERED}"

# Add the soul
FULL_PROMPT="$(cat "$BASE_DIR/SOUL.md")\n\n---\n\n$PROMPT"

# Run Claude
log "Running Claude"
RESULT=$(claude -p \
  --dangerously-skip-permissions \
  -d "$BASE_DIR" \
  "$FULL_PROMPT")

# Save as this week's memory
echo "$RESULT" > "$BASE_DIR/memories/last_weekly.md"
log "Report saved to memories/last_weekly.md"

# Send to Telegram
if [ -n "$TELEGRAM_TOKEN" ] && [ -n "$TELEGRAM_CHAT_ID" ]; then
  curl -s -X POST \
    "https://api.telegram.org/bot${TELEGRAM_TOKEN}/sendMessage" \
    -d "chat_id=${TELEGRAM_CHAT_ID}" \
    -d "text=${RESULT:0:4000}" \
    -d "parse_mode=Markdown"
  log "Delivered to Telegram"
fi

log "Done"
```

Scheduling

```
# Create the log directory
mkdir -p /home/user/assistant/logs

# Edit crontab
crontab -e

# Weekly on Sunday at 8 AM
0 8 * * 0   /home/user/assistant/scripts/weekly-brief.sh

# Daily standup at 9 AM weekdays
0 9 * * 1-5 /home/user/assistant/scripts/daily-standup.sh
```

Why This Beats a Framework

```
flowchart LR
    subgraph Framework[Framework approach]
        F1[Install 50 dependencies]
        F2[Learn framework API]
        F3[Debug framework behaviour]
        F4[Handle framework updates]
        F5[Vendor lock-in]
    end
    subgraph Shell[Shell approach]
        S1[3 shell scripts]
        S2[Standard Unix tools]
        S3[Transparent and auditable]
        S4[No dependencies to update]
        S5[Runs anywhere with bash]
    end
```

Security advantage: Frameworks that accept inbound messages (Slack DM, email) are vulnerable to prompt injection — someone sends a message that tricks the assistant into doing something harmful. The shell script approach is outbound-only: cron triggers the script, which reads local files, which you control. No external input reaches the assistant during a run.

Transparency: Every step is a file you can read. The prompt is a markdown file. The memory is a markdown file. The output is a markdown file. There is nothing hidden in a framework's internal state.

Portability: These scripts run on any machine with Claude Code installed. No Docker image, no framework version pinning, no cloud dependency.

Extending It

Once the weekly briefing works, the pattern extends to any scheduled task:

```
# Daily: What merged yesterday?
0 9 * * 1-5 scripts/daily-git-summary.sh

# After deploys: Write a deploy summary
# Triggered by CI webhook
scripts/deploy-summary.sh

# Monthly: Infrastructure cost report
0 8 1 * *   scripts/monthly-costs.sh
```

Each script follows the same pattern: load context, build prompt, run `claude -p`, save output, notify. The soul and memory system stay shared across all scripts.

The most powerful thing about this approach is that you start with something that works in an afternoon and grow it incrementally. No framework design decisions to make upfront, no architecture to commit to. Just scripts that do one thing well.